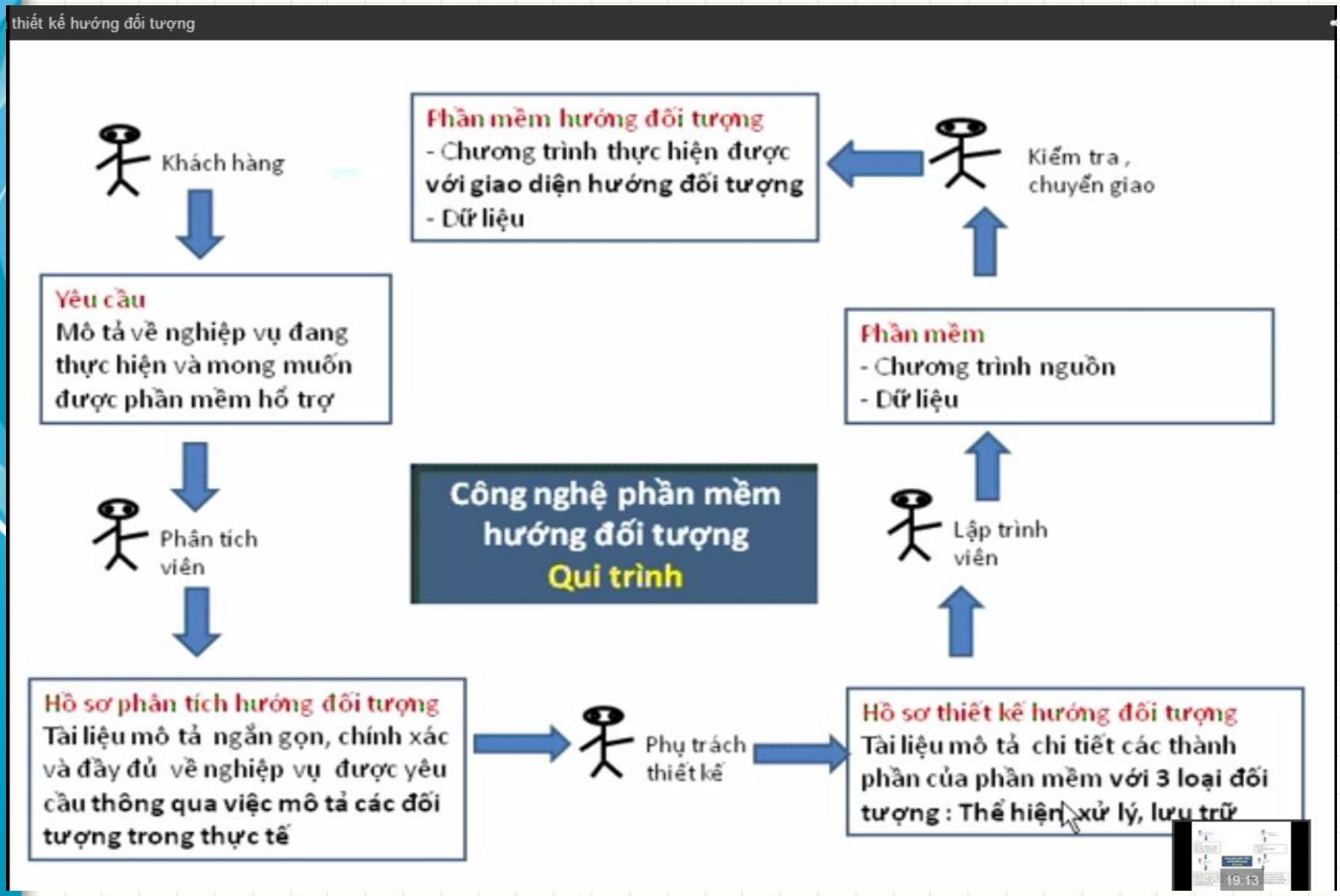
PHÂN TÍCH THIẾT KẾ HƯỚNG ĐỐI TƯỢNG UML

Phan Nguyễn Ý Nhi
KHOA CNTT TT
TRƯỜNG CĐCN HUẾ

CH1. Tổng quan về OOAD và UML

1. Giới thiệu về OOAD

- Object Oriented Analysis and Design: phân tích thiết kế hướng đối tượng
  - "Phân tích" và "Thiết kế" cần được coi trọng
  - Cần thiết lập một cơ chế hiệu quả để nắm bắt yêu cầu phân tích thiết kế
- Mục đích
  - Cung cấp cho khách hàng, người dùng, kỹ sư phân tích thiết kế nhiều cái nhìn khác nhau về một hệ thống
- OOAD được chia làm hai giai đoạn
  - OOA
  - OOD


Khách hàng
Yêu cầu
Mô tả về nghiệp vụ đang thực hiện và mong muốn được phần mềm hỗ trợ
Phân tích viên
Hồ sơ phân tích hướng đối tượng
Tài liệu mô tả ngắn gọn, chính xác và đầy đủ về nghiệp vụ được yêu cầu thông qua việc mô tả các đối tượng trong thực tế
Phụ trách thiết kế
Hồ sơ thiết kế hướng đối tượng
Tài liệu mô tả chi tiết các thành phần của phần mềm với 3 loại đối tượng : Thể hiện, xử lý, lưu trữ
Lập trình viên
Phần mềm
- Chương trình nguồn
- Dữ liệu
Kiểm tra , chuyển giao
Phần mềm hướng đối tượng
- Chương trình thực hiện được với giao diện hướng đối tượng
- Dữ liệu
Công nghệ phần mềm hướng đối tượng
Qui trình
19:13

2. Giới thiệu về UML

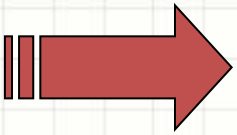


- Unified Modeling Language: ngôn ngữ mô hình hóa thống nhất
- UML là ngôn ngữ mô hình hoá, ngôn ngữ đặc tả và ngôn ngữ xây dựng mô hình trong quá trình phát triển phần mềm, đặc biệt là trong phân tích và thiết kế hệ thống hướng đối tượng.
- UML là ngôn ngữ hình thức, thống nhất và chuẩn hoá mô hình hệ thống một cách trực quan.

Ví dụ về mô hình



Thế giới thực

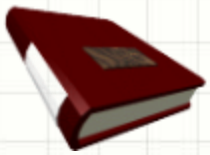




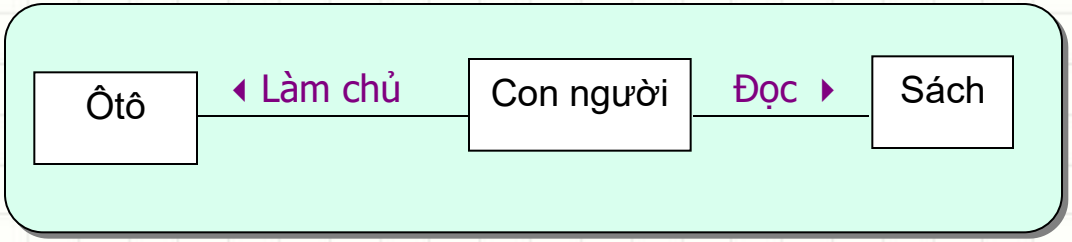
Mô hình: Quả địa cầu học sinh





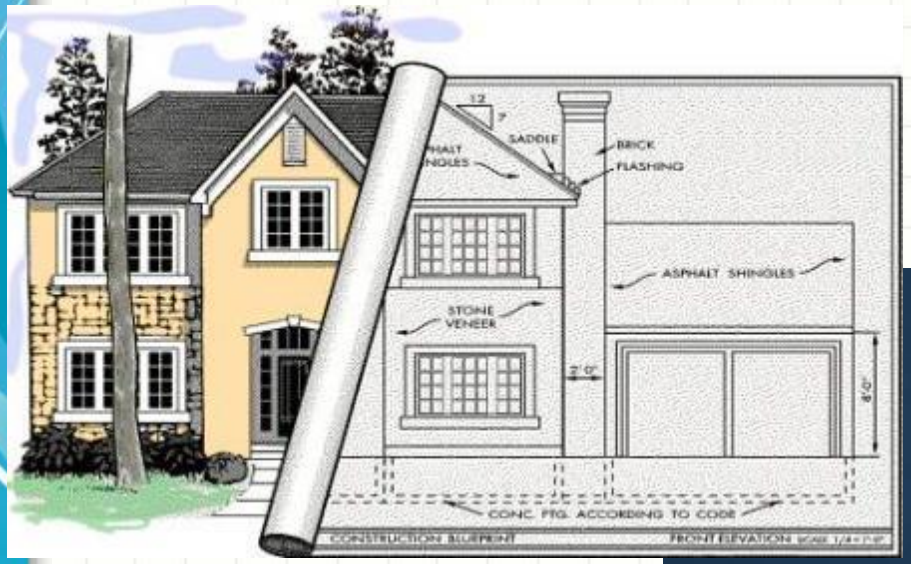


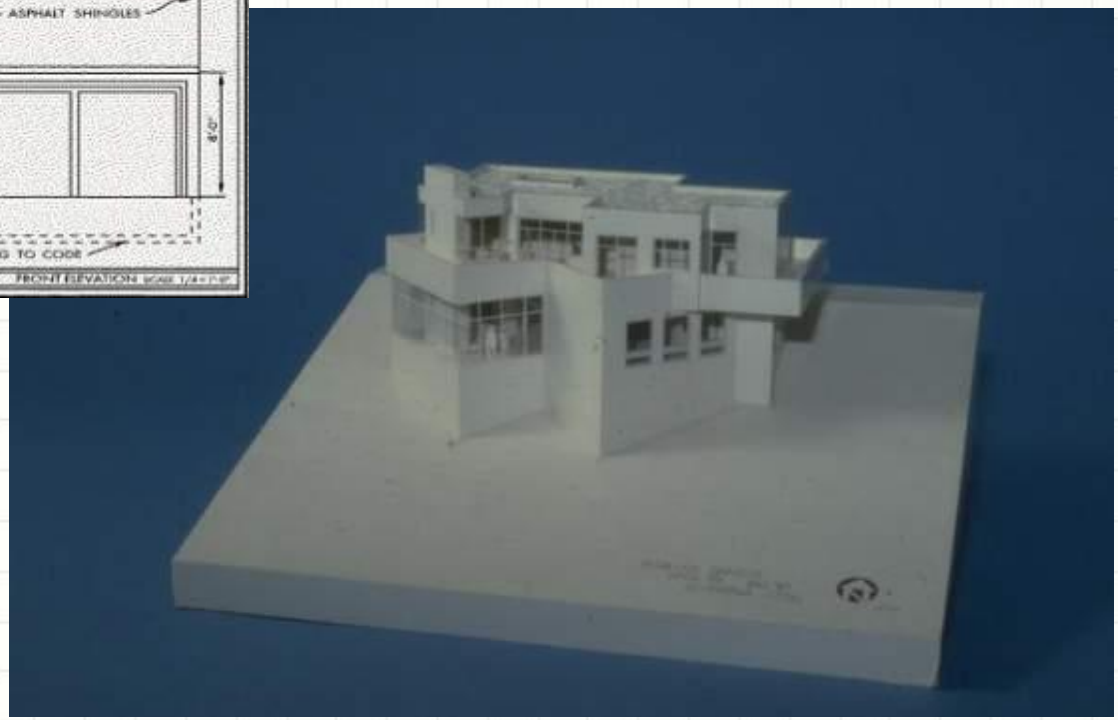
Thế giới thực



Mô hình

Mô hình hóa ngôi nhà





Tại sao cần mô hình hóa?

- Một mô hình là sự đơn giản hóa thực tế, nó cho phép hiểu rõ hơn hệ thống cần phát triển
- Ngoài ra, nó còn cho phép:
  - Hiển thị hệ thống như nó vốn có hoặc nó cần đạt tới
  - Kiểm chứng hệ thống bởi khách hàng
  - Cung cấp những chỉ dẫn để xây dựng hệ thống
  - Tài liệu hóa hệ thống

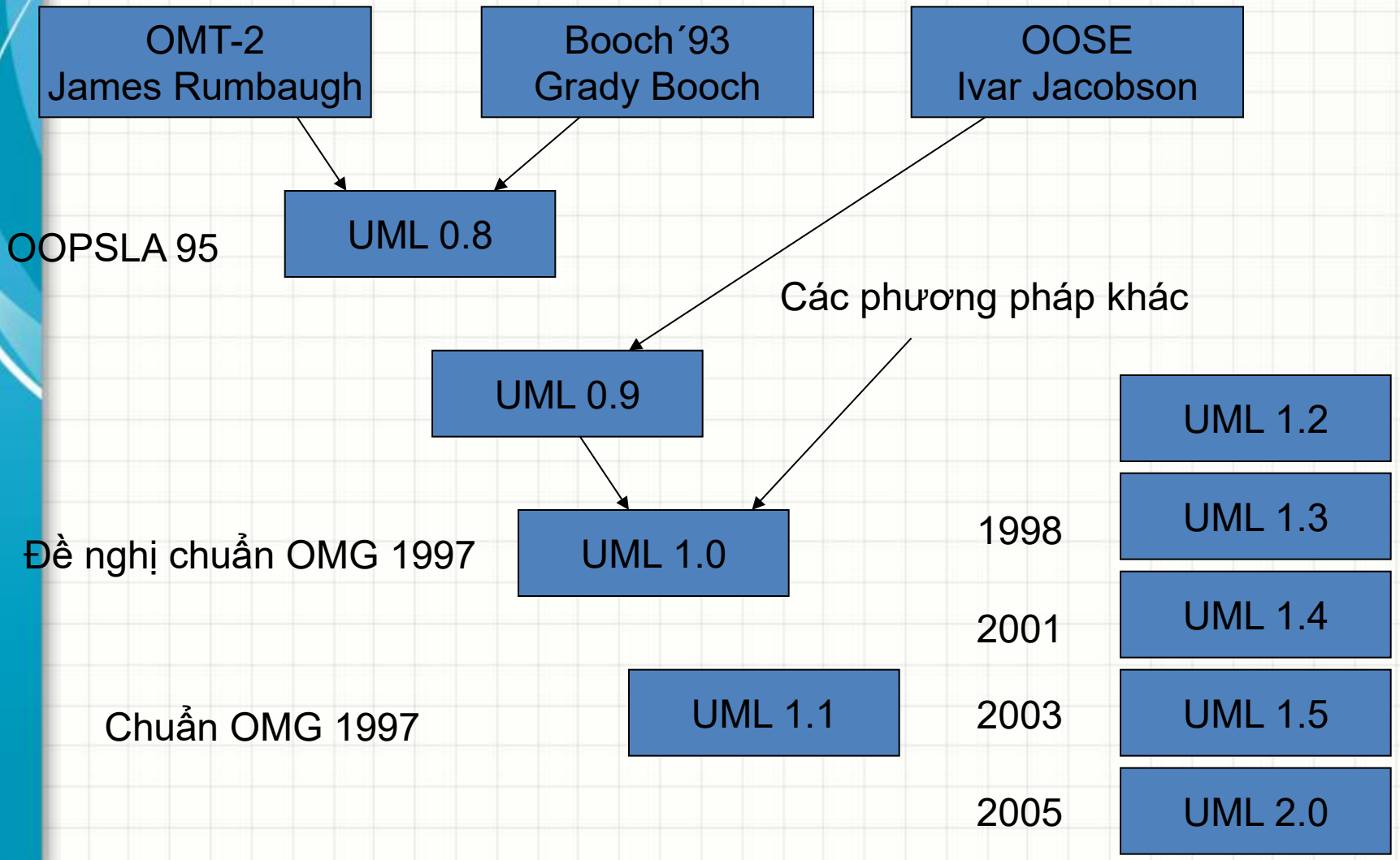
Các nguyên tắc của mô hình hóa

- Việc chọn mô hình nào để tạo lập có ảnh hưởng sâu sắc đến cách giải quyết vấn đề và cách hình thành các giải pháp
- Mỗi mô hình biểu diễn hệ thống với mức độ chính xác khác nhau
- Mô hình tốt nhất phải là mô hình phù hợp với thế giới thực
- Không mô hình nào là đầy đủ. Mỗi hệ thống thường được tiếp cận thông qua tập mô hình gần như độc lập nhau.

Lợi ích của mô hình hóa hướng đối tượng?

- Tăng tính độc lập của mô hình với các chức năng yêu cầu
- Có thể thay đổi hoặc thêm bớt các chức năng mà mô hình đối tượng không thay đổi
- Gần hơn với thế giới thực

Lịch sử phát triển



Mục đích của UML

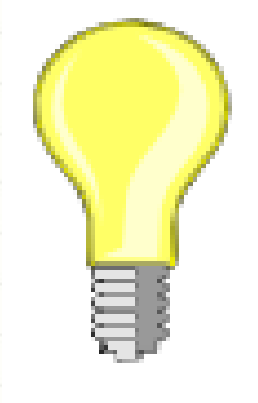
- Mô hình được các hệ thống và sử dụng được tất cả các khái niệm hướng đối tượng một cách thống nhất.
- Cho phép đặc tả, hỗ trợ để đặc tả tường minh mối quan hệ giữa các khái niệm cơ bản trong hệ thống, đồng thời mô tả được mọi trạng thái hoạt động của hệ thống đối tượng.
- Tận dụng được những khả năng sử dụng lại và kế thừa ở phạm vi diện rộng để xây dựng được những hệ thống phức tạp và nhạy cảm
- Tạo ra những ngôn ngữ mô hình hoá sử dụng được cho cả người lẫn máy tính.

UML là một ngôn ngữ

- UML không phải là một phương pháp
- UML là một ngôn ngữ mô hình hóa đối tượng
- UML đã được công nhận bởi tất cả các phương pháp đối tượng
- UML được sử dụng chung trong cộng đồng CNTT, đó là một chuẩn.

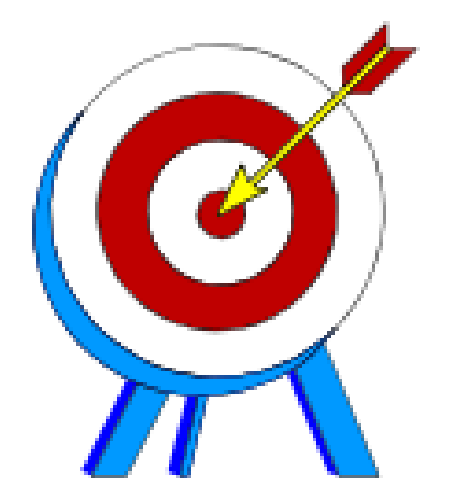
UML là ngôn ngữ trực quan

- UML là ngôn ngữ thống nhất, trực quan, giúp công việc được xử lý nhất quán, giảm thiểu lỗi xảy ra
- Mỗi ký pháp đồ họa mang một ngữ nghĩa



UML là ngôn ngữ đặc tả

- UML xây xựng các mô hình chính xác, rõ ràng và đầy đủ

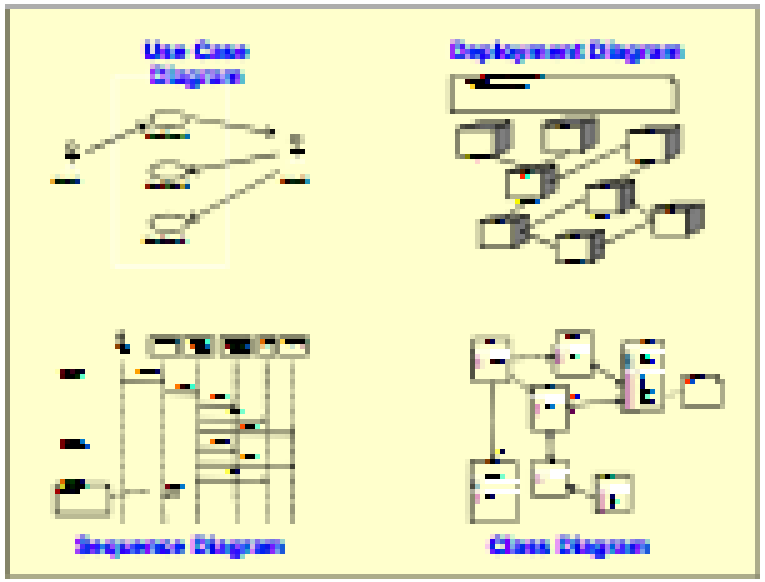


UML để xây dựng hệ thống

- Các mô hình UML có thể kết nối với rất nhiều ngôn ngữ lập trình
- Các lớp, các quan hệ có thể xây dựng một cách tự động

UML là ngôn ngữ để tài liệu hóa

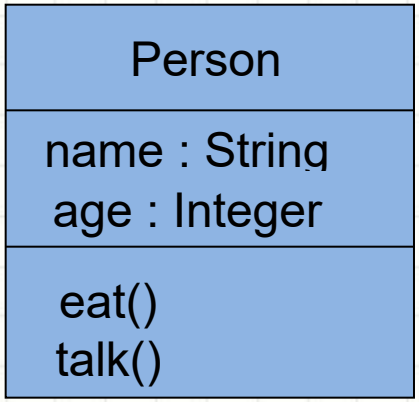
- Các biểu đồ khác nhau, các ghi chú, ràng buộc được giới thiệu trong tài liệu



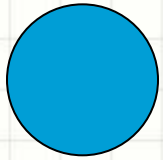
UML và các lĩnh vực được sử dụng

- Hệ thống thông tin các doanh nghiệp
- Ngân hàng và các dịch vụ tài chính
- Viễn thông
- Giao thông
- Quân sự và hàng không
- Khoa học
- Các ứng dụng phân tán trên web

Các thực thể cấu trúc



Class



Comparable

Interface



Use case

Các thực thể hành vi



Ký pháp quan hệ



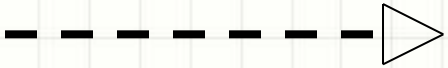
Dependence



Association



Inheritance

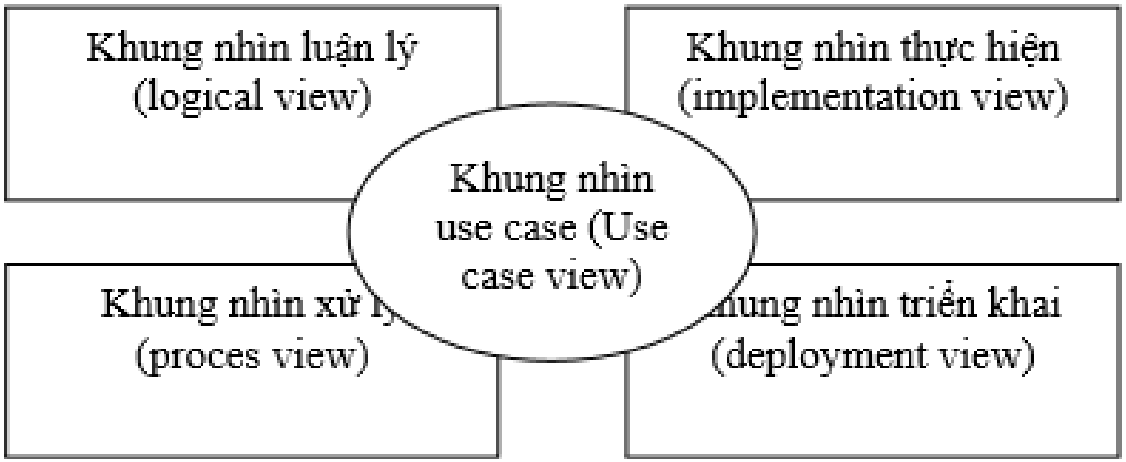


Realisation

4+1 cách nhìn một hệ thống

Người dùng
Chức năng

Lập trình viên
Quản trị phần mềm



Quản trị viên tích hợp hệ thống
Hiệu năng
Tính co giản
Thông lượng

Thiết kế viên hệ thống
Hình thái hệ thống
Chuyển giao, cài đặt
Truyền thông

Khung nhìn ca sử dụng – Use Case View

- Nhìn hệ thống bởi những người dùng cuối
- Nhóm các hành vi của hệ thống theo
  - Độ ưu tiên: cực kỳ quan trọng
  - Các mục đích khác về chiến thuật và ràng buộc
- Sử dụng biểu đồ UC để minh họa

Khung nhìn logic

- Phân rã hệ thống theo hướng đối tượng
  - Xác định các lớp và đối tượng
  - Nhóm vào các gói
  - Kết nối bằng các quan hệ
  - Sự trừu tượng, đa hình, đồng nhất
  - Thực hiện các kịch bản của các ca sử dụng
- Được thể hiện bằng sơ đồ lớp, sơ đồ đối tượng, sơ đồ mô tả các gói, hệ thống con

Khung nhìn xử lý

- Mô tả khía cạnh xảy ra đồng thời của hệ thống thời gian thực
- Khung nhìn tập trung vào sự đồng hành, song song, khởi động và đóng hệ thống
- Khả năng chịu hư hỏng và sự phân tán các đối tượng

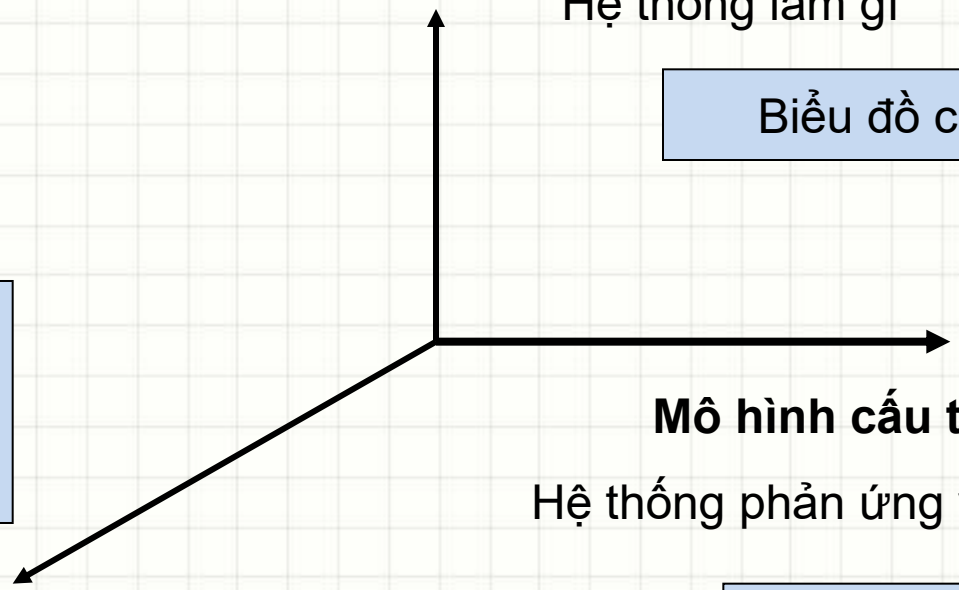
Khung nhìn thực thi

- Phân rã theo module
- Nhóm thành các module bằng các gói
- Tổ chức thành các hệ thống con theo định mức để:
  - Tăng độ chắc chắn
  - Giảm sự kết dính và nhìn thấy
- Các đặc điểm:
  - Dễ dàng phát triển
  - Tiềm năng tái sử dụng
  - Quản lý cấu hình
- Được thể hiện bằng biểu đồ thành phần

Khung nhìn triển khai

- Phân rã theo nút thực hiện
- Vai trò của một nút
- Liên quan giữa các nút
- Thông tin trên các đặc điểm sau:
  - Hiệu năng, tính sẵn sàng
  - Cài đặt, bảo trì
- Sử dụng biểu đồ triển khai để mô tả

3 thành phần của mô hình hóa



Mô hình chức năng

Hệ thống làm gì

Biểu đồ ca sử dụng

Biểu đồ tuần tự
Biểu đồ tương tác
Biểu đồ trạng thái
Biểu đồ hoạt động

Mô hình cấu trúc

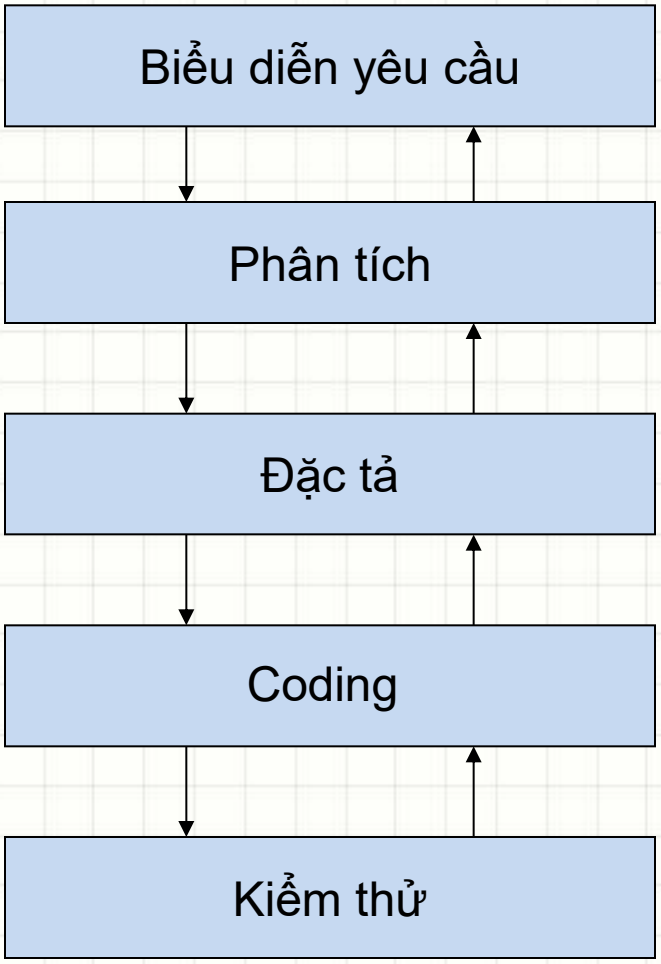
Hệ thống phản ứng với cái gì

Biểu đồ lớp và đối tượng

Mô hình thời gian

Thứ tự giữa các hành động trong hệ thống

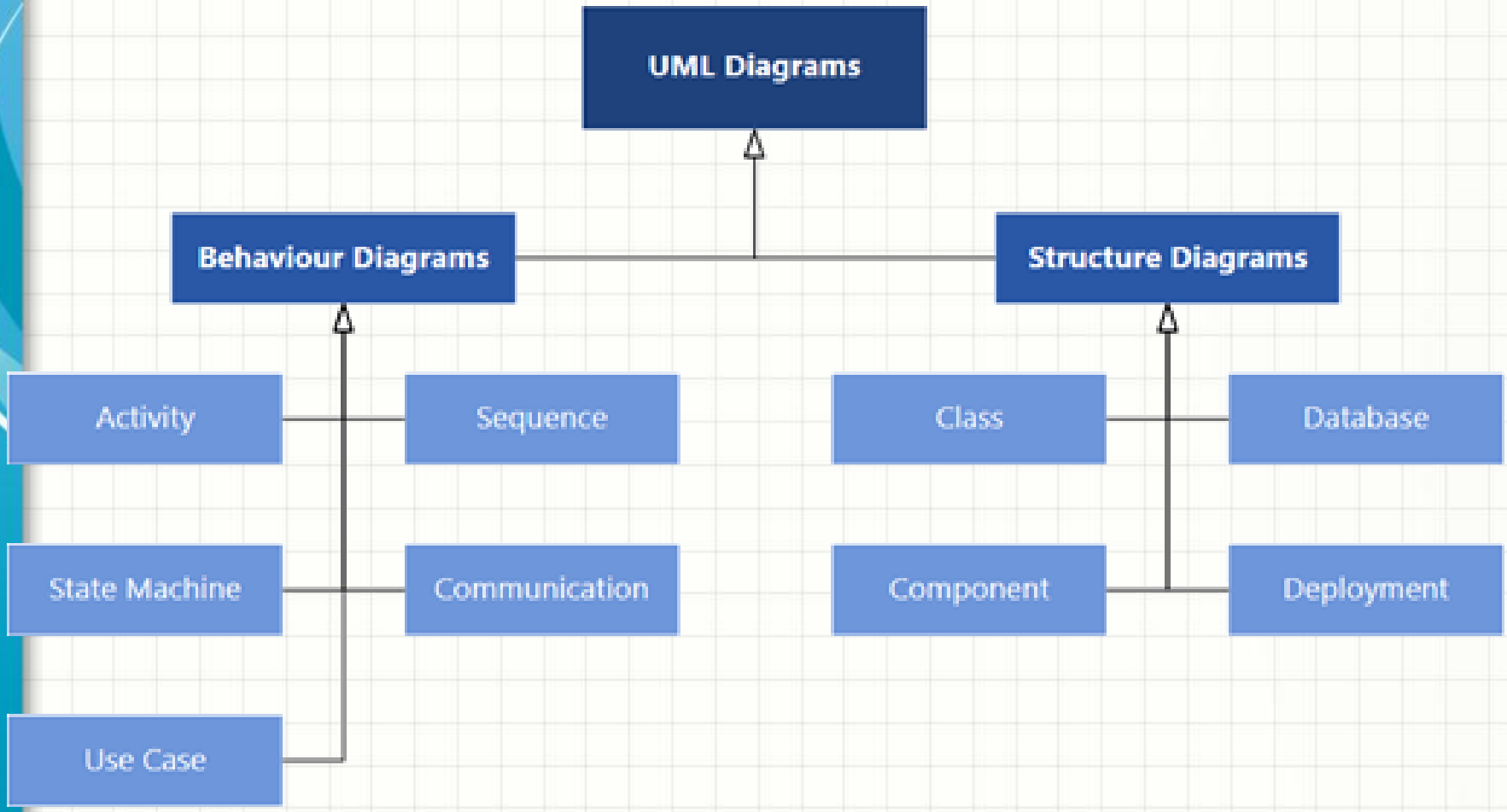
Các giai đoạn của mô hình hóa



Thí dụ ứng dụng UML

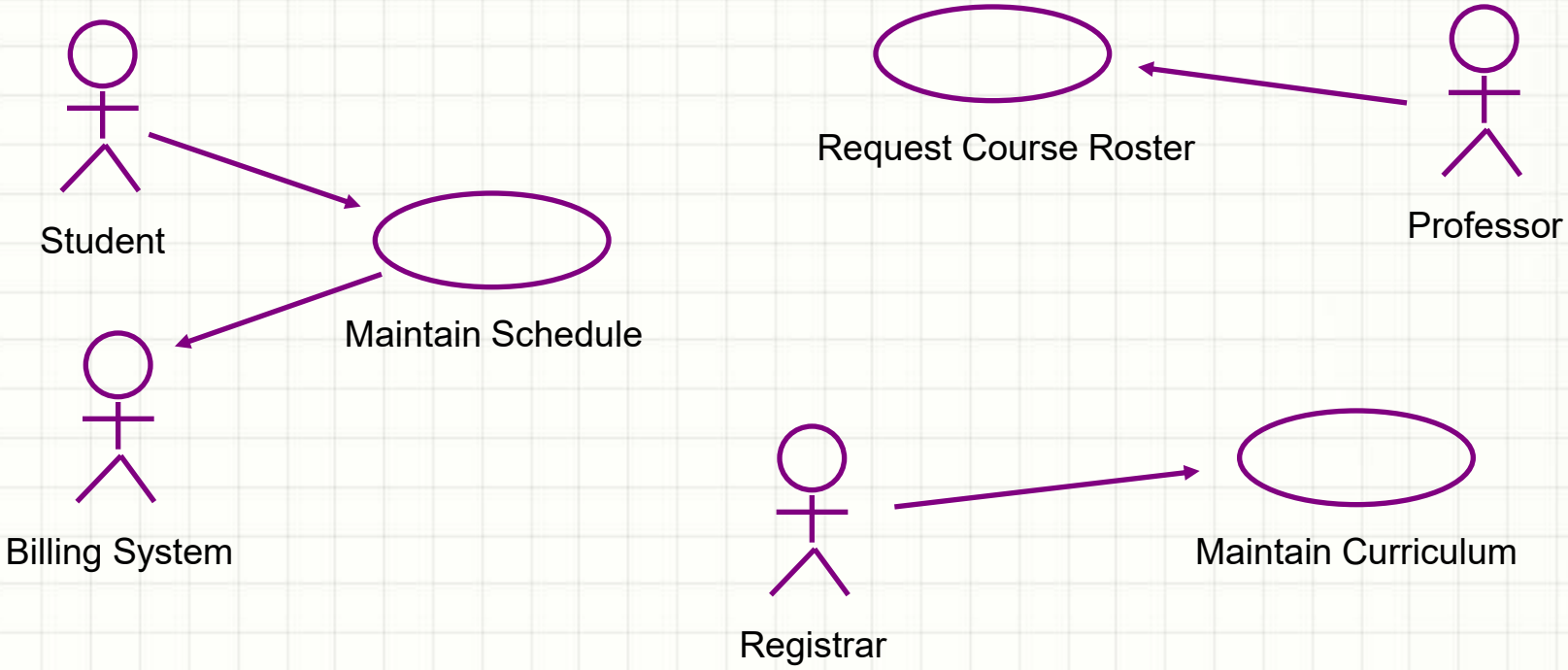
- Một trường đại học thực hiện tin học hóa hệ thống đăng ký học và dạy:
  - Giáo vụ (Registrar) lập chương trình giảng dạy (curriculum) cho học kỳ
  - Sinh viên (Student) chọn 4 môn học chính và 2 môn dự bị
  - Khi sinh viên đăng ký học thì hệ thống thanh toán (billing system) in hóa đơn học phí cho sinh viên
  - Sinh viên có thể sử dụng hệ thống để bổ sung/loại bỏ môn học sau khi đã đăng ký (trong khoảng thời gian)
  - Giáo sư (Professors) sử dụng hệ thống để xem bảng phân công dạy học (course rosters)
  - Người sử dụng hệ thống đăng ký được cấp passwords để vào máy

9 biểu đồ của UML



Use case Diagram

- Biểu đồ ca sử dụng (Use case diagrams) được dùng để hiển thị quan hệ giữa tác nhân và các use cases

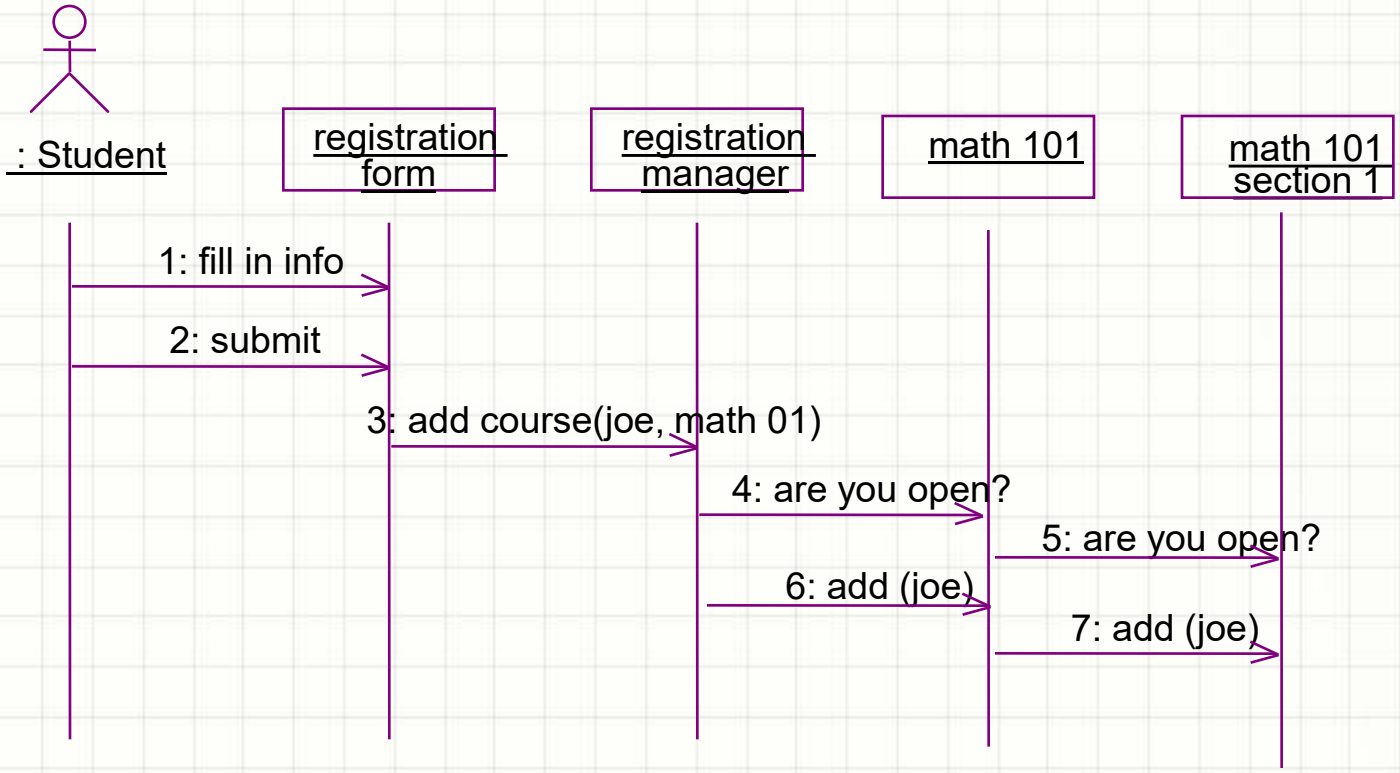


Use case Diagram

- Xác định các chức năng theo nhìn nhận của người sử dụng
- Xây dựng ở giai đoạn ban đầu
- Mục đích
  - Xác định ngữ cảnh của hệ thống
  - Nắm bắt các yêu cầu của hệ thống
  - Kiểm chứng kiến trúc hệ thống
  - Hướng dẫn thực thi và sinh test cases
- Phát triển bởi người phân tích và chuyên gia trong lĩnh vực

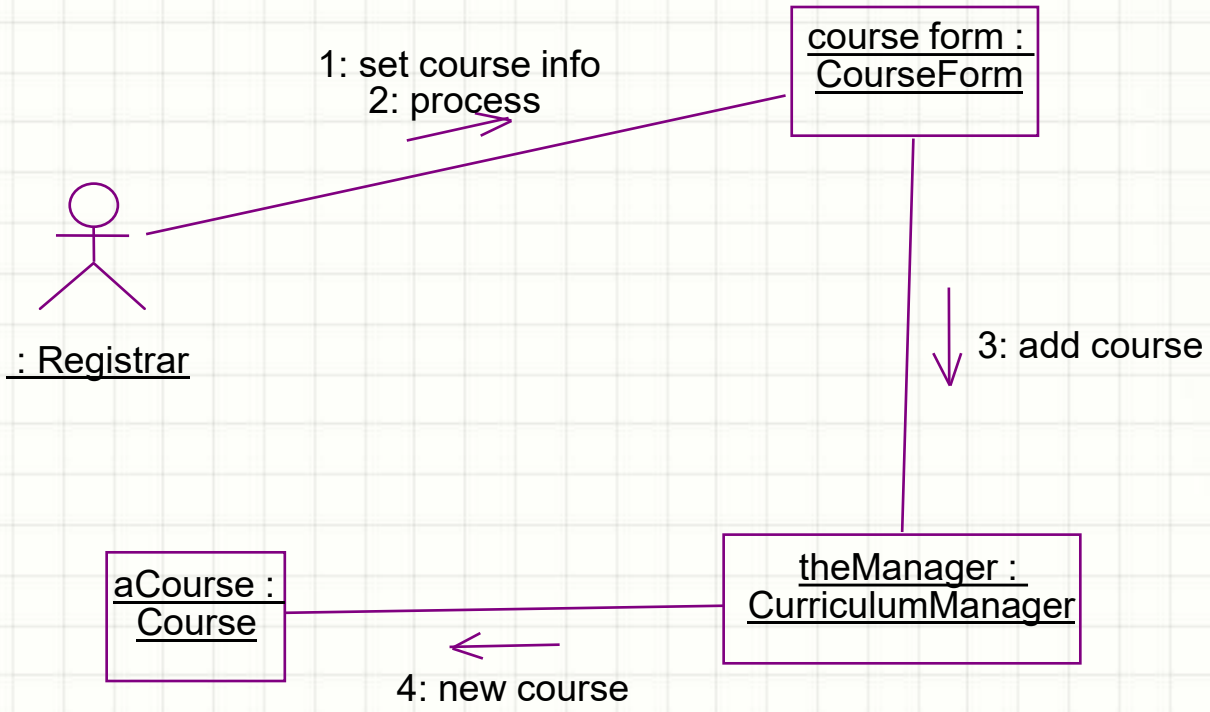
Sequence Diagram

- Biểu đồ tuần tự (sequence diagram) biểu diễn sự tương tác giữa các đối tượng theo sự sắp xếp về thời gian



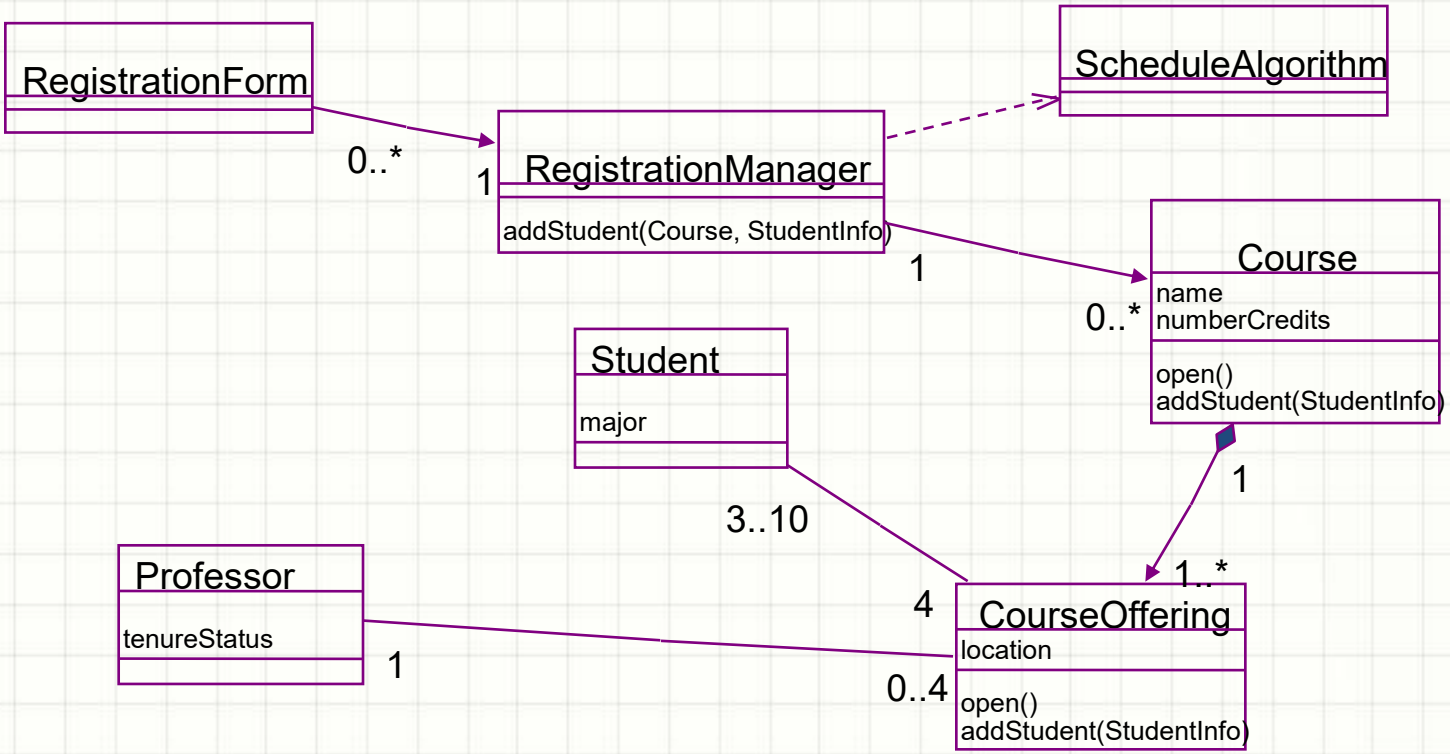
Collaboration Diagram

- Biểu đồ cộng tác (collaboration diagram) hiển thị sự tương tác giữa các đối tượng



Class Diagram

- Biểu đồ lớp (class diagram) biểu diễn sự tồn tại của các lớp và quạn hệ giữa chúng

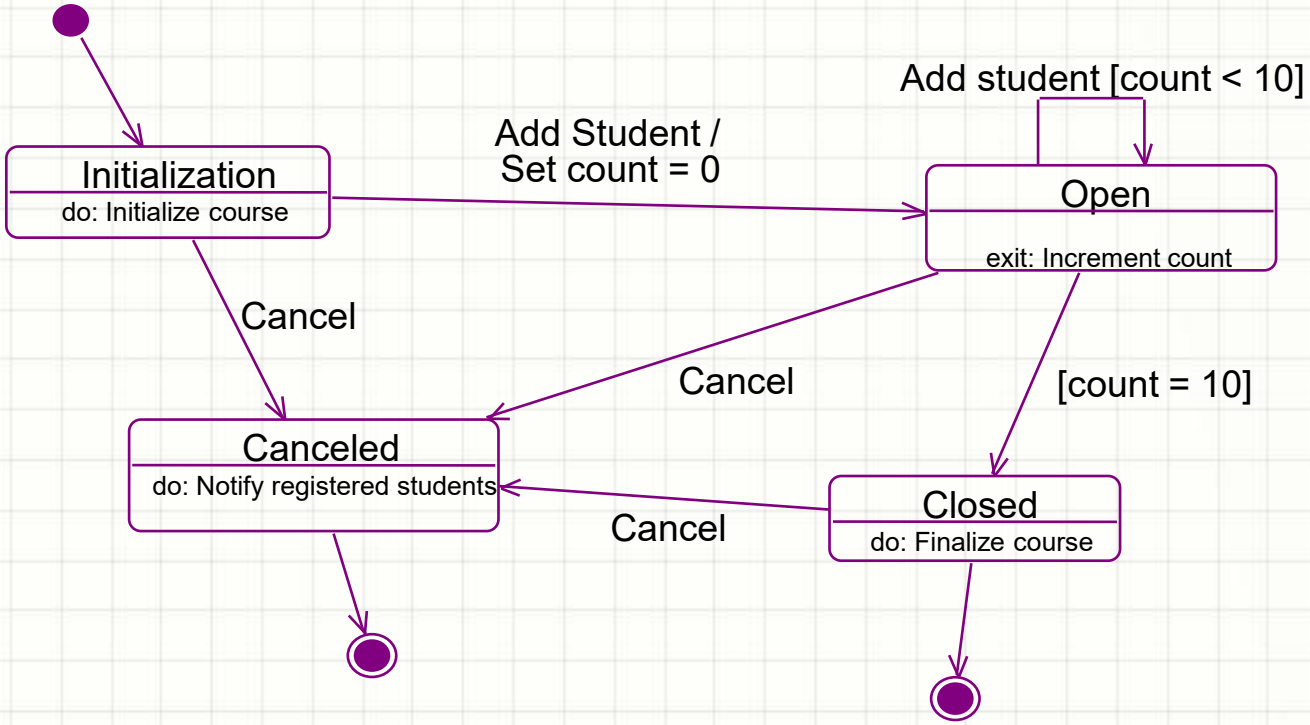


Object Diagram

- Biểu diễn thực thể và liên kết
- Được xây dựng ở giai đoạn phân tích và thiết kế
- Mục đích
  - Minh họa cấu trúc dữ liệu/đối tượng
  - Đặc tả snapshots

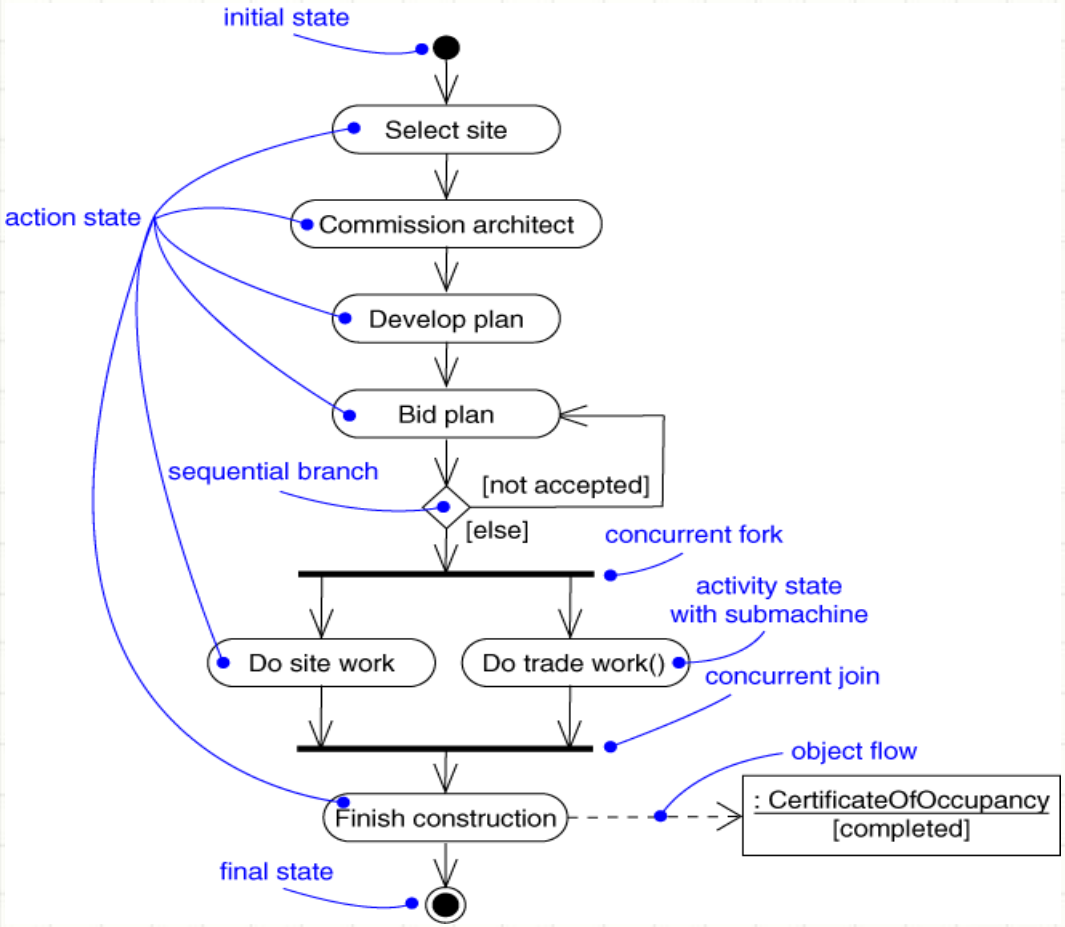
State Transition Diagram

- Biểu đồ chuyển trạng thái (State transition diagrams) dùng để biểu diễn sự chuyển đổi giữa các trạng thái trong đối tượng



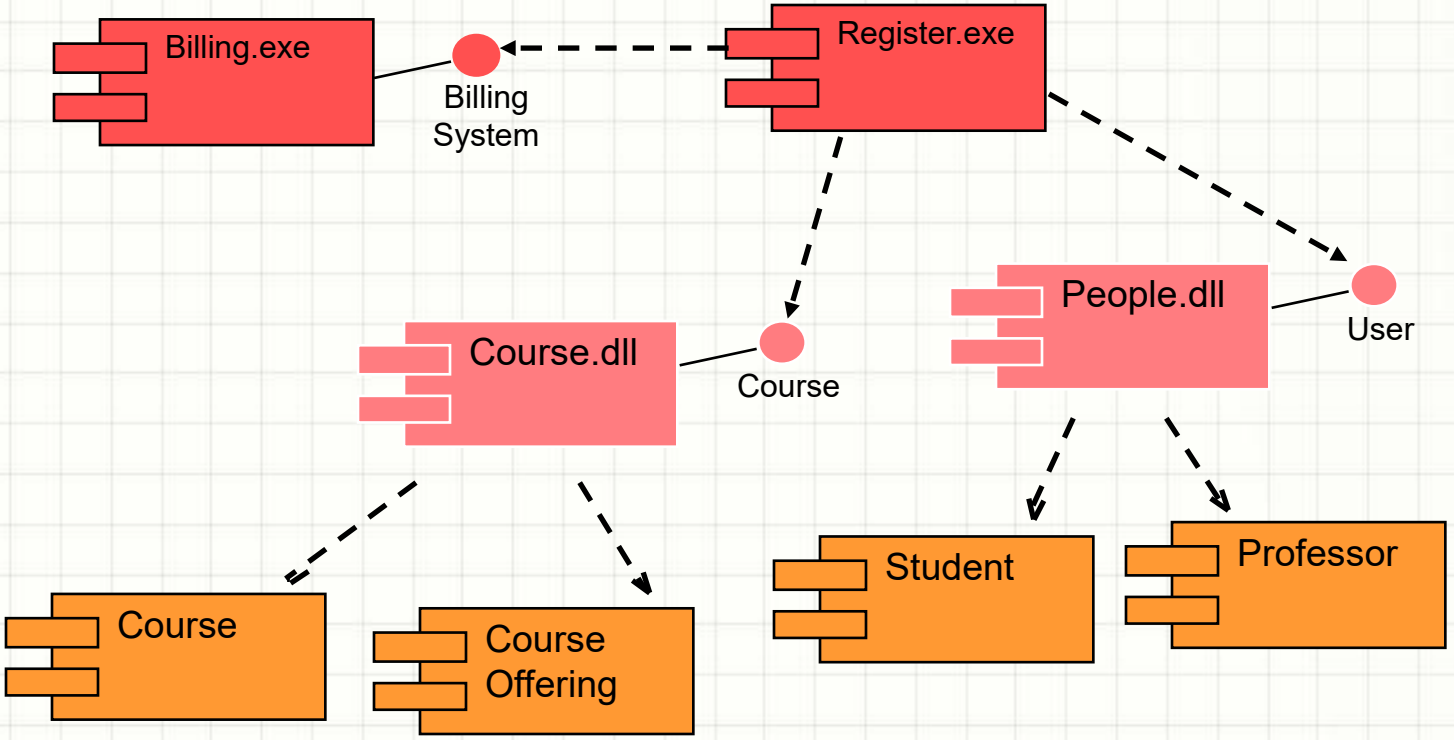
Activity Diagram

- Biểu diễn các luồng công việc
- Hướng hoạt động



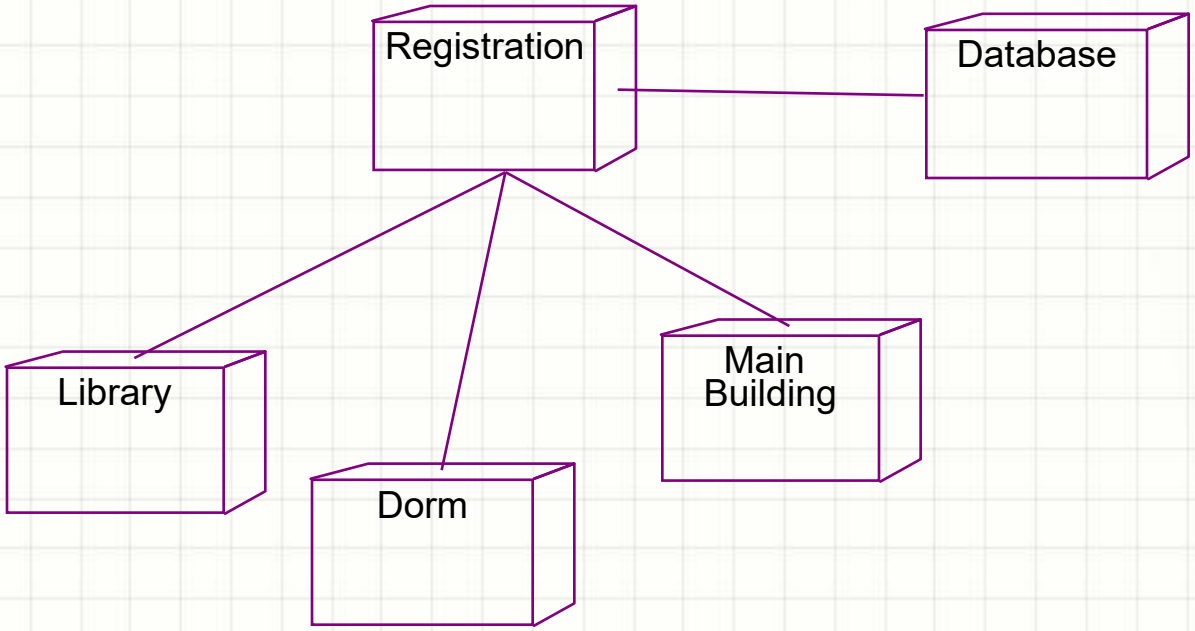
Component Diagram

- Biểu đồ thành phần (Component diagrams) biểu diễn sự tổ chức và phụ thuộc giữa các thành phần phần mềm

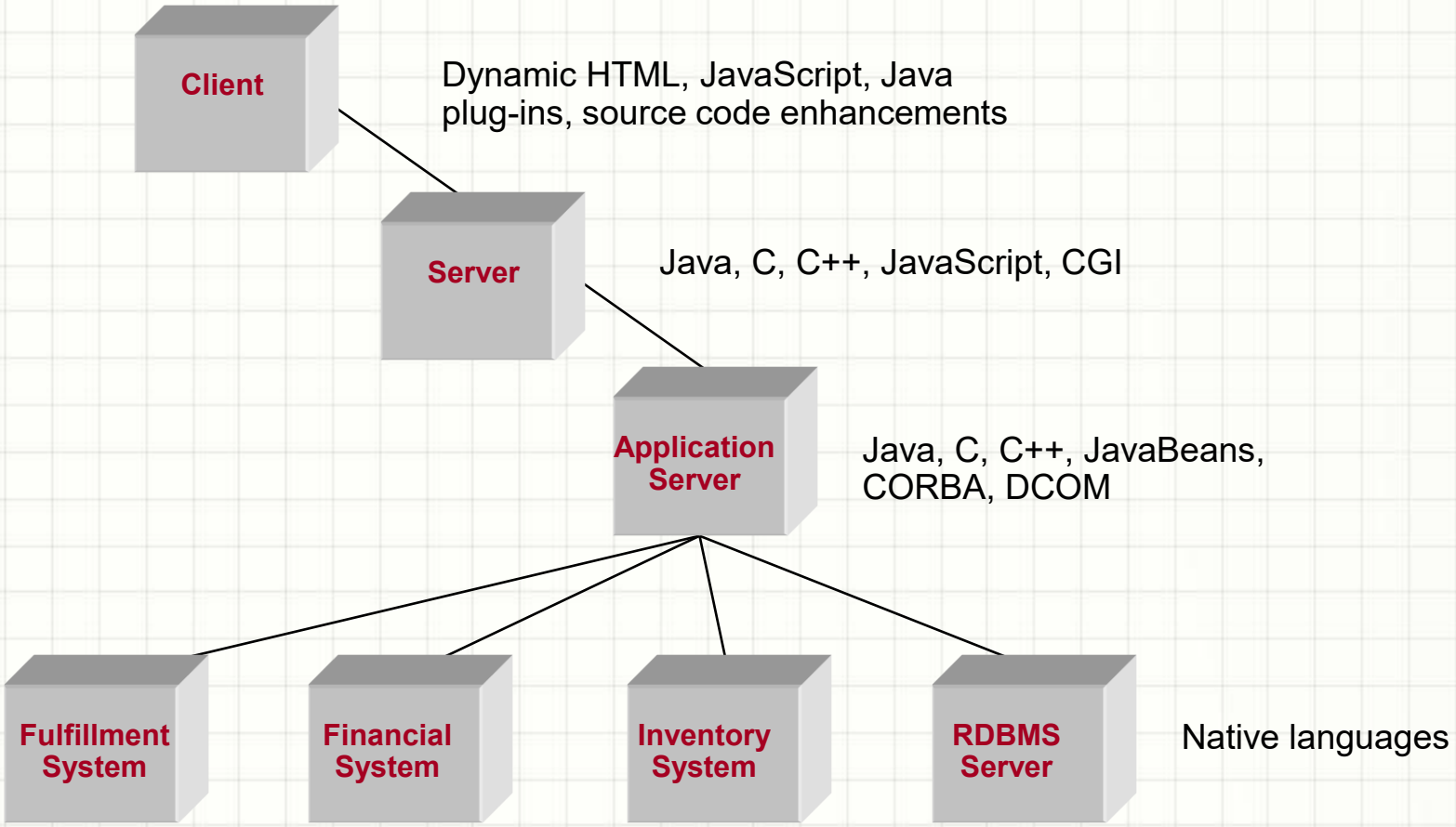


Deployment Diagram

- Biểu đồ triển khai (deployment diagram) biểu diễn cấu hình của các phần tử thực hiện tại run-time và các tiến trình phần mềm ở trong nó



Deployment Diagram

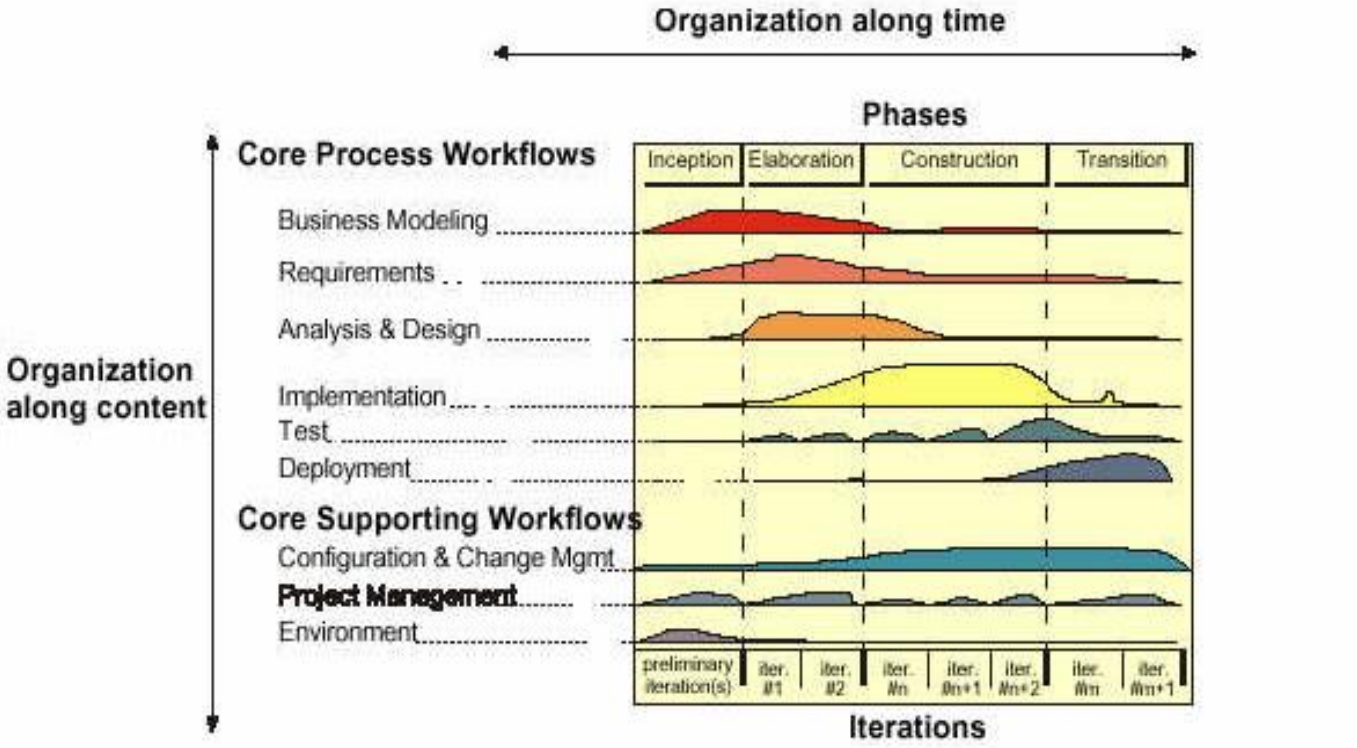


3. Giới thiệu về RUP

- ***RUP là gì?***
  - Là qui trình CNPM được phát triển bởi hãng Rational
  - RUP hỗ trợ các hoạt động giữa các nhóm, phân chia công việc cho từng thành viên trong từng giai đoạn khác nhau của quá trình phát triển phần mềm
  - RUP sử dụng hệ thống ký hiệu trực quan của UML
  - RUP được phát triển song song với UML

Các đặc điểm của RUP

- Là một qui trình CNPM hoàn chỉnh
- Là một sản phẩm tiến trình
- Hỗ trợ tăng năng suất làm việc nhóm
- Tạo, duy trì, quản lý các loại mô hình
- Có hướng sử dụng UML
- Được hỗ trợ bởi nhiều công cụ phát triển PM
- Là tiến trình có thể tuỳ biến


Organization along time
Phases
Inception
Elaboration
Construction
Transition
Core Process Workflows
Business Modeling
Requirements
Analysis & Design
Implementation
Test
Deployment
Core Supporting Workflows
Configuration & Change Mgmt
Project Management
Environment
Organization along content
preliminary iteration(s)
iter. #1
iter. #2
iter. #n
iter. #n+1
iter. #n+2
iter. #m
iter. #m+1
Iterations

Rational Rose

- Rose is available in three editions:
  - Rose Modeler – no language support
  - Rose Professional – support for 1 language
  - Rose Enterprise – supports multiple languages including (VC++, VB, Java, CORBA and XML)
- Why should we use Rational Rose?
  - Common standard language--the Unified Modeling Language (UML) --results in improved team communication
  - Reverse-engineering capabilities allow you to integrate with legacy OO systems
  - Models and code remain synchronized through the development cycle
  - …

Demo Rose



ÔN TẬP

-Một sản phẩm phần mềm người ta chia quá trình phát triển sản phẩm ra nhiều giai đoạn như **Thu thập và phân tích yêu cầu-> phân tích -> thiết kế hệ thống->phát triển (coding)-> kiểm thử-> triển khai và bảo trì**

- Giai đoạn **phân tích -> thiết kế hệ thống** giúp chúng ta hiểu rõ yêu cầu đặt ra, xác định giải pháp, mô tả chi tiết giải pháp. Nó trả lời 2 câu hỏi What (phần mềm này làm cái gì?) và How (làm nó như thế nào?).
- OOAD là xem hệ thống gồm những đối tượng sống trong đó và tương tác với nhau, mô tả được tất cả các đối tượng và sự tương tác của chúng sẽ giúp chúng ta hiểu rõ hệ thống và cài đặt được nó

- **Khái niệm về UML (Unified Modeling Language)**

- UML là ngôn ngữ mô hình hóa hợp nhất dùng để biểu diễn hệ thống.
- nó dùng để tạo ra các bản vẽ(lược đồ -diagram) nhằm mô tả thiết kế hệ thống. Các bản vẽ này được sử dụng để các nhóm thiết kế trao đổi với nhau cũng như dùng để thi công hệ thống (phát triển), thuyết phục khách hàng, các nhà đầu tư v.v..

Ôn tập

- **Tại sao lại là OOAD và UML?**

- OOAD cần các bản vẽ để mô tả hệ thống được thiết kế.
- UML là ngôn ngữ mô tả các bản vẽ nên cần nội dung thể hiện. Do vậy, chúng ta phân tích và thiết kế theo hướng đối tượng và sử dụng UML để biểu diễn các thiết kế đó nên chúng thường đi đôi với nhau.

OOAD sử dụng UML

- UML sử dụng để vẽ cho nhiều lĩnh vực khác nhau như phần mềm, cơ khí, xây dựng v
- OOAD sử dụng UML bao gồm các thành phần sau:

- **View** (góc nhìn): nó không thể hiện hết hệ thống nhưng thể hiện rõ hệ thống ở một khía cạnh.

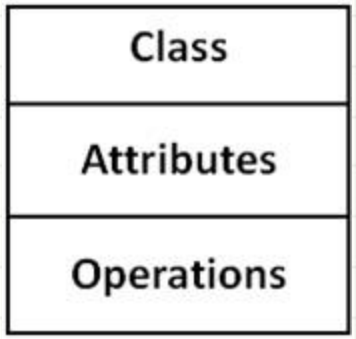
  Gồm ***Logical View, Process View, Component View, Deployment View, +*** **Use Case View.**

  Tập hợp các góc nhìn này sẽ giúp chúng ta hiểu rõ hệ thống cần phân tích, thiết kế

- Diagram (bản vẽ, sơ đồ): Các bản vẽ được dùng để thể hiện các góc nhìn của hệ thống. ***Logical View****(class diagram, object diagram)* **,** ***Process View (sequence diagram, state diagram, collaboration diagram, Activity diagram),*** ***Component View(Component diagram),*** ***Deployment View (Deployment diagram)*** **+ Use Case View(** Use Case ***diagram*****)**

- Notations (ký hiệu)





- Mechanisms (qui tắc_ **(Rules)**)

  Mechanisms là các qui tắc để lập nên bản vẽ, mỗi bản vẽ có qui tắc riêng và bạn phải nắm được để tạo nên các bản vẽ thiết kế đúng.



Chương 2: **Phân tích hệ thống**

Số giờ: **Thời gian**: Từ ngày đến ngày

I. MỤC TIÊU, YÊU CẦU

1. Mục tiêu: Sau khi học xong bài này người học có năng lực: phân tích được yêu cầu người dùng, xây dựng được các mô hình phù hợp

2. Yêu cầu: sử dụng phần mềm để xây dựng các mô hình

3. Dụng cụ, trang thiết bị: phòng học đủ máy tính, máy chiếu hoặc ti vi, Internet, bảng, phấn.

Chương 2: **Phân tích hệ thống**

2.1 Xác định yêu cầu hệ thống

- Mục tiêu
  - Hình thành tài liệu đặc tả yêu cầu (Requirement Specification)
- Tài liệu đặc tả yêu cầu được sử dụng như
  - Cam kết giữa khách hàng và tổ chức phát triển hệ thống về cái mà hệ thống có thể làm (và cái mà hệ thống không thể làm)
  - Cơ sở để đội ngũ phát triển hệ thống thực thi
  - Mô hình tương đối đầy đủ về cái hệ thống đòi hỏi

Đối tượng khảo sát

- Người dùng
  - Cán bộ lãnh đạo, quản lý
  - Người sử dụng, nhân viên nghiệp vụ
  - Nhân viên kỹ thuật
- Tài liệu
  - Sổ sách, biểu mẩu, tập tin
  - Quy trình, thủ tục
  - Các thông báo
- Chương trình máy tính
  - Xác định cấu trúc dữ liệu hệ thống
  - Thói quen người sử dụng
  - Số liệu thử nghiệm

Các hoạt động của phân tích yêu cầu

- Hiểu lĩnh vực vấn đề
  - Phân tích viên trình bày hiểu biết về lĩnh vực vấn đề
  - Khám phá các quan niệm
  - Suy ra các yêu cầu khách hàng
- Thu thập yêu cầu
  - Đưa ra các phương pháp xác định yêu cầu
    - Phỏng vấn: hình thức phỏng vấn, loại câu hỏi,…
    - Quan sát trực tiếp
    - Phân tích tài liệu và thủ tục

Các hoạt động của phân tích yêu cầu

- Phân lớp
  - Đầu vào của hoạt động này là tập hợp phi cấu trúc của các yêu cầu thu thập được trong pha trước để tổ chức chúng thành các nhóm dính liền nhau
  - Gắn mức ưu tiên cho các yêu cầu theo tầm quan trọng của chúng đối với khách hàng và người sử dụng
- Đánh giá
  - Kiểm tra xem các yêu cầu có nhất quán và đầy đủ
  - Giải quyết các mâu thuẫn giữa các yêu cầu
- Nghiên cứu khả thi
  - Dự báo khả năng thỏa mãn sử dụng phần cứng, phần mềm của các yêu cầu đã nhận ra
  - Quyết định các bước tiếp theo nếu nếu hệ thống đề xuất có hiệu quả

Phân tích yêu cầu

- Khi nào kết thúc phân tích yêu cầu?
  - Không có quy luật nhất định
- Để tiến tới bước phát triển phần mềm tiếp theo, hãy trả lời các câu hỏi sau:
  - Khách hàng, người sử dụng cuối cùng và người phát triển đã hiểu trọn vẹn hệ thống?
  - Mô hình của hệ thống đòi hỏi xây dựng đã được hình thành đầy đủ?
    - có đầy đủ các chức năng (dịch vụ)
    - có đầy đủ đầu vào- đầu ra
    - cần loại dữ liệu nào
- Chú ý: Chưa mô tả quyết định cài đặt nào ở mô hình này
- Đặc tả yêu cầu và mô hình của hệ thống tại mức này cần phải được hiệu chỉnh, bổ sung khi cần thiết trong các pha phát triển tiếp theo.

Phân tích yêu cầu

- Đặc tả yêu cầu
  - là thông báo chính thức cái đòi hỏi hệ thống phải được phát triển
  - Nó không phải là tài liệu thiết kế
- Mô tả đặc tả yêu cầu
  - Ngôn ngữ đặc tả
  - Ký pháp đồ họa

Pha thu thập và phân tích yêu cầu rất quan trọng. Cố gắng thu thập đầy đủ và tránh các sai sót các yêu cầu người dùng.
Nếu không phát hiện ra thiếu sót tại pha này thì rất khó và tốn kém để phát hiện ra nó ở pha tiếp theo.

2.2 Mô hình hóa yêu cầu

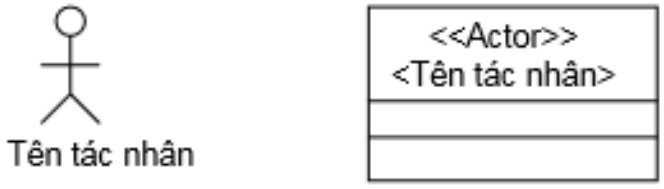
2.2.1 Giới thiệu

- Mục đích của Use Case
  - UC biểu diễn những chức năng mà hệ thống cần làm
  - Các UC cho phép:
    - Biết được hành vi của hệ thống mà không cần xác định làm thế nào hành vi này thực hiện
    - Định nghĩa những hạn chế chính xác của hệ thống
    - Cho người phát triển hiểu rõ hơn những gì mà khách hàng và người sử dụng chờ đợi

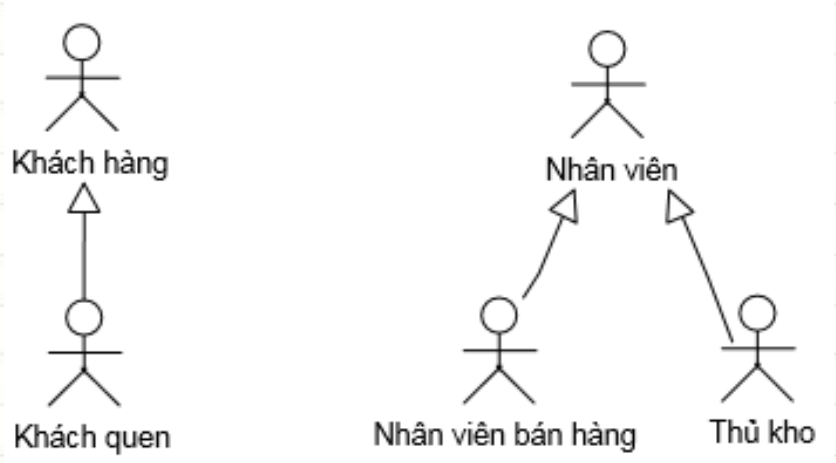
Mô hình UC (Mô hình ca sử dụng)

- Một biểu đồ UC định nghĩa:
  - Các tác nhân (Actor)
  - Các ca sử dụng (Use Case)
  - Quan hệ giữa các tác nhân và các ca sử dụng
- Một mô hình UC được định nghĩa bởi:
  - Các biểu đồ UC
  - Phần mô tả bằng lời các kịch bản sử dụng
  - Phần mô tả các kịch bản dùng:
    - Biểu đồ tuần tự
    - Biểu đồ tương tác

Các tác nhân (Actor)



- Một tác nhân là một người hoặc một thiết bị có tương tác với hệ thống
- Tên tác nhân là một danh từ
- Quan hệ giữa các tác nhân: tổng quát hóa và chuyên biệt hóa. Ví dụ



2.2.2 Xác định tác nhân

- Hãy trả lời các câu hỏi sau để tìm ra tác nhân hệ thống
  - Ai sẽ sử dụng chức năng chính của hệ thống?
  - Ai giúp hệ thống làm việc hàng ngày?
  - Ai quản trị, bảo dưỡng để hệ thống làm việc liên tục?
  - Hệ thống quản lý thiết bị phần cứng nào?
  - Hệ thống đang xây dựng tương tác với hệ thống khác nào?
  - Ai hay cái gì quan tâm đến kết quả hệ thống trả lại?

Ví dụ

- Trong hoạt động máy ATM của ngân hàng các tác nhân được xác định gồm:



2.2.3. Xác định UC

- Một UC là một chức năng khác nhau của hệ thống
- Nó biểu diễn một chuỗi tương tác giữa tác nhân và ứng dụng
- Nó định nghĩa một chức năng có thể sử dụng được bởi tác nhân



Xác định Use case

- Với mỗi tác nhân đã tìm ra, hãy trả lời các câu hỏi sau để tìm ra các Use case hệ thống
  - Tác nhân yêu cầu hệ thống thực hiện chức năng nào?
  - Tác nhân cần đọc, tạo lập, bãi bỏ, lưu trữ, sửa đổi các thông tin nào trong hệ thống?
  - Tác nhân cần thông báo cho hệ thống sự kiện xảy ra trong nó?
  - Hệ thống cần thông báo cái gì đó cho tác nhân?
  - Hệ thống cần vào/ra nào? Vào/ra đi đến đâu hay từ đâu?
- Đặt tên UC hệ thống
  - Theo khái niệm nghiệp vụ của tổ chức
  - Không sử dụng từ kỹ thuật, chuyên môn
  - Sử dụng các động từ, cụm từ ngắn gọn
- Tùy theo tầm cỡ dự án mà mỗi hệ thống có từ 20-70 UC

Đã tìm đầy đủ UC cho hệ thống?

- Các câu hỏi sau giúp xác định đã tìm đầy đủ UC?
  - Mỗi yêu cầu chức năng ở trong ít nhất một UC?
    - Nếu yêu cầu chức năng không ở trong UC nào thì nó sẽ không được cài đặt sau này.
  - Đã khảo sát mọi tác nhân tương tác với hệ thống?
  - Tác nhân cung cấp cho hệ thống thông tin nào?
  - Tác nhân nhận thông tin nào từ hệ thống?
  - Đã nhận biết mọi hệ thống bên ngoài tương tác với hệ thống đang xây dựng?
  - Thông tin nào hệ thống bên ngoài nhận và gửi cho hệ thống đang xây dựng?

Ví dụ

- Trong hệ thống ATM
  - Tác nhân khách hàng sẽ sử dụng hệ thống qua các chức năng: gửi tiền, rút tiền, truy vấn thông tin tài khoản



BT1

- Vẽ lược đồ use case cho đoạn mô tả bài toán sau
- Để mua hàng trên website, khách hàng cần lựa chọn hàng cần mua, nếu không nhìn thấy mặt hàng cần mua thì có thể sử dụng chức năng tìm kiếm. Sau khi chọn hàng, khách hàng kích hoạt chức năng thanh toán và thanh toán online thông qua một hệ thống ngân hàng trung gian. Trên cơ sở kiểm tra các đơn hàng đã thanh toán, bộ phận kinh doanh sẽ giao hàng tận nhà và xuất hóa đơn cho khách hàng. Ngoài ra, trên website bán hàng trực tuyến này, khách hàng còn được cung cấp các chức năng để theo dõi tình trạng đơn hàng, gửi khiếu kiện. Bộ phận chăm sóc khách hàng sẽ cùng với khách hàng giải quyết các khiếu kiện (nếu có).

BT2

- Vẽ lược đồ use case cho đoạn mô tả bài toán như sau:
- Để trở thành thành viên của diễn đàn, người sử dụng phải đăng ký. Sau khi đăng ký, thành viên có thể đăng nhập vào hệ thống để đăng bài viết, tải tài liệu, gửi và nhận tin nhắn. Nếu thành viên hoạt động tích cực và được cấp quyền quản trị, thì có thể kiểm duyệt bài, xóa bài của thành viên khác nếu họ vi phạm nội quy. Ngoài ra, nếu không phải là thành viên, người sử chỉ có thể đọc và tìm kiếm bài viết.

Luồng sự kiện trong UC

- Tài liệu luồng sự kiện (flow of events) mô tả hành vi của UC
  - mô tả luồng logic đi qua UC
  - mô tả người sử dụng làm gì, hệ thống làm gì
  - Trong một UC có nhiều luồng sự kiện: luồng chính, luồng phụ
- Kịch bản (Scenario)
  - Một luồng sự kiện trong một hiện thực của UC
  - Là trình tự hành động cụ thể để mô tả hành vi
  - Kịch bản đi xuyên suốt UC theo nhánh chính, nhánh phụ, nhánh đặc biệt

Tài liệu luồng sự kiện

- Tài liệu luồng sự kiện bao gồm
  - Mô tả vắn tắt UC
    - Mô tả ngắn gọn UC làm gì?
    - Những ai sử dụng UC?
    - Nó trả lại kết quả gì?
  - Tiền điều kiện (pre-condition)
    - Điều kiện cần thực hiện trước khi UC khởi động
    - Không phải UC nào cũng có tiền điều kiện
  - Luồng sự kiện chính và luồng sự kiện rẽ nhánh
  - Hậu điều kiện (post-condition)

Mô tả

- Gửi tiền: khách hàng đăng nhập vào hệ thống và yêu cầu gửi tiền vào tài khoản. Khách hàng sẽ xác định tài khoản và số tiền gửi, hệ thống sẽ tạo một giao tác gửi tiền và lưu vào hệ thống. Các bước như sau:
  - Yêu cầu xác định tài khoản
  - Hệ thống hỏi số tiền gửi
  - Nhập vào số tiền gửi
  - Khách hàng đưa tiền vào máy ATM
  - Hệ thống tự động upadate tài khoản



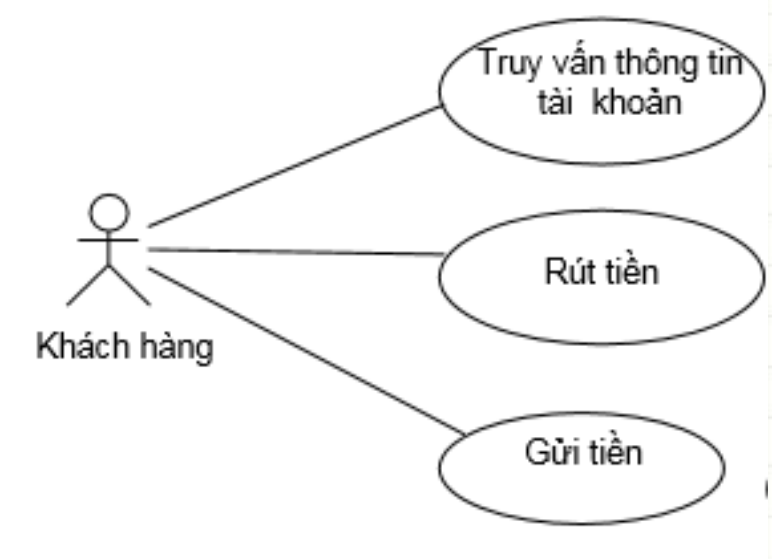
Mô tả



- Rút tiền: khách hàng đăng nhập hệ thống và yêu cầu rút tiền từ tài khoản. Khách hàng xác định tài khoản và lượng tiền rút. Sau khi kiểm tra số dư tài khoản còn đủ, hệ thống sẽ tạo một giao tác rút tiền và lưu vào hệ thống. Các bước như sau:
  - Yêu cầu xác định tài khoản
  - Yêu cầu xác định số tiền cần rút
  - Nhập số tiền rút
  - Kiểm tra số dư tài khoản và số tiền hiện có ở máy có đủ không ?
  - Chuyển tiền ra ngoài

2.2.4 Xác định mối quan hệ

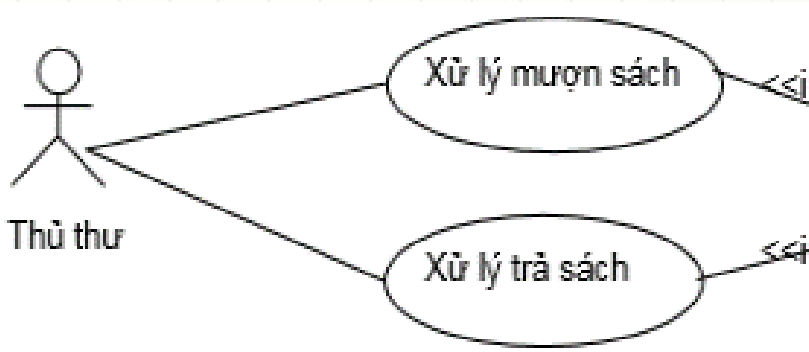
- Quan hệ này cho biết tác nhân sẽ tương tác với UC.
- Một UC luôn được khởi tạo bởi một tác nhân và tương tác với nhiều tác nhân



2.2.4 Xác định mối quan hệ

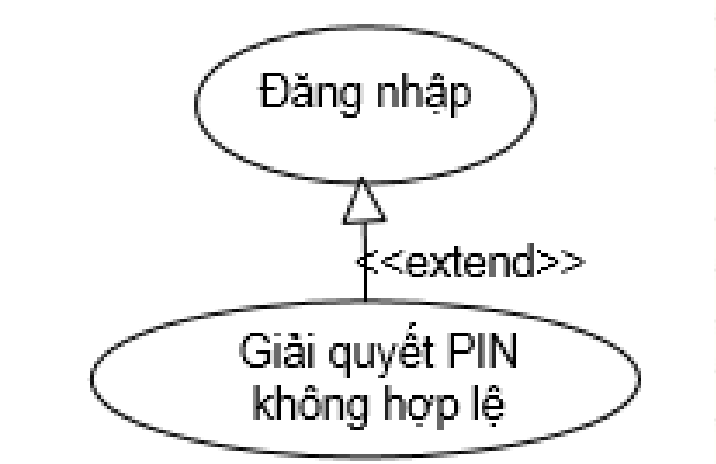
- Quan hệ Include
  - Quan hệ «*include*» biểu diễn một ca sử dụng chứa hành vi định nghĩa trong một ca sử dụng khác
  - Quan hệ này cho phép biểu diễn phần chung các hành vi của nhiều ca sử dụng





Quan hệ Extend

- Một UC tùy ý mở rộng chức năng do UC khác cung cấp
- Sử dụng để mô hình hóa một vài chức năng dùng chung, sử dụng lại giữa hai hay nhiều UC



So sánh

- Giống nhau giữa liên kết <<extend>> và liên kết <<include>> là tất cả đều được xem như là một loại kế thừa.
- Khác nhau:
  - Khi chúng ta muốn chia sẽ một số hoạt động chung trong nhiều UC, dùng liên kết <<include>> bằng cách trích các hoạt động chia sẽ đó thành một UC mới.
  - Khi chúng ta muốn thêm vào một ít khác biệt cho một UC để mô tả một tình huống đặc biệt trong một tình huống chung, chúng ta sẽ tạo một UC mới có liên kết <<extend>> với UC chung đó.

Ví dụ có Extend và Include

- Để mua hàng trên website, khách hàng cần lựa chọn hàng cần mua, nếu không nhìn thấy mặt hàng cần mua thì có thể sử dụng chức năng tìm kiếm. Sau khi chọn hàng, khách hàng kích hoạt chức năng thanh toán và thanh toán online thông qua một hệ thống ngân hàng trung gian. Trên cơ sở kiểm tra các đơn hàng đã thanh toán, bộ phận kinh doanh sẽ giao hàng tận nhà và xuất hóa đơn cho khách hàng. Ngoài ra, trên website bán hàng trực tuyến này, khách hàng còn được cung cấp các chức năng để theo dõi tình trạng đơn hàng, gửi khiếu kiện. Bộ phận chăm sóc khách hàng sẽ cùng với khách hàng giải quyết các khiếu kiện (nếu có).

Ví dụ có Extend và Include

- Vẽ lược đồ use case cho đoạn mô tả bài toán như sau:
- Để trở thành thành viên của diễn đàn, người sử dụng phải đăng ký. Sau khi đăng ký, thành viên có thể đăng nhập vào hệ thống để đăng bài viết, tải tài liệu, gửi và nhận tin nhắn. Nếu thành viên hoạt động tích cực và được cấp quyền quản trị, thì có thể kiểm duyệt bài, xóa bài của thành viên khác nếu họ vi phạm nội quy. Ngoài ra, nếu không phải là thành viên, người sử chỉ có thể đọc và tìm kiếm bài viết.

Các bước Mô hình hóa một hệ thống: xác định ca sử dụng

- Xác định các tác nhân sử dụng
- Với mỗi tác nhân, tìm kiếm các ca sử dụng với hệ thống. Đặc biệt những gì thay đổi trạng thái của hệ thống hoặc chờ đợi phản ứng từ hệ thống
- Tổ chức lại các ca sử dụng với các quan hệ sử dụng, mở rộng…

Bài tập

- Vẽ biểu đồ use cases cho các hệ thống phần mềm:
  - Bán vé máy bay
  - Máy rút tiền ATM

Biểu đồ hoạt động

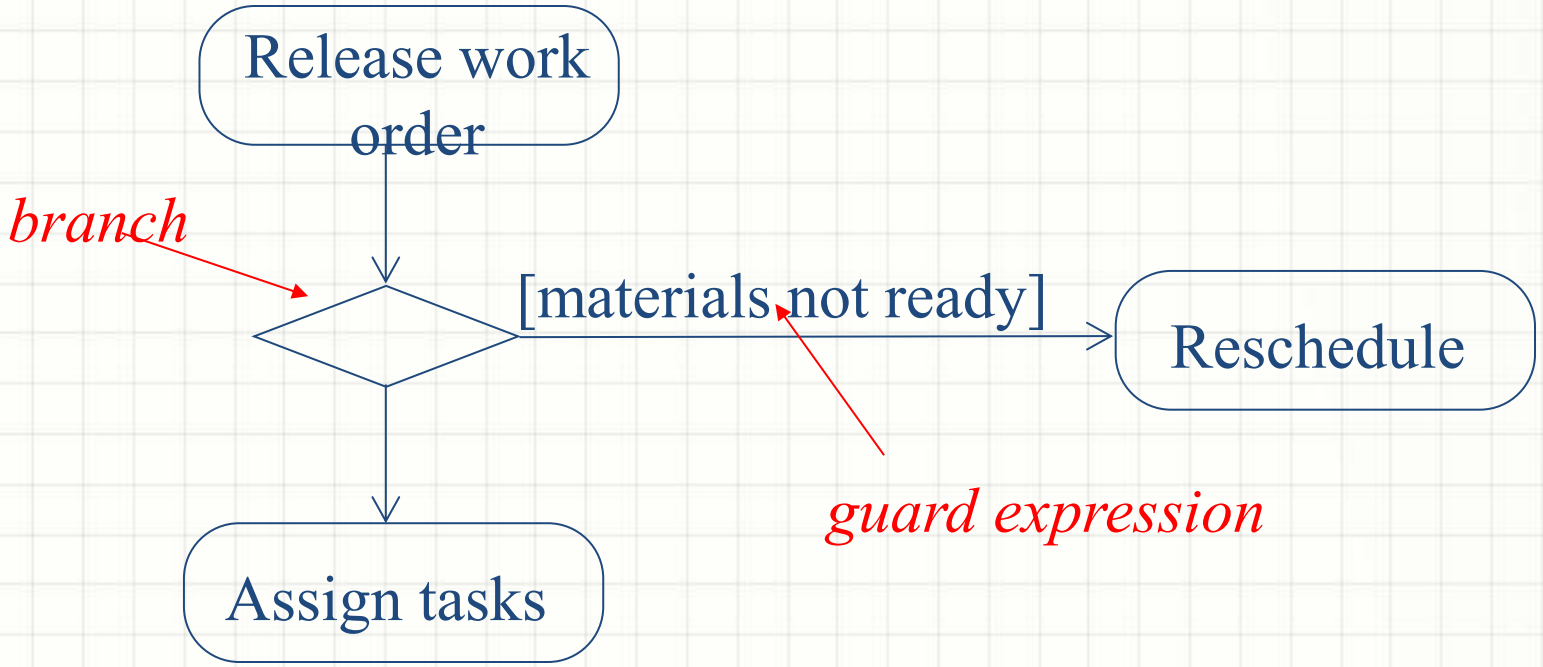
- Biểu đồ hoạt động (Activity diagram) do Odell đề xuất cho UML để
  - mô tả luồng công việc trong tiến trình nghiệp vụ trong mô hình hóa nghiệp vụ
  - mô tả luồng sự kiện trong mô hình hóa hệ thống
    - Sử dụng text như trước đây sẽ khó đọc khi logíc phức tạp, có nhiều rẽ nhánh
- Biểu đồ hoạt động sử dụng để mô hình hóa
  - khía cạnh động của hệ thống
  - các bước trình tự hay tương tranh trong quá trình tính toán

Biểu đồ hoạt động

- Biểu đồ hoạt động bao gồm *activities*, *states* *và* *transitions*
  - Một *hoạt động* là đặc tả của hành vi được biểu diễn bởi một luồng các hành động
  - Một *trạng thái* là điểm mà các sự kiện cần đạt tới trước khi hoạt động tiếp tục
  - Một *chuyển tiếp* là việc chuyển đổi giữa các hoạt động hoặc trạng thái

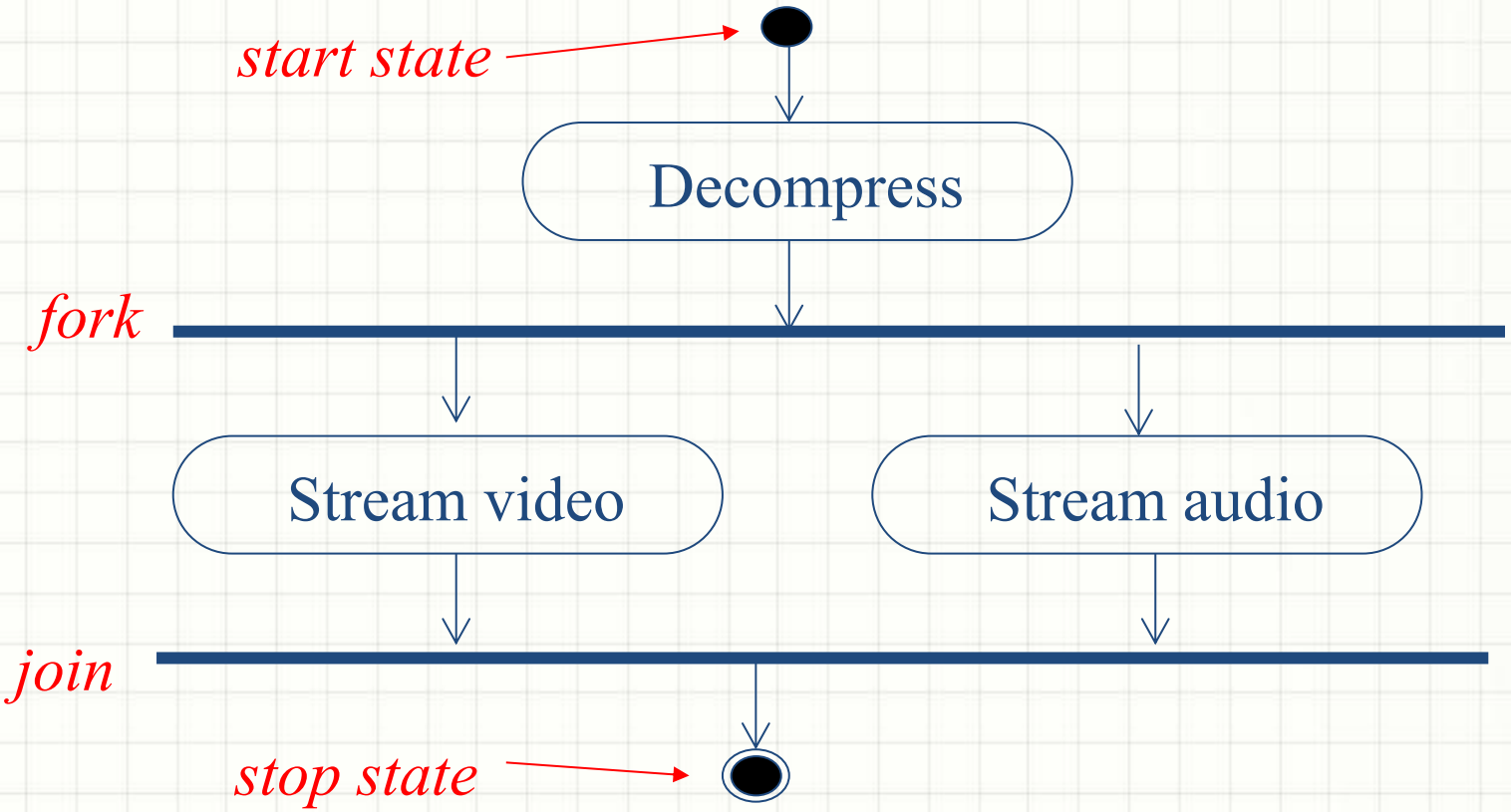
Biểu đồ hoạt động

- *Điểm quyết định* là điểm trong một luồng công việc mà ở đó việc chuyển tiếp từ một trạng thái hoặc một hoạt động phân theo các nhánh khác nhau tùy theo điều kiện



Thanh tương tranh hay thanh đồng bộ

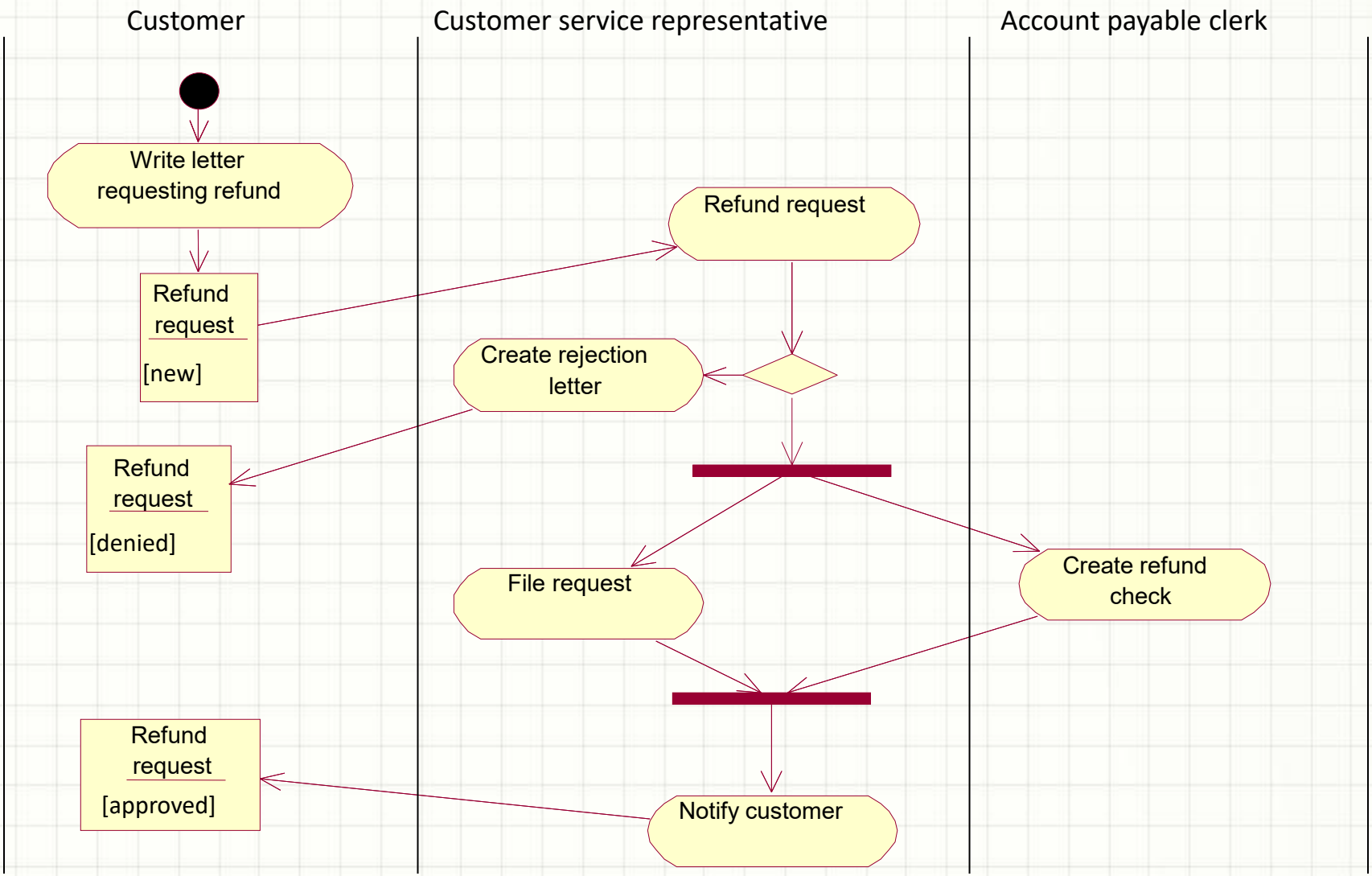
- **Thanh tương tranh hay thanh đồng bộ** Trong hoạt động của hệ thống, có thể có nhiều luồng hành động được bắt đầu thực hiện hay kết thúc đồng thời.



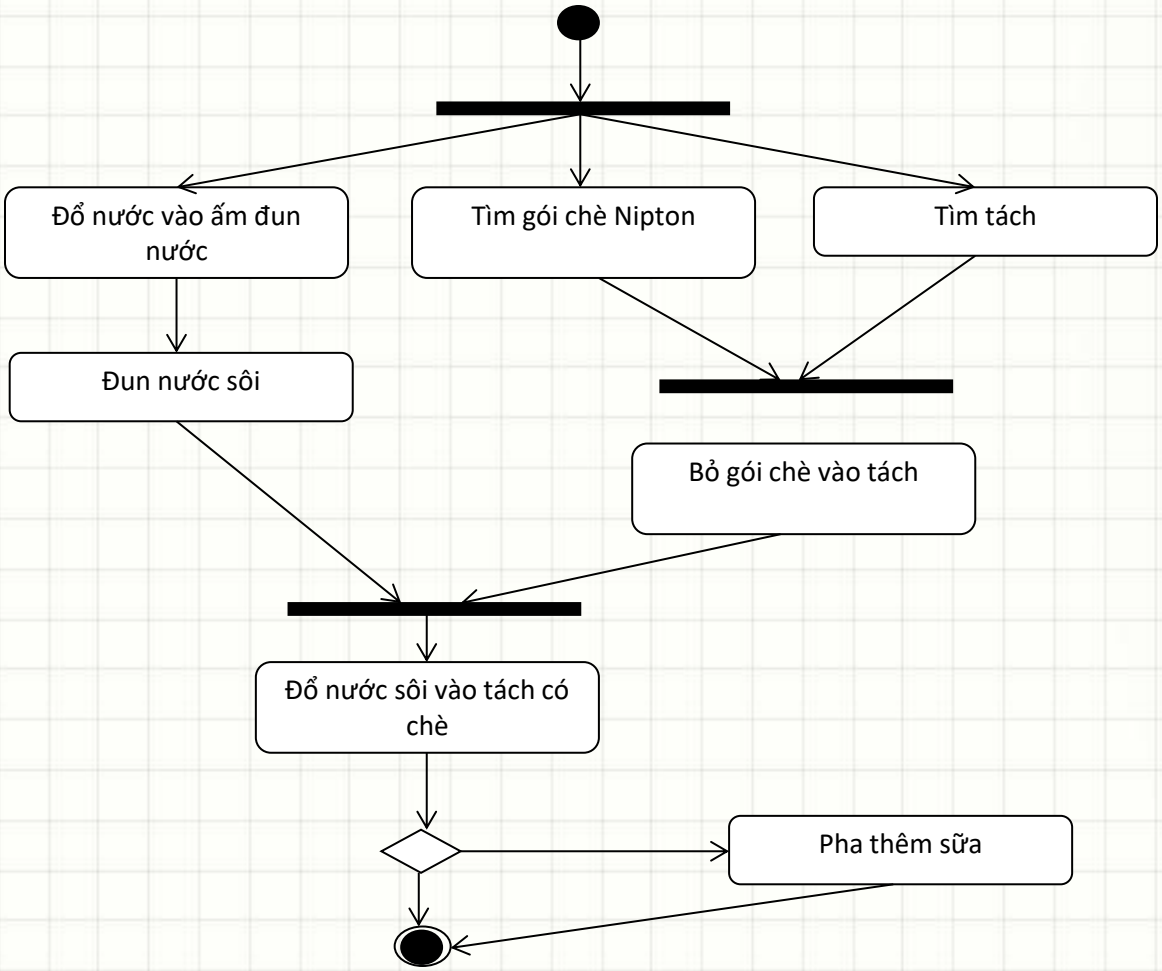
Làn bơi (Swimlanes)

- Sử dụng để mô hình hóa luồng công việc trong tiến trình nghiệp vụ
- Chỉ ra ai có trách nhiệm thực hiện từng hoạt động
- Để phân hoạch các trạng thái hoạt động vào nhóm
- Mỗi hoạt động thuộc về một làn bơi
- Chuyển tiếp có thể được vẽ từ làn bơi này đến làn bơi khác
- Mỗi làn bơi có thể được cài đặt bởi một hay nhiều lớp

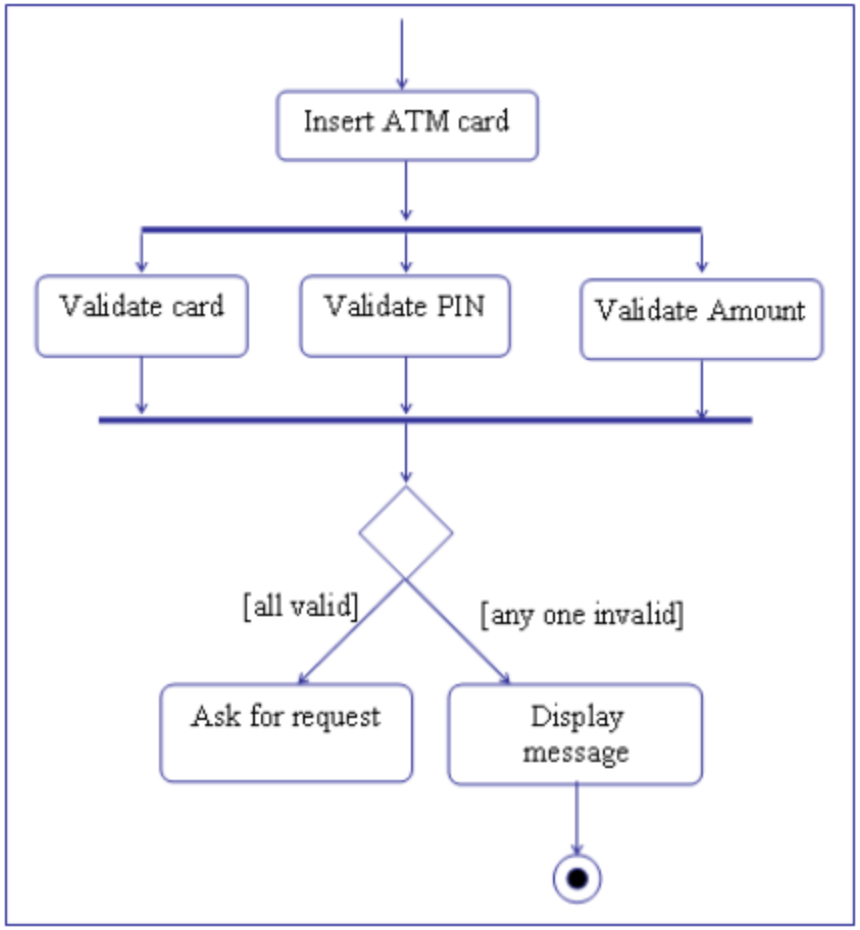
Làn bơi


Customer
Customer service representative
Account payable clerk
Write letter requesting refund
Refund request
[new]
Refund request
Create rejection letter
Refund request
[denied]
File request
Create refund check
Refund request
[approved]
Notify customer

Biểu đồ hành động mô tả các hoạt động "*Đun nước và pha một tách chè*



- Hình miêu tả một đoạn biểu đồ hoạt động của máy ATM. Sau khi thẻ được đưa vào máy, ta thấy có ba hoạt động song song:
  - Xác nhận thẻ
  - Xác nhận mã số PIN
  - Xác nhận số tiền yêu cầu được rút
- Chỉ khi sử dụng biểu đồ hoạt động, các hoạt động song song như vậy mới có thể được miêu tả. Mỗi một hoạt động xác nhận bản thân nó cũng đã có thể là một quá trình riêng biệt.



Biểu đồ hoạt động của máy ATM

Lưu ý

- Tại các thanh tương tranh/thanh đồng bộ chỉ thoã mãn
  - Vô 1 ra nhiều
  - Ra 1 vô nhiều
- Không thoã mãn cho trường hợp vô nhiều ra nhiều

Khi nào dùng biểu đồ hoạt động

- Không cần phải vẽ tất cả các loại biểu đồ động cho tất cả các loại hệ thống. Mặc dù vậy, trong một số trường hợp khác nhau chúng ta nhất thiết phải cần đến một số loại biểu đồ động nhất định.
- Biểu đồ hoạt động được sử dụng để thể hiện lối ứng xử xuyên suốt nhiều trường hợp sử dụng (UC) hoặc các tiểu trình xảy ra song song của một lần thực thi.

Bài tập

- Vẽ biểu đồ hoạt động của luồng nghiệp vụ mua hàng trực tuyến
- Vẽ biểu đồ hoạt động của luồng nghiệp vụ mua sách trên mạng

2.4 Xây dựng sơ đồ lớp đối tượng

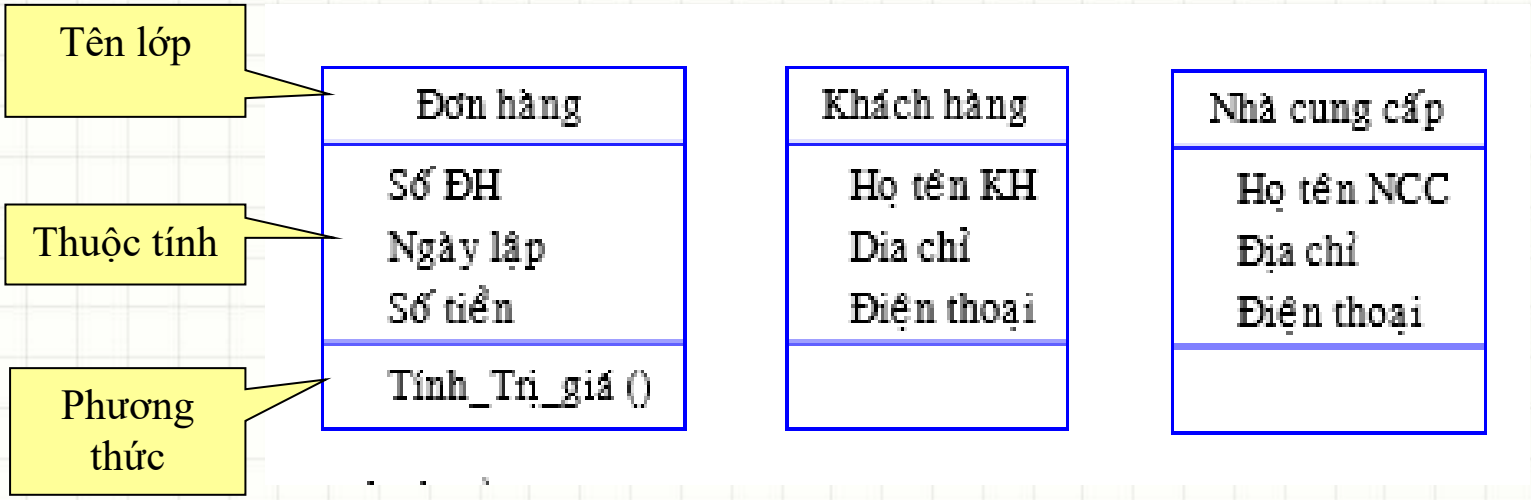
- Đối tượng là một thực thể có một vai trò xác định rõ ràng trong lãnh vực ứng dụng, có trạng thái, hành vi và định danh
- Lớp là một tập hợp các đối tượng chia sẽ chung một cấu trúc và hành vi
- Biểu đồ lớp mô tả kiểu của các đối tượng trong hệ thống và các loại quan hệ khác nhau tồn tại giữa chúng
- Là một kỹ thuật mô hình hóa tồn tại ở tất cả các phương pháp phát triển hướng đối tượng
- Biểu đồ hay dùng nhất trong UML.

Các phần tử của biểu đồ lớp

- Lớp
- Thuộc tính
- Phương thức
- Quan hệ
  - Liên kết (Associations)
  - Tổng quát hóa (Generalization)
  - Phụ thuộc (Dependency)
  - Thực hiện (Realization)
- Các luật ràng buộc và ghi chú

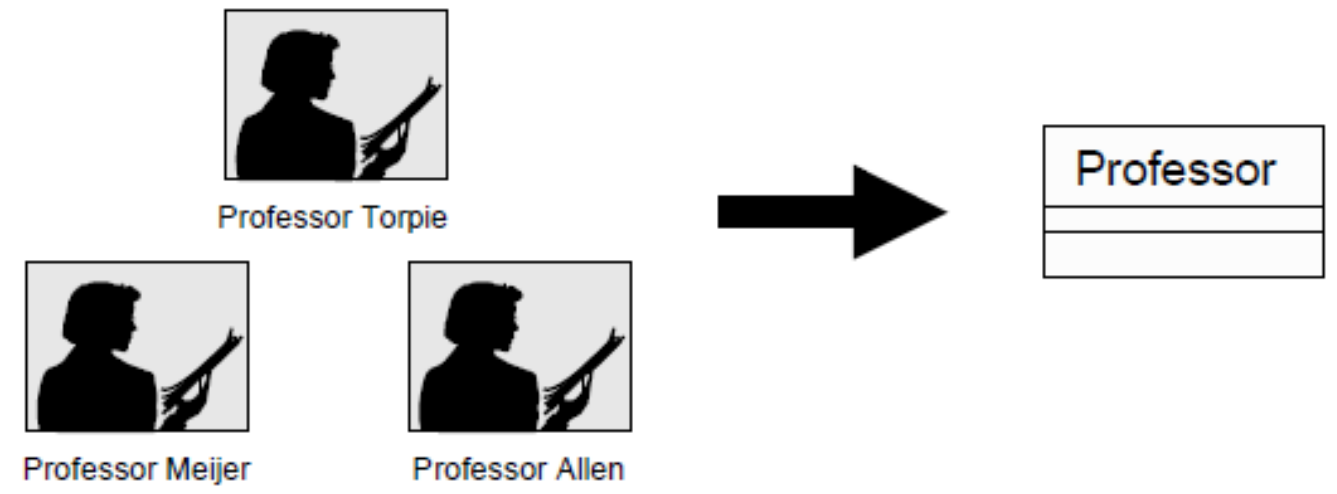
Lớp

- Một lớp là một mô tả của một tập các đối tượng có chung thuộc tính, phương thức và quan hệ



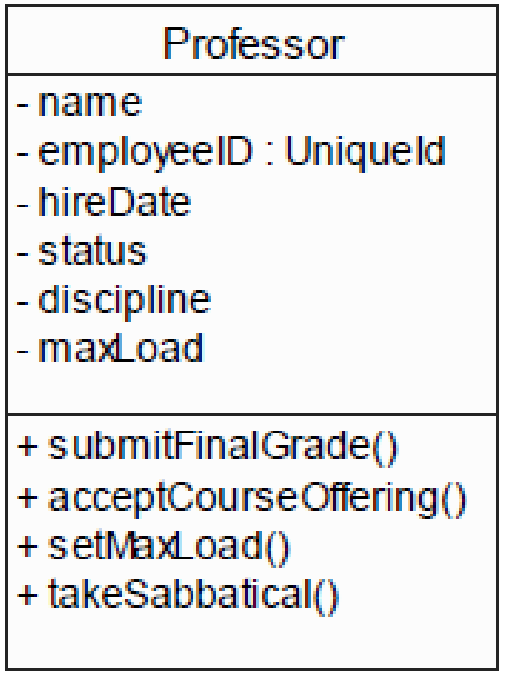
Quan hệ giữa lớp và đối tượng

- Lớp là định nghĩa trừu tượng của đối tượng
  - Lớp định nghĩa cấu trúc và hành vi của mỗi đối tượng thuộc về lớp
  - Lớp đóng vai trò như là mẫu để tạo ra đối tượng
- Lớp không phải là tập hợp các đối tượng



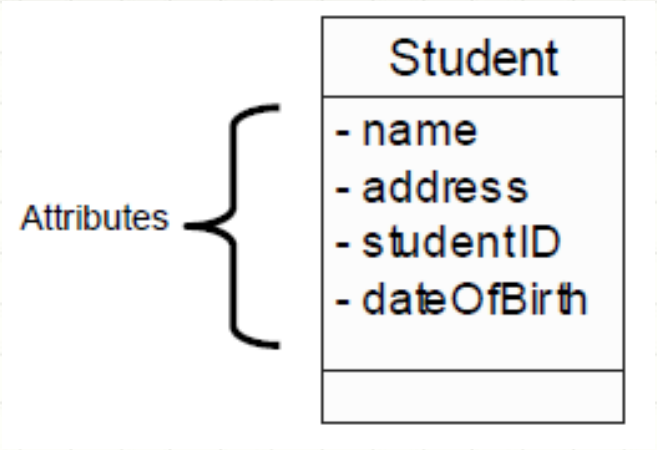
Các phần trong lớp

- Một lớp có 3 phần
  - Tên lớp
  - Cấu trúc (các thuộc tính - attributes)
  - Hành vi (các thao tác - operations)

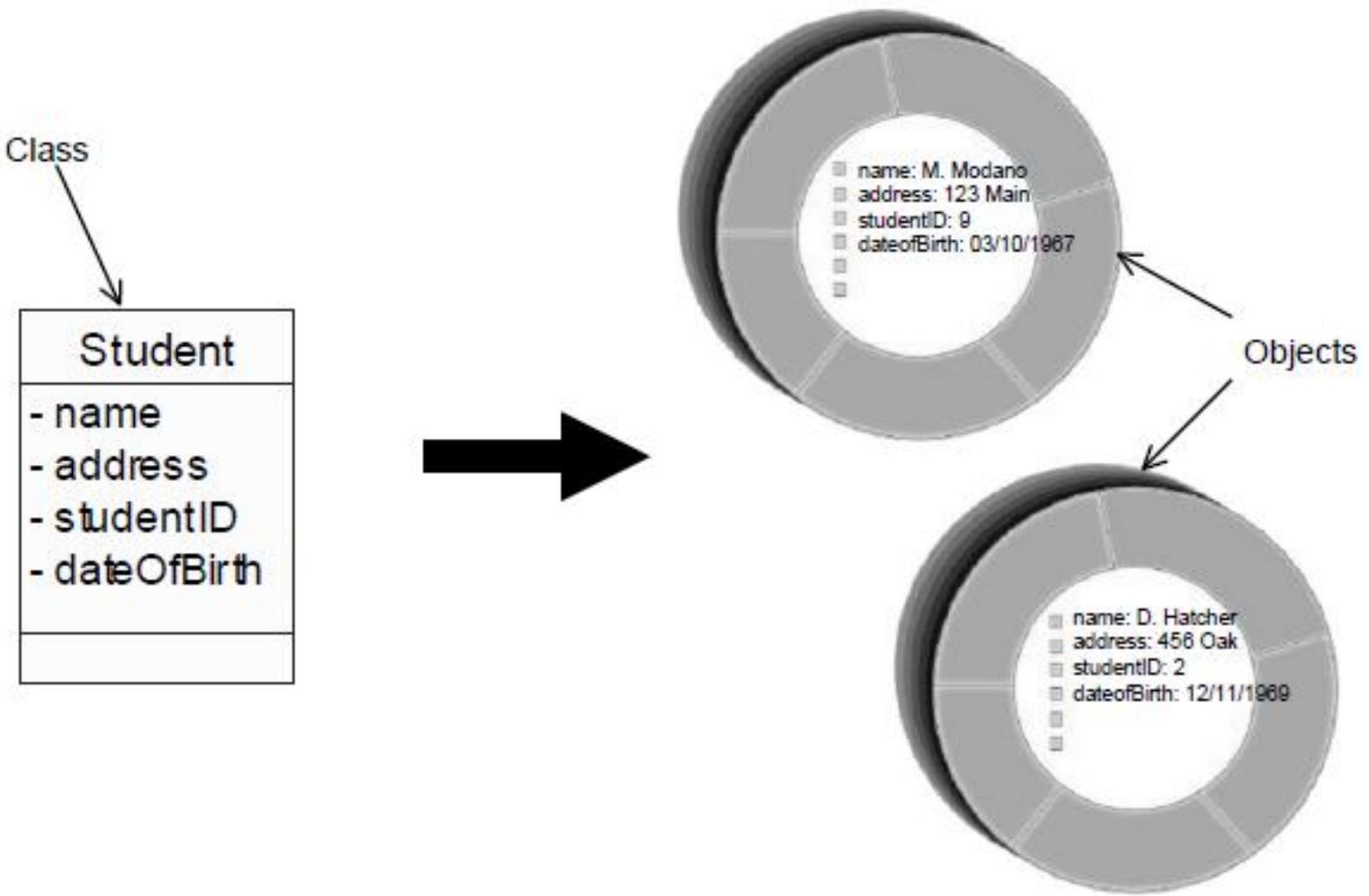


Thuộc tính

- Thuộc tính là *tính chất được đặt tên* của lớp, mô tả dải giá trị mà *tính chất đó* của đối tượng có thể nhận
  - Lớp có thể có 1, nhiều, hoặc không có thuộc tính nào

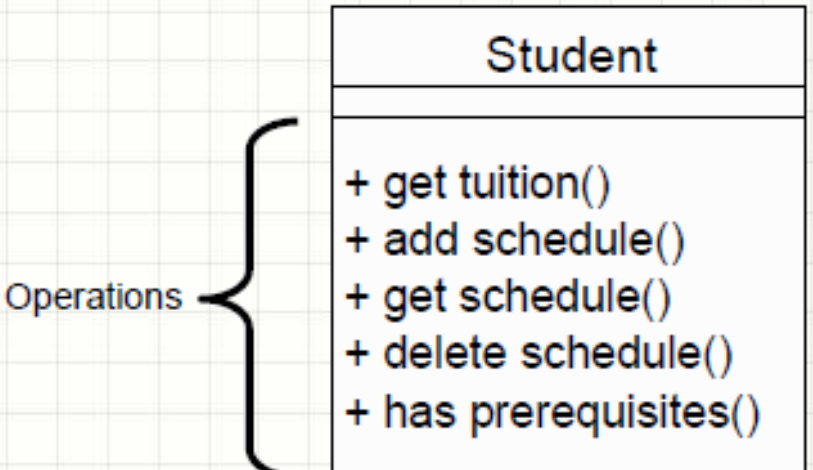


Thuộc tính của lớp và của đối tượng



Phương thức

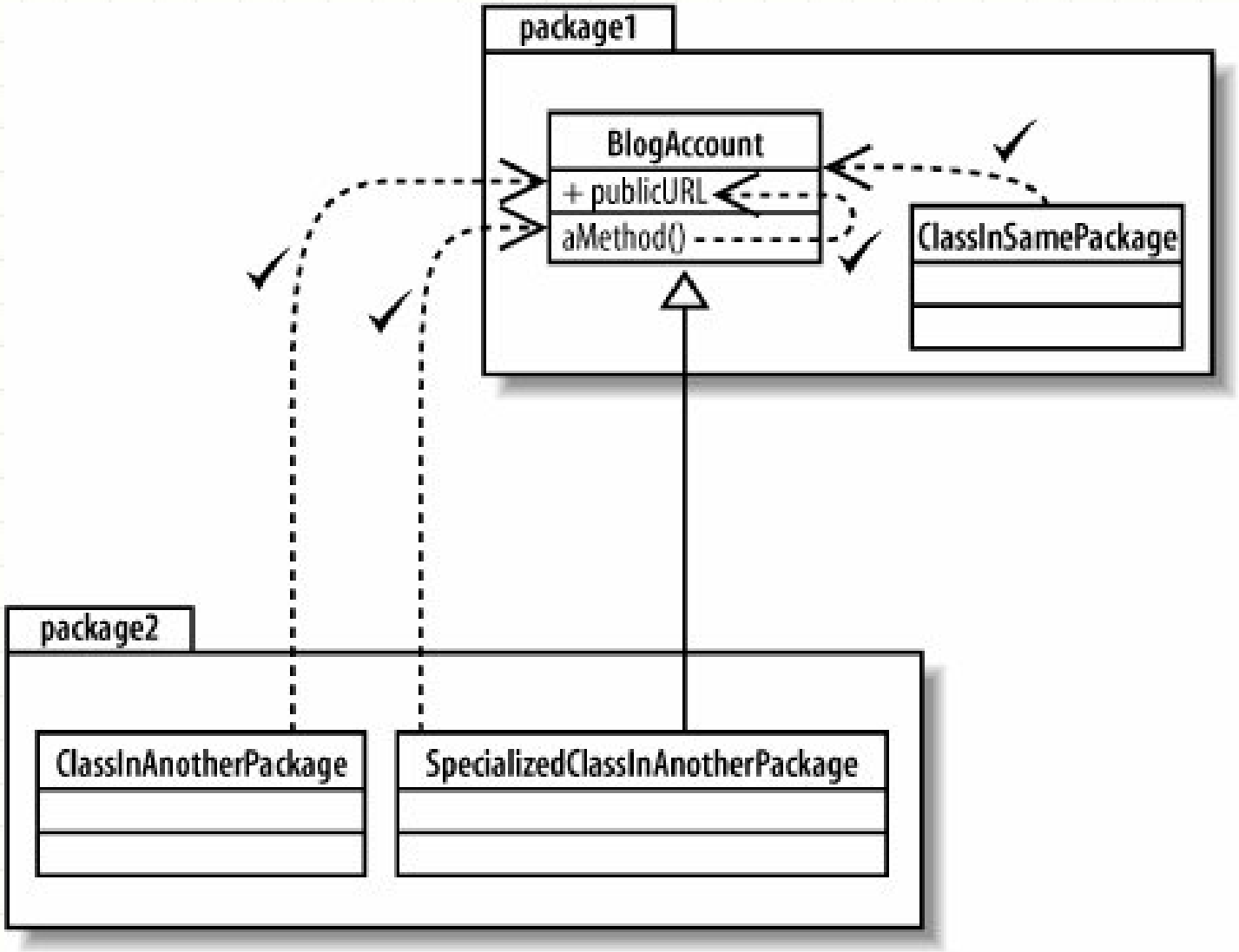
- Một dịch vụ có thể được yêu cầu từ một đối tượng để tác động đến hành vi.
- Một Phương thức có một ký hiệu để giới hạn các tham số có thể có
- Một lớp có thể có 1, nhiều, hoặc không có Phương thức nào



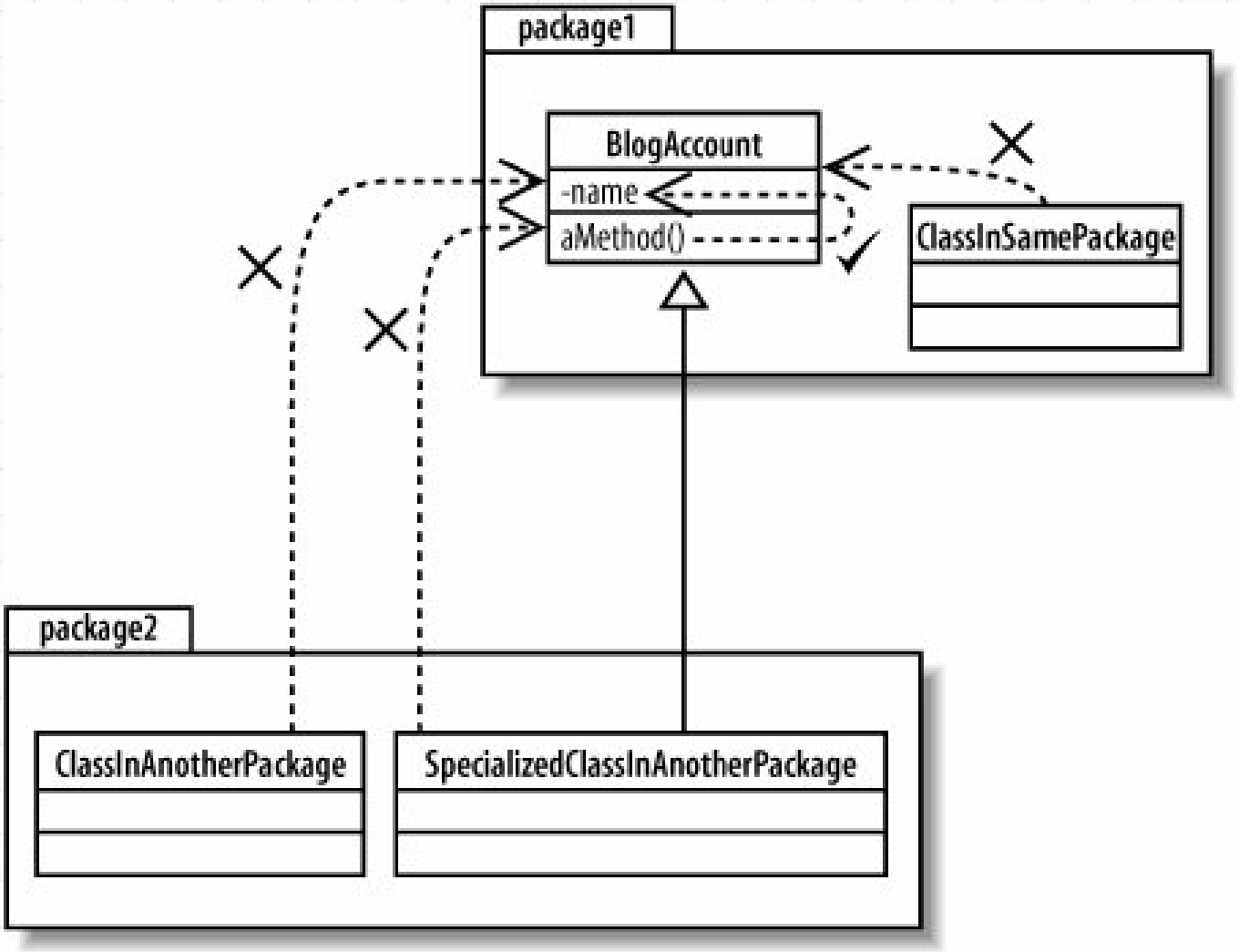
Đặc tả thuộc tính lớp

- Visibility
  - Đóng gói trong lập trình hướng đối tượng
  - Bốn lựa chọn phạm vi cho thuộc tính
    - Public: Mọi lớp đều nhìn thấy thuộc tính (+)
    - Private: Lớp khác không nhìn thấy thuộc tính (-)
    - Protected: Các lớp kế thừa có thể nhìn thấy (#)
    - Package và Implementation: Thuộc tính là public đối với các lớp trong cùng gói

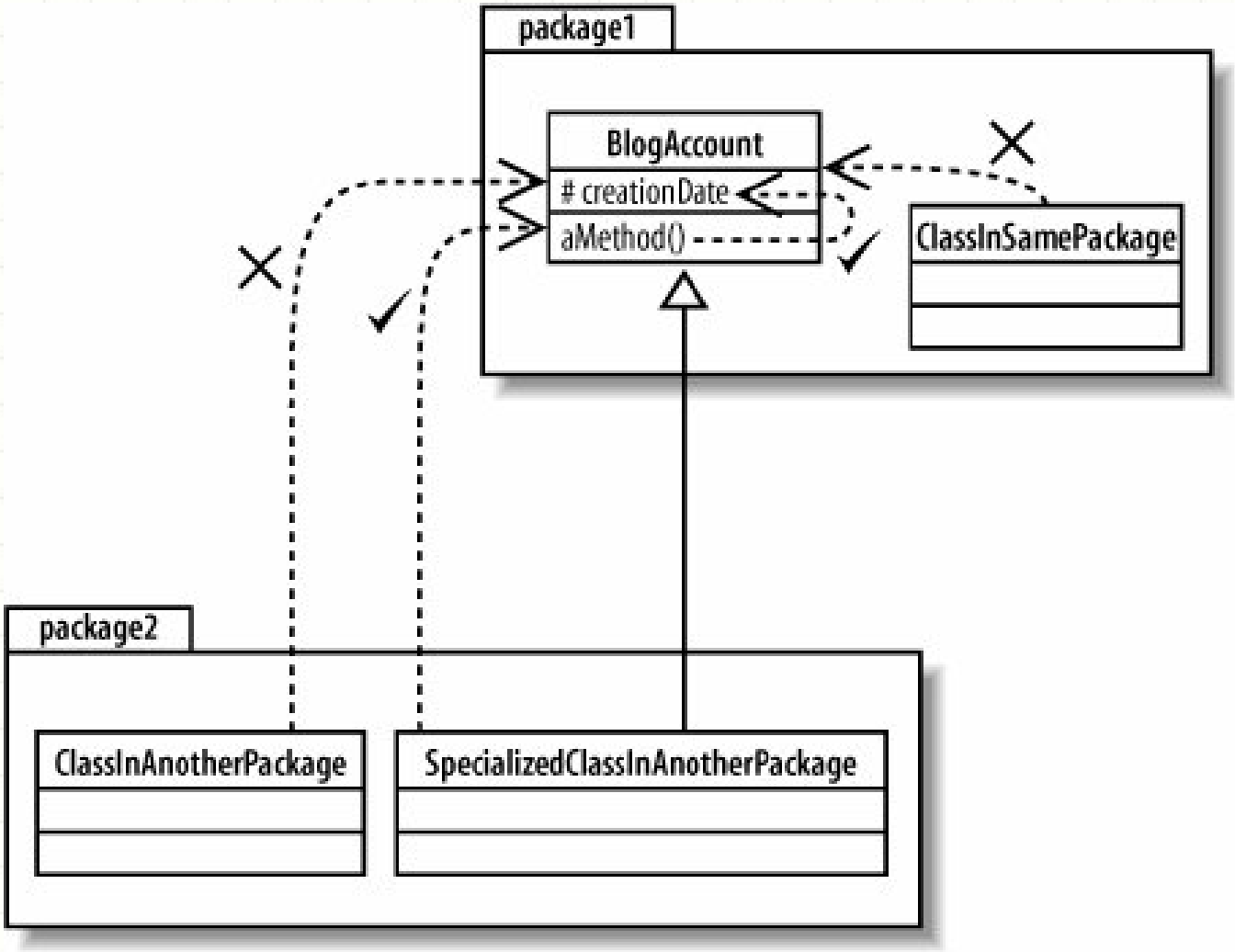
Public


package1
BlogAccount
+ publicURL
aMethod()
ClassInSamePackage
package2
ClassInAnotherPackage
SpecializedClassInAnotherPackage

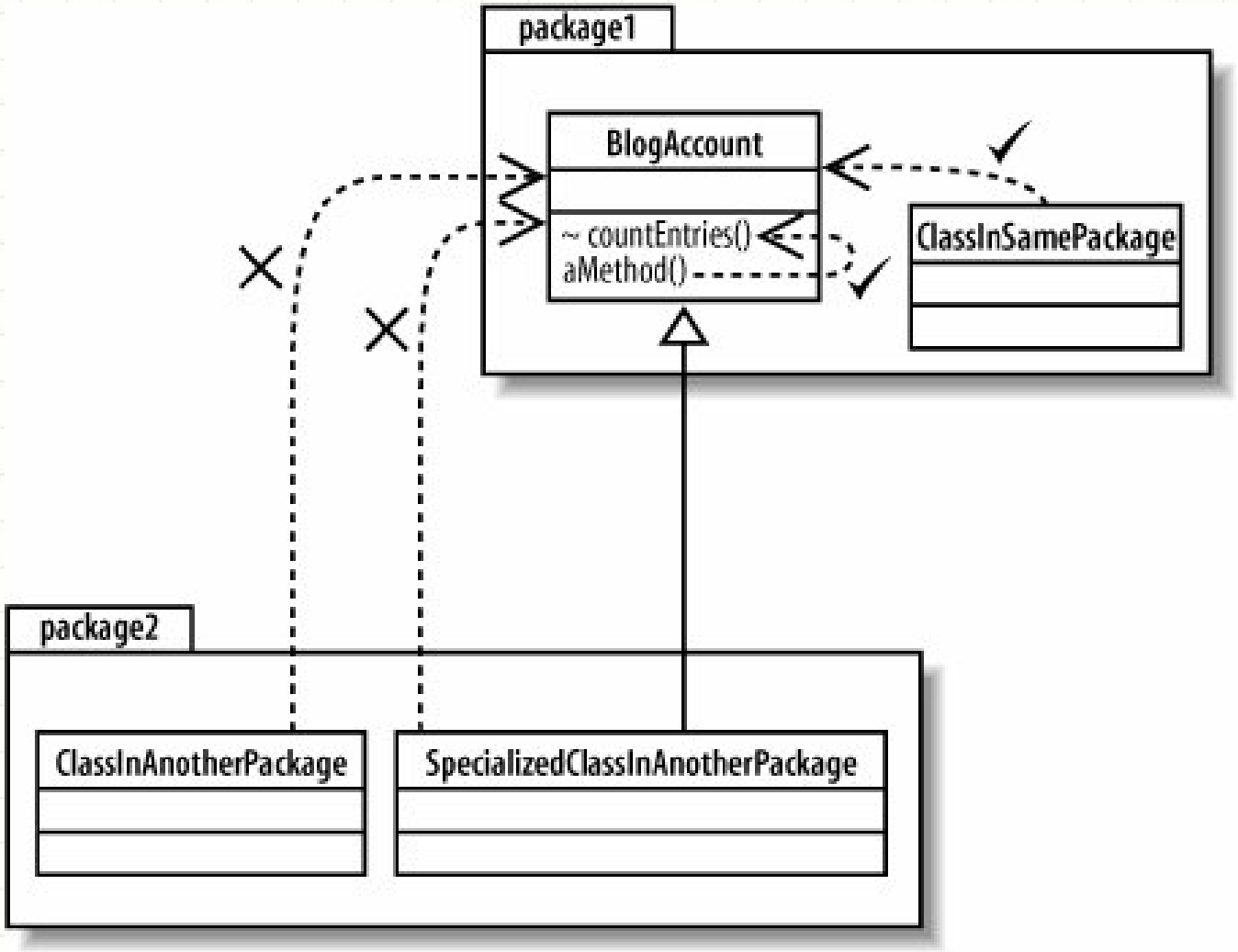
Private


package1
BlogAccount
-name
aMethod()
ClassInSamePackage
package2
ClassInAnotherPackage
SpecializedClassInAnotherPackage

Protected

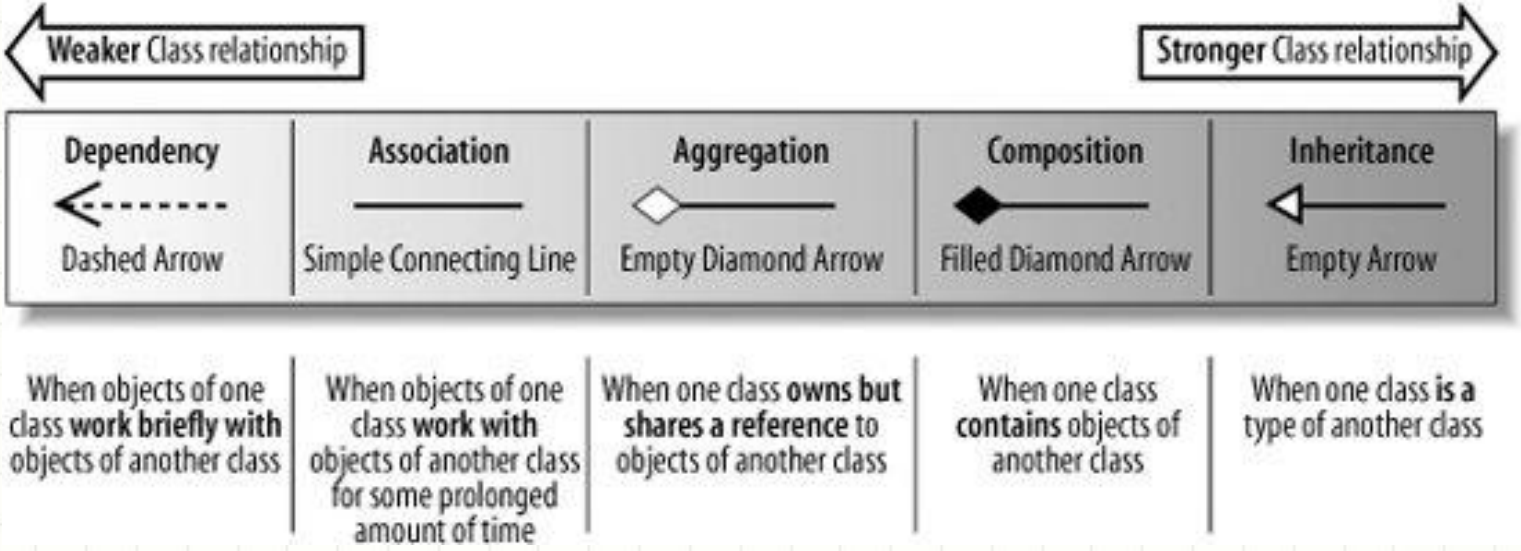

package1
BlogAccount
creationDate
aMethod()
ClassInSamePackage
package2
ClassInAnotherPackage
SpecializedClassInAnotherPackage

Package


package1
BlogAccount
~ countEntries()
aMethod()
ClassInSamePackage
package2
ClassInAnotherPackage
SpecializedClassInAnotherPackage

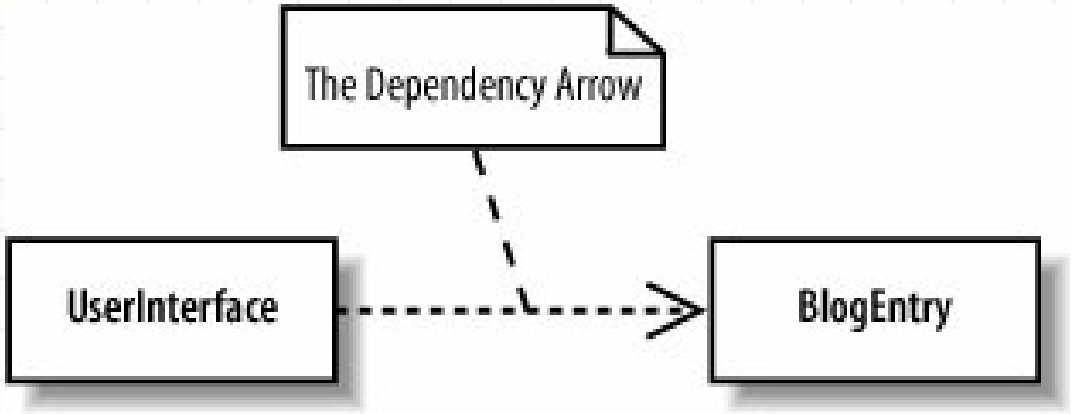
Mối quan hệ giữa các lớp

- Mối quan hệ tổng quát hoá (generalization)
- Mối quan hệ kết hợp (association)
- Mối quan hệ aggregation
- Mối quan hệ composite



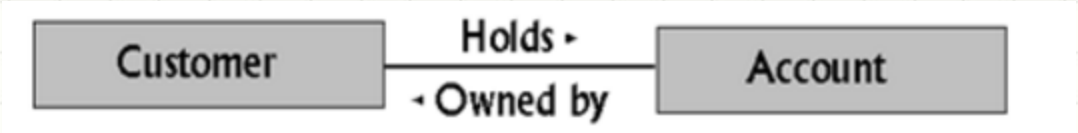
Mối quan hệ phụ thuộc

- Class A phụ thuộc class B: các đối tượng của class A có thể làm việc với các đối tượng của class B, hoặc với class B
- Mối quan hệ phụ thuộc thường được dùng khi bạn có 1 class cung cấp một số hàm tiện ích. Các class dùng các tiện ích này sẽ phụ thuộc vào class tiện ích đó.



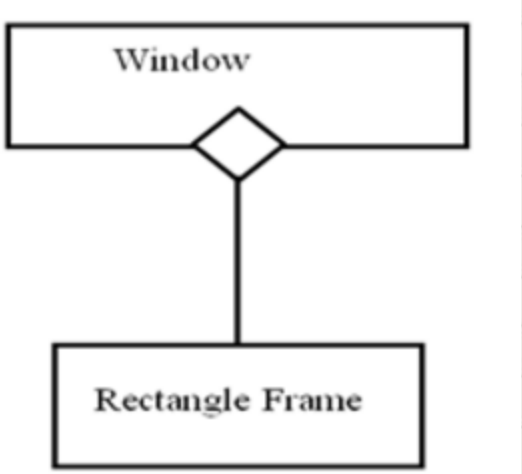
Association

- Association là quan hệ giữa hai lớp với nhau, thể hiện chúng có liên quan với nhau. Association thể hiện qua các quan hệ như “has: có”, “Own: sở hữu” v.v…

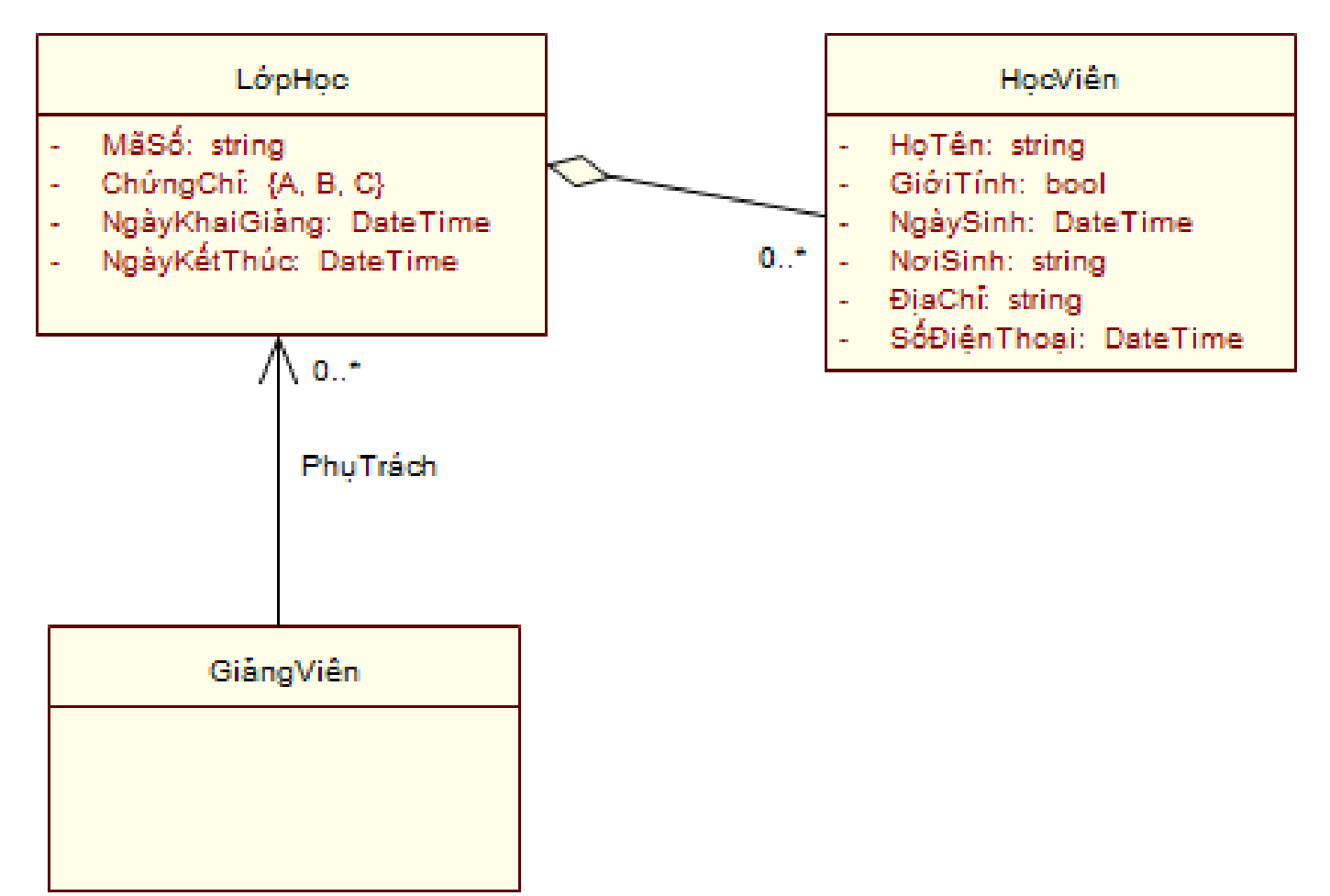


Aggregation – Quan hệ tụ hợp

- Aggregation là một loại của quan hệ Association nhưng mạnh hơn. Nó có thể cùng thời gian sống (cùng sinh ra hoặc cùng chết đi)

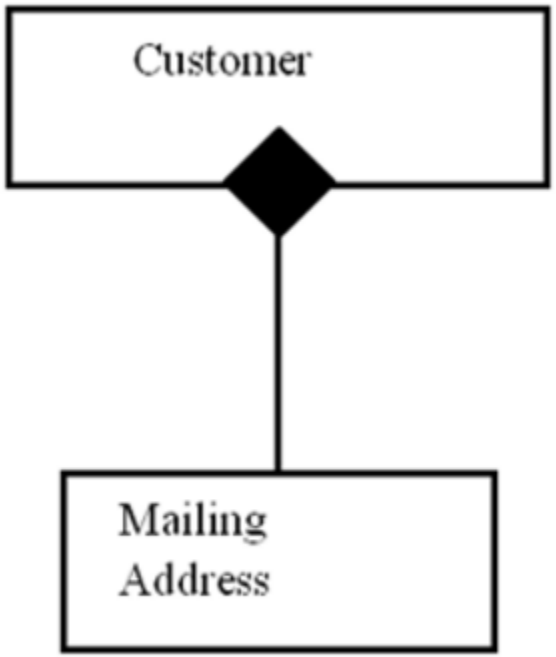


Ví dụ về mối quan hệ Aggregation



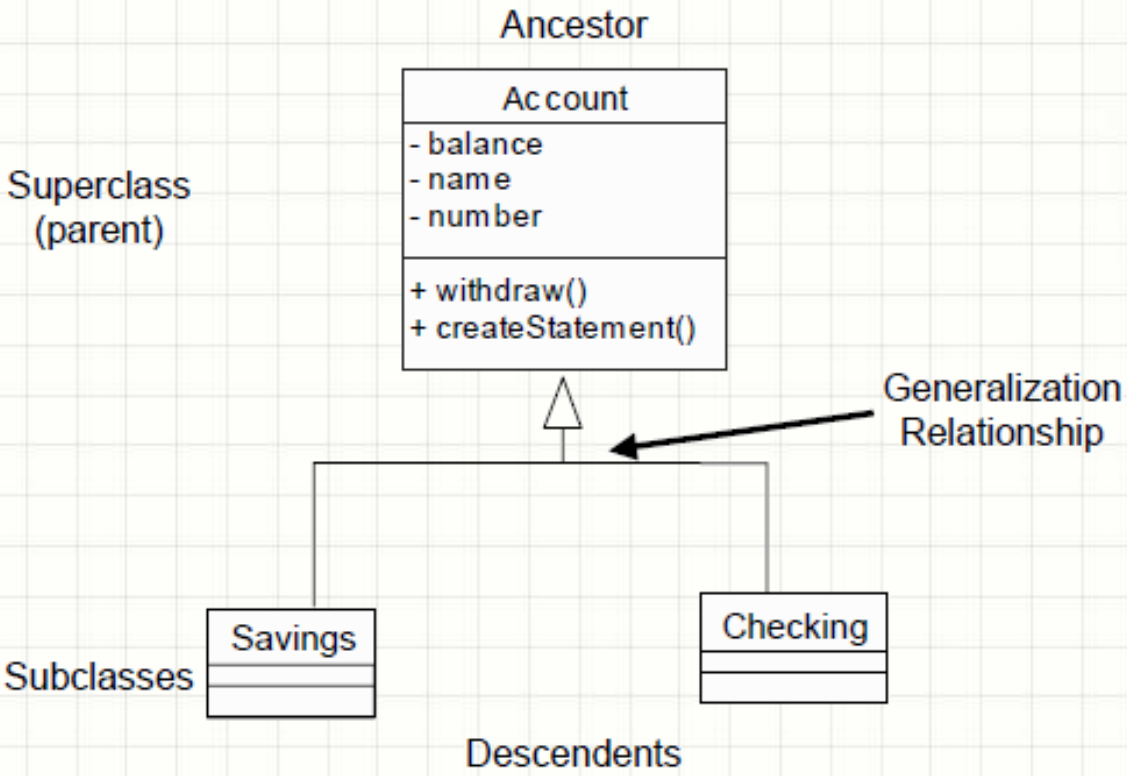
Composition – Cấu thành

- Composition là một loại mạnh hơn của Aggregation thể hiện quan hệ class này là một phần của class kia nên dẫn đến cùng tạo ra hoặc cùng chết đi.

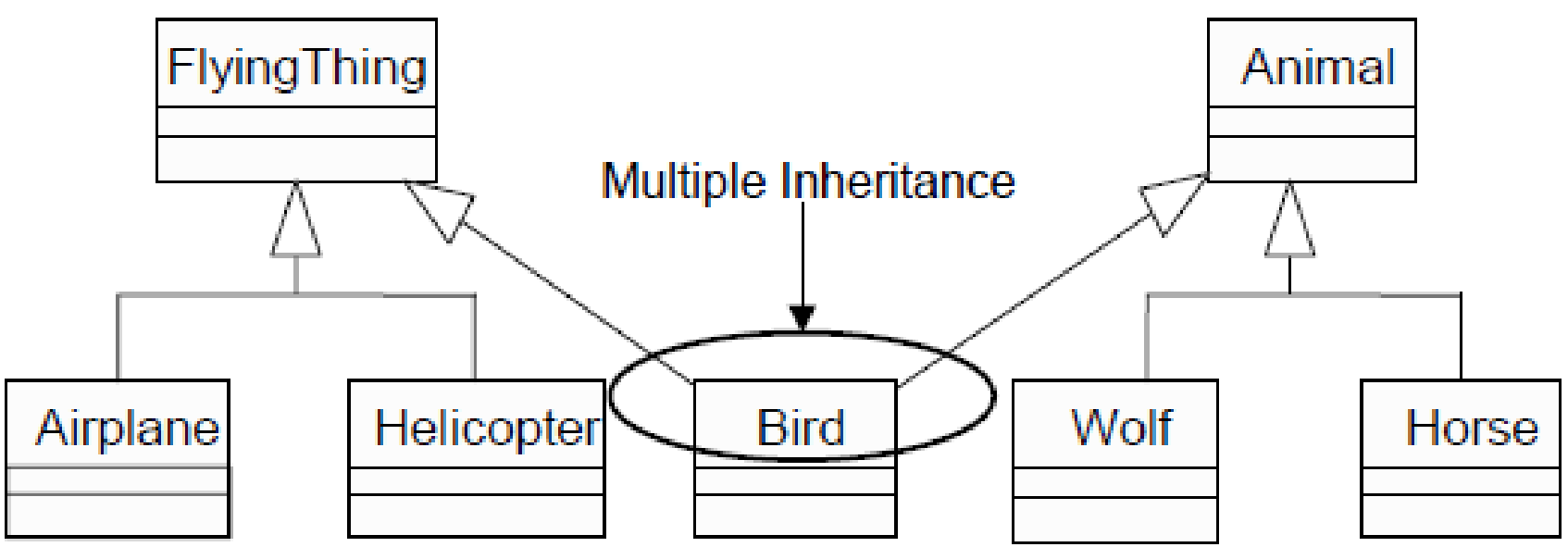


Genelization - Tổng quát hóa

- Generalization là quan hệ thừa kế được sử dụng rộng rãi trong lập trình hướng đối tượng.



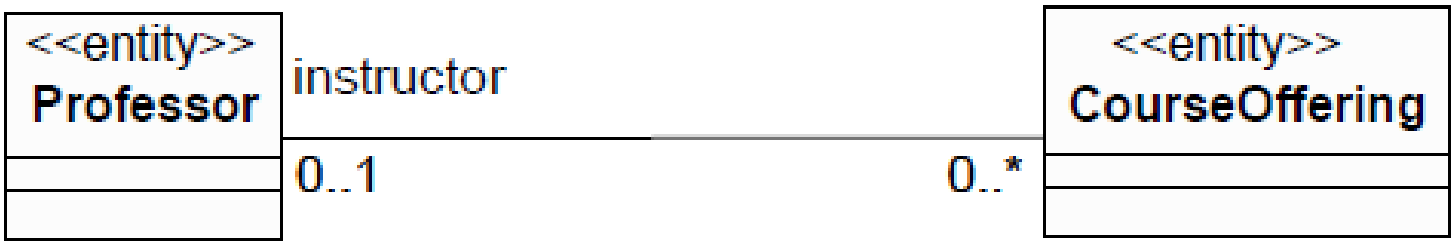
Ví dụ đa kế thừa



Use multiple inheritance only when needed and always with caution!

Bản số (Multiplicity)

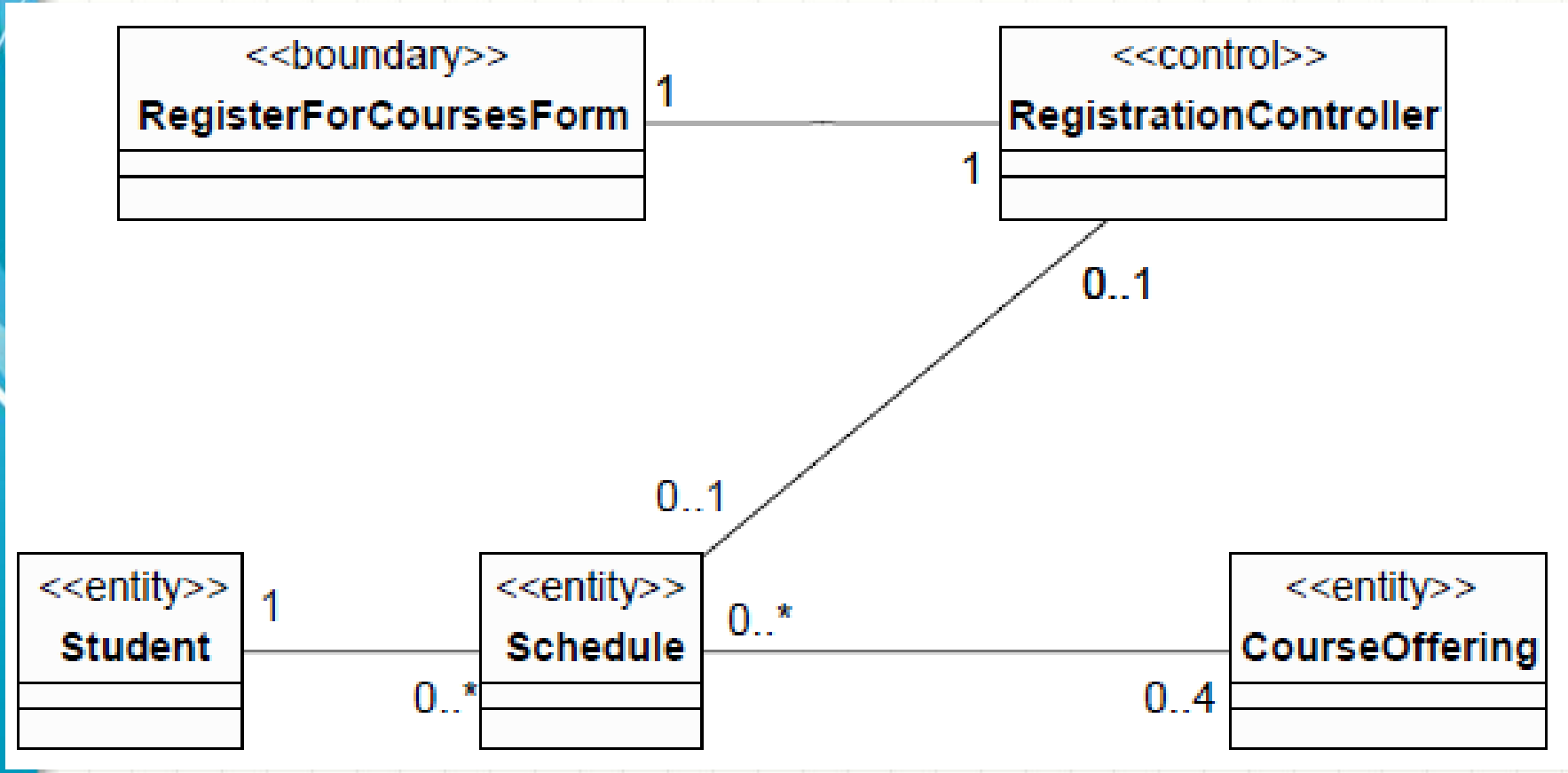
- Số lượng thể hiện của một lớp liên quan đến MỘT thể hiện thuộc về lớp khác)
- Mỗi mối quan hệ kết hợp xác định 2 bản số (ở 2 đầu của mối quan hệ)
  - Mỗi thể hiện của Professor có thể dạy nhiều Course Offering
  - Mỗi thể hiện của Course Offering, có thể không có hoặc chỉ có 1 Proffessor nhận dạy



Biểu diễn bản số

Unspecified	
Exactly One	1
Zero or More	0..*
Zero or More	*
One or More	1..*
Zero or One (optional scalar role)	0..1
Specified Range	2..4
Multiple, Disjoint Ranges	2, 4..6

Ví dụ về bản số

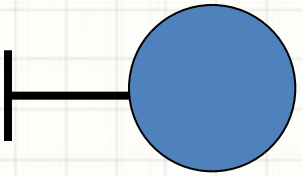


Stereotype của lớp

- Trong biểu đồ lớp, stereotype là cơ chế để phân nhóm lớp
- UML có sẵn nhiều stereotype để sử dụng
- Ba stereotype lớp cơ sở sử dụng trong pha phân tích là
  - Boundary
  - Entity
  - Control

Stereotype của lớp

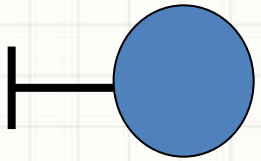
- Ba stereotype lớp cơ sở sử dụng trong pha phân tích là
  - Boundary
    - Dành cho lớp nằm trên biên hệ thống với thế giới còn lại
    - Chúng có thể là form, report, giao diện với phần cứng như máy in, scanner...
    - Khảo sát biểu đồ UC để tìm kiếm lớp biên
  - Entity
  - Control



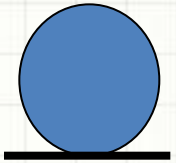
Form

Stereotype của lớp

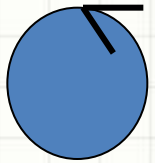
- Ba stereotype lớp cơ sở sử dụng trong pha phân tích là
  - Boundary
  - Entity
    - Lớp thực thể là lớp lưu trữ thông tin sẽ ghi vào bộ nhớ ngoài
    - Thông thường phải tạo ra bảng CSDL cho lớp loại này
  - Control
    - Có trách nhiệm điều phối hoạt động của các lớp khác
    - Thông thường mỗi UC có một lớp điều khiển
    - Nó không thực hiện chức năng nghiệp vụ nào
    - Các lớp điều khiển khác: điều khiển sự kiện liên quan đến an ninh và liên quan đến giao dịch CSDL



BoundaryClass



EntityClass



ControlClass

Ví dụ về Interface – Biểu đồ lớp chi tiết

Thông tin đăng nhập được lưu trong tài khoản (mã số, usename, password). Mỗi nhân viên được cấp một tài khoản để đăng nhập. Mỗi khi có một nhân viên mới, phòng hành chính sẽ nhập thông tin và cấp tài khoản cho nhân viên thông qua kích hoạt Form cập nhật. Các thông tin cần nhập bao gồm: username, họ tên, ngày sinh, địa chỉ, điện thoại, email. Dữ liệu được cho là hợp lệ nếu hệ thống kiểm tra mà không trùng usename.

Áp dụng biểu đồ lớp

- Mô tả khung nhìn tĩnh của hệ thống (static view)
  - Từ vựng của một hệ thống
  - Sự cộng tác (collaboration)
  - Lược đồ CSDL logic
- Biểu diễn các yêu cầu của hệ thống
- Minh họa cách đọc và diễn giải biểu đồ lớp

Tips

- Không cố gắng sử dụng tất cả các ký hiệu khác nhau
- Không vẽ mô hình cho mọi thứ, tập trung vào các thông tin quan trọng

2.5 Biểu đồ trạng thái

- Dựa trên các lớp đã tìm ra trong mô hình lớp, ***biểu đồ trạng thái*** sẽ được xây dựng cho mỗi lớp. Biểu đồ này sẽ cho biết các trạng thái có thể có của các đối tượng lớp đó và các điều kiện chuyển đổi giữa các trạng thái.
- Có hai dạng biểu đồ trạng thái:

- Biểu đồ trạng thái cho một use case: mô tả các trạng thái và chuyển tiếp trạng thái của một đối tượng thuộc một lớp nào đó trong hoạt động của một use case cụ thể.
- Biểu đồ trạng thái hệ thống mô tả tất cả các trạng thái của một đối tượng trong toàn bộ hoạt động của cả hệ thống.

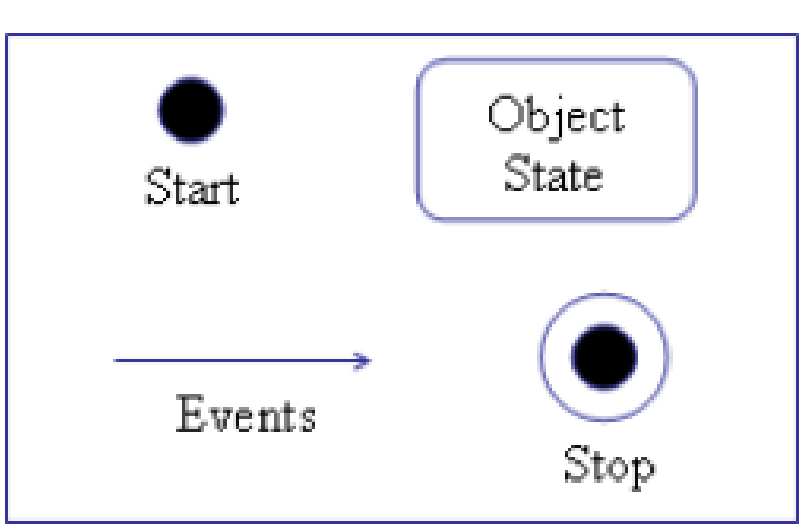
Ví dụ về trạng thái

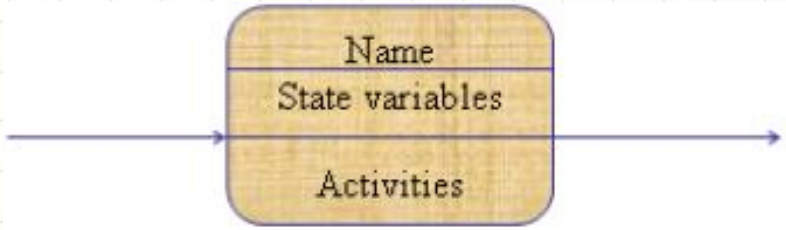
- Hóa đơn (đối tượng) đã được trả tiền (trạng thái).
- Chiếc xe ô tô (đối tượng) đang đứng yên (trạng thái).
- Động cơ (đối tượng) đang chạy (trạng thái).
- Jen (đối tượng) đang đóng vai trò người bán hàng (trạng thái).
- Kate (đối tượng) đã lấy chồng (trạng thái).

→ Một đối tượng sẽ thay đổi trạng thái khi có một việc nào đó xảy ra

Các trạng thái

- Trạng thái ban đầu
- Một số trạng thái ở giữa
- Một hoặc nhiều trạng thái kết thúc
- Sự biến đổi giữa các trạng thái
- Những sự kiện gây nên sự biến đổi từ một trạng thái này sang trạng thái khác



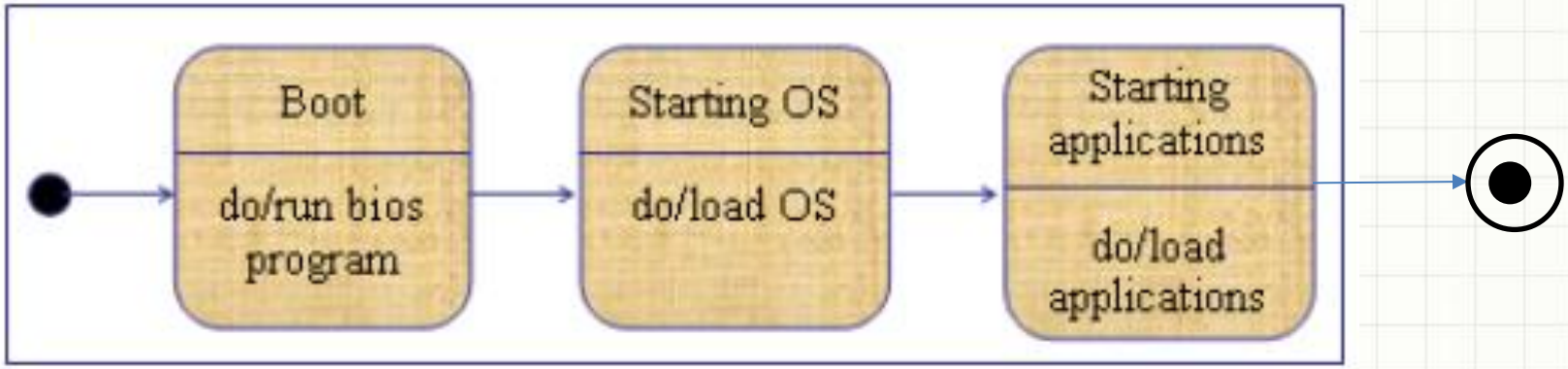


2.5.1 Xây dựng biểu đồ trạng thái

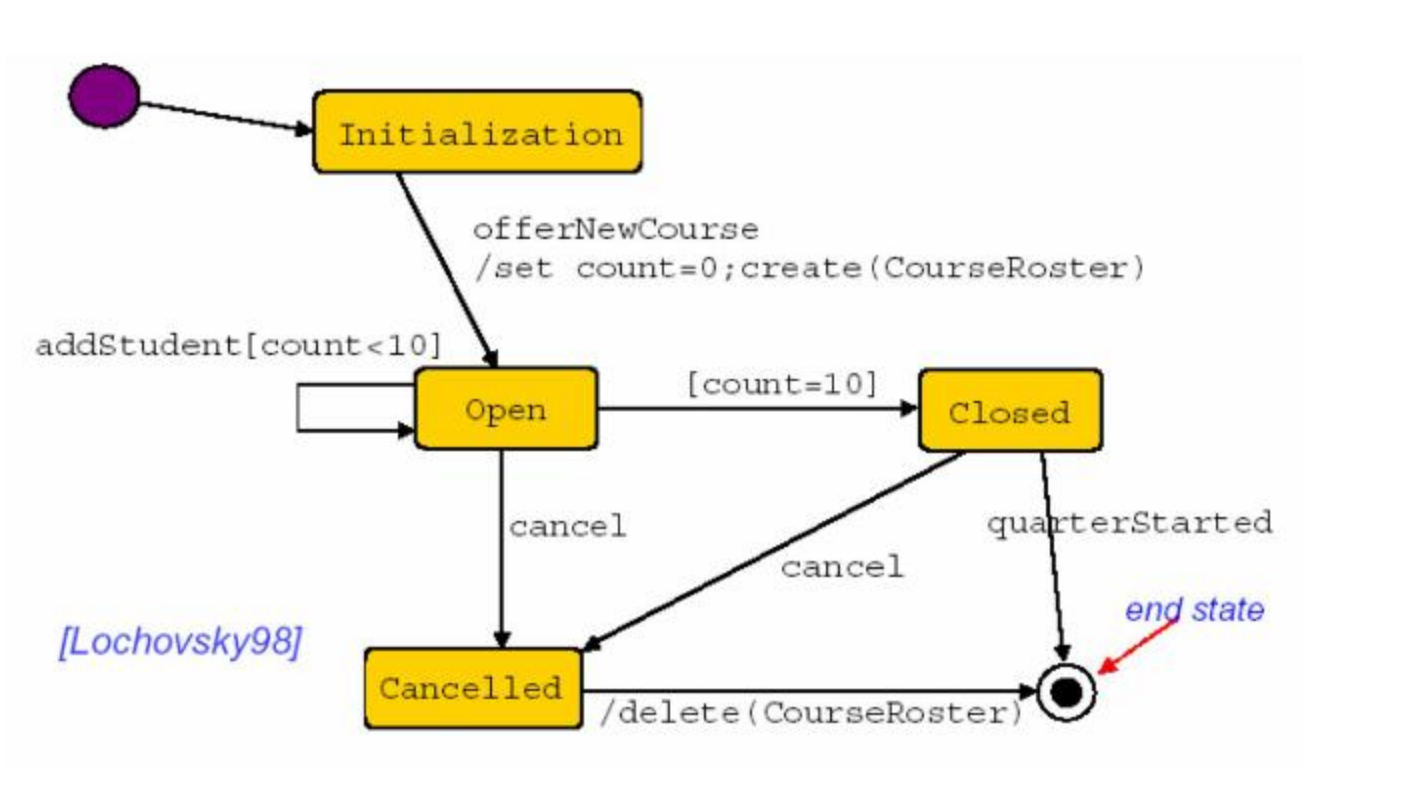
Biểu đồ trạng thái cho mỗi lớp được xây dựng theo các bước sau:

- Bước 1: Nhận biết các trạng thái và sự kiện
- Bước 2: Xây dựng biểu đồ
- Bước 3: Hiệu chỉnh biểu đồ

Ví dụ biến đổi trạng thái không có sự kiện



Biểu đồ trạng thái: Course – Khóa học



Bài tập

- Vẽ biểu đồ chuyển trạng thái của nồi cơm điện
- Vẽ biểu đồ chuyển trạng thái của máy điều hoà không khí

Chương 3. Thiết kế hệ thống

Số giờ:

Thời gian: Từ ngày đến ngày

I. MỤC TIÊU, YÊU CẦU

1. Mục tiêu: Sau khi học xong bài này người học có năng lực: thiết kế được hệ thống ứng dụng, xây dựng được các mô hình phù hợp

2. Yêu cầu: sử dụng phần mềm để xây dựng các mô hình

3. Dụng cụ, trang thiết bị: phòng học đủ máy tính, máy chiếu hoặc ti vi, Internet, bảng, phấn.

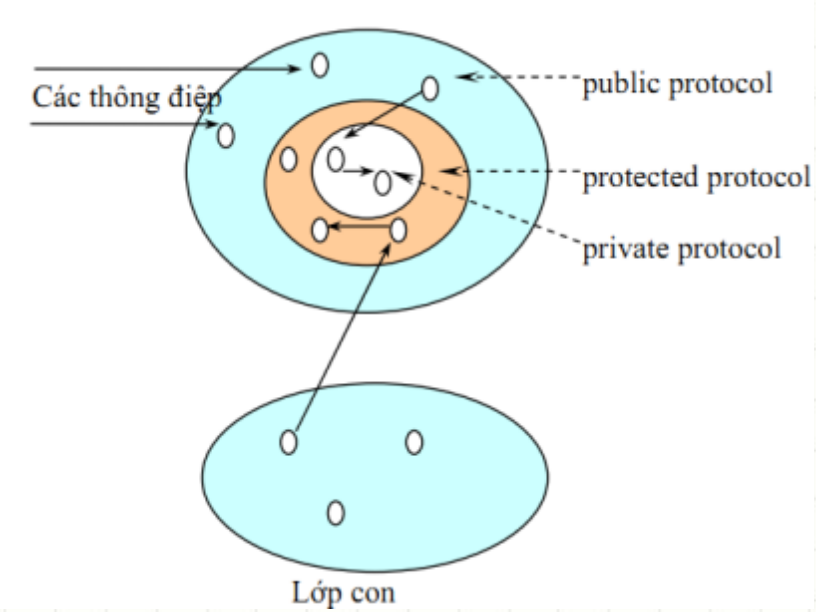
Chương 3. Thiết kế hệ thống

3.1. Thiết kế lớp

- Một tiến trình thiết kế lớp bao gồm
  - Tinh chế thuộc tính
  - Tinh chế hành vi (method) và nghi thức (protocol)
  - Tinh chế quan hệ giữa các lớp
  - Tinh chế sự phân cấp và thiết kế sự kế thừa

Phạm vi ảnh hưởng của lớp

- public protocol: định nghĩ các hành vi trạng thái của lớp
- private protocol: dùng để xác định các đặc trưng của lớp chỉ được truy cập bởi chính lớp đó
- Protected protocol: xác định đặc trưng của một lớp có thể sử dụng bởi chính nó và lớp con của nó.



Tinh chế thuộc tính

- Trong giai đoạn thiết kế thông tin chi tiết của thuộc tính cần được thêm vào
- Các loại thuộc tính
  - Đơn trị: hoten, ngaysinh
  - Đa trị: sdt, diachi (ví dụ diachi[3]: string)
  - Tham chiếu đối tượng khác đáp ứng các trách nhiệm của nó

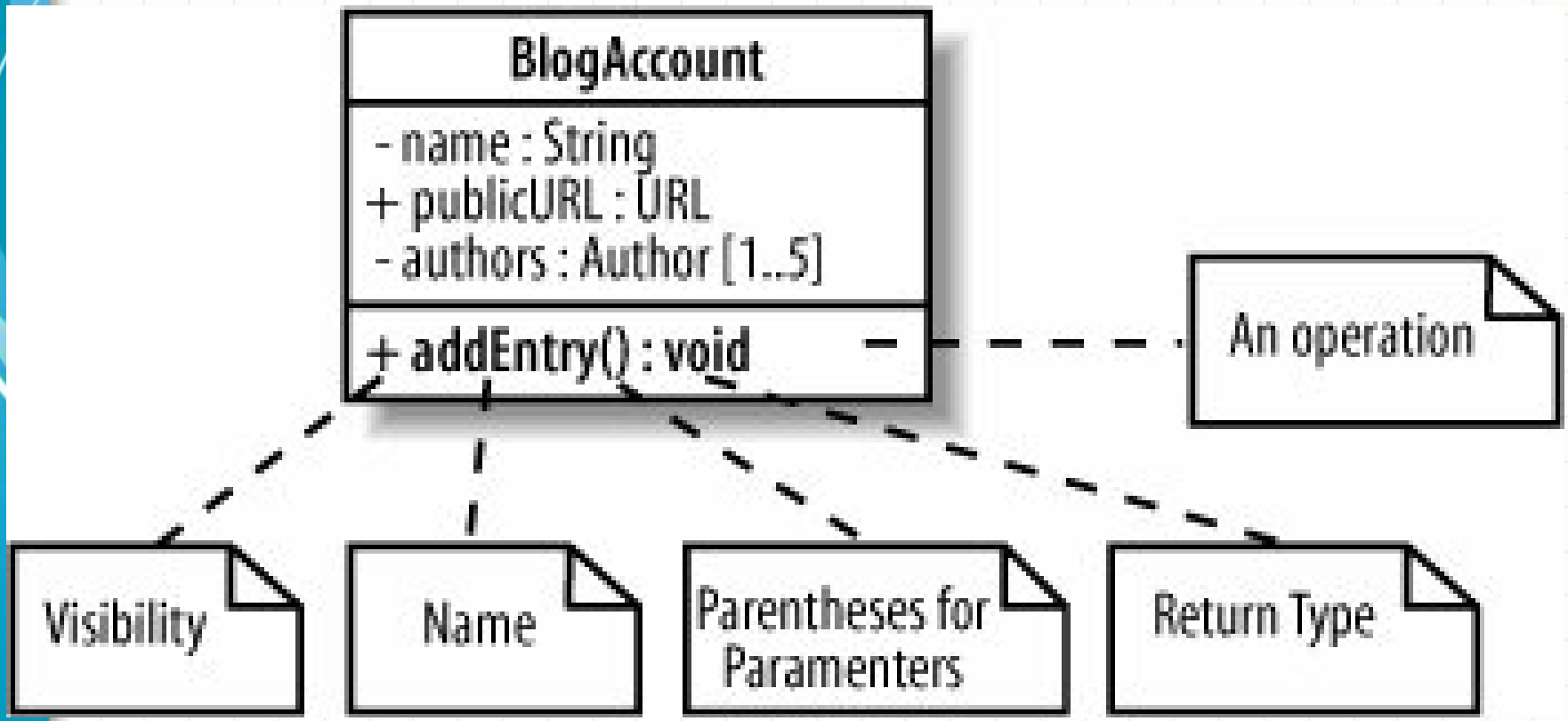
Tinh chế thuộc tính

- Hiển thị thuộc tính

  <Phạm vi> <tên> : <kiểu thuộc tính> = <giá trị khởi tạo>

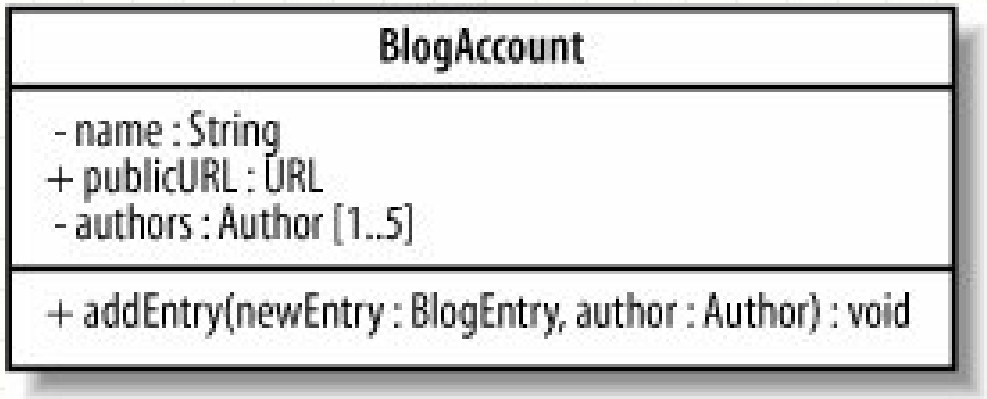
- <phạm vi> sẽ là:
  - + : toàn cục (public protocol)
  - # : bảo vệ (protected protocol)
  - - : cục bộ (private protocol)
- <kiểu thuộc tính> là một đặc tả cài đặt thuộc tính độc lập ngôn ngữ.
- <giá trị khởi tạo> là một biểu thức độc lập ngôn ngữ xác định giá trị khởi tạo khi một đối tượng được tạo mới

Biểu diễn lớp trong UML



Biểu diễn các tham số trong operation





Ví dụ: Quản lý thư viện.

Hệ thống này được thể hiện qua các yêu cầu sau:

- Tài liệu trong thư viện bao gồm: sách, báo, tạp chí ... được mô tả chung

gồm các thuộc tính: tên tài liệu, tác giả, nhà xuất bản, năm xuất bản, số lượng hiện có.

- Đối với các bạn đọc: thực hiện các thao tác tìm tài liệu, mượn, trả tài liệu

và xem xét các thông tin về tài liệu mà mình đang mượn. Việc tìm kiếm tài liệu được thực hiện trực tiếp qua mạng. Tuy nhiên, giao dịch mượn và trả sách phải thực hiện trực tiếp tại thư viện.

- Quá trình mượn và trả tài liệu thông qua một *thẻ mượn* ghi đầy đủ nội

dung liên quan đến bạn đọc và tài liệu được mượn; thời gian bắt đầu mượn và thời hạn phải trả.

- Đối với người quản lý thư viện (thủ thư): được phép cập nhật các thông

tin liên quan đến tài liệu và bạn đọc.

Hãy xây dựng biểu đồ lớp trong pha phân tích

Xây dựng biểu đồ lớp trong pha phân tích

Bước 1: Xác định các lớp từ các use case

- Nghiên cứu kỹ tất cả **các use case** và scenario để tìm ra các danh từ có vai trò nào đó trong các scenario . Các danh từ này sẽ trở thành các lớp ứng cử viên.
- Loại bỏ các lớp ứng cử viên không thích hợp. Các danh từ không thích hợp sau:

  + Lớp dư thừa: do có hai hay nhiều danh từ cùng chỉ một thực thể

  +Danh từ không hích hợp: đó là các danh từ không liên quan đến phạm vi của bài toán.

  +Danh từ mô tả những lớp không rõ ràng: đó là các danh từ hoặc không biểu diễn một thực thể cụ thể hoặc các khái niệm không rõ nghĩa.

  +Các danh từ chỉ là một vai trò (role) trong mối quan hệ với một lớp khác.

  +Các danh từ biểu diễn các công cụ xây dựng phần mềm hoặc các thuật ngữ trong lập trình hay thuật toán (ví dụ stack, list, array, …).

Xây dựng biểu đồ lớp trong pha phân tích

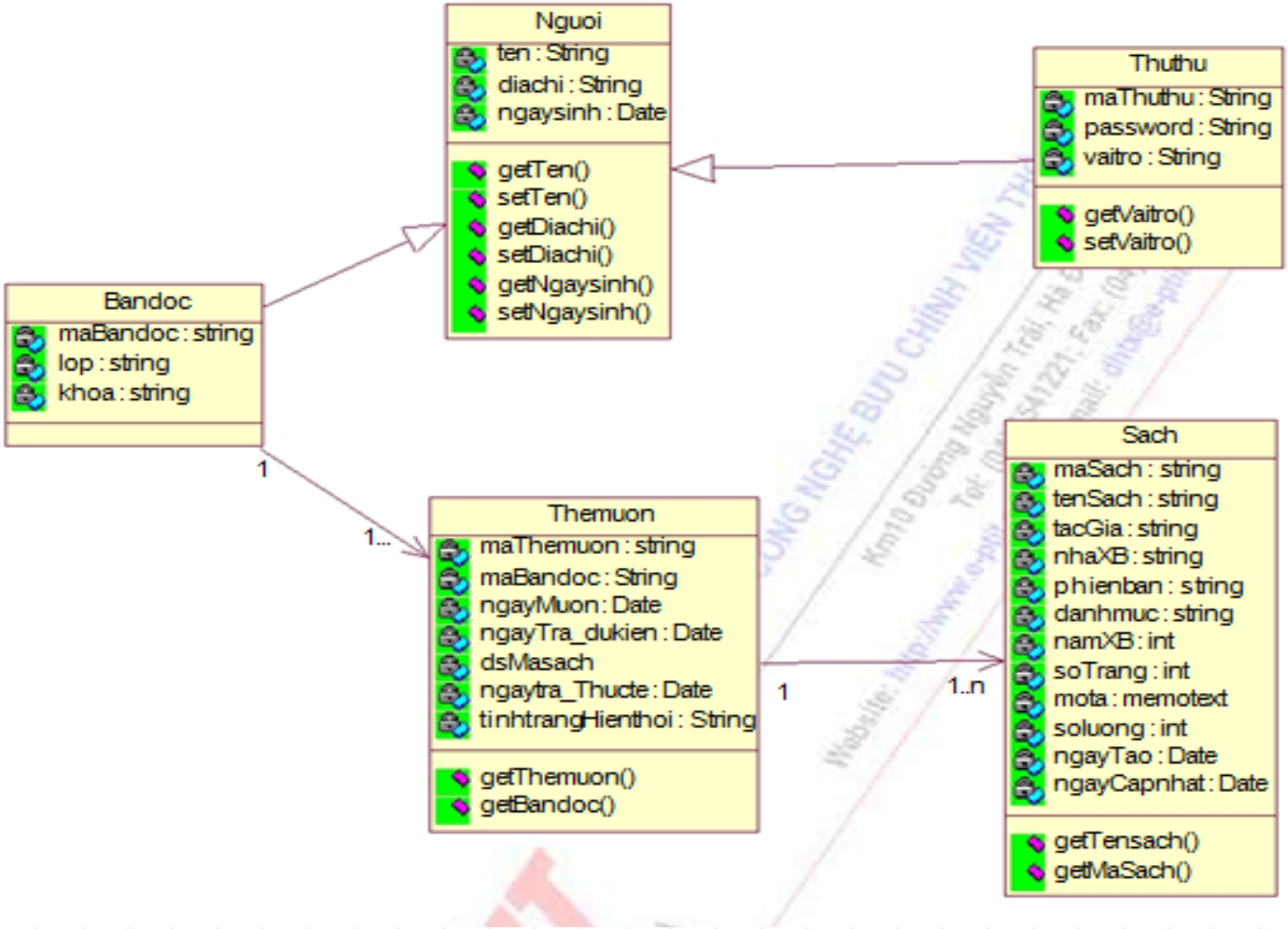
Bước 2: Xác định các thuộc tính và một số phương thức cơ bản

Dựa trên tập các lớp đã được xác định, người phát triển hệ thống tiếp tục nghiên cứu kỹ các use case và scenario và trả lời các câu hỏi sau:

- Với mỗi lớp, những danh từ nào mô tả thông tin của lớp đó. Trả lời câu hỏi này sẽ giúp ta tìm ra các thuộc tính.
- Những thông tin nào của lớp thực sự liên quan đến lĩnh vực quan tâm của hệ thống. Trả lời câu hỏi này giúp ta loại các thuộc tính không cần thiết.
- Những thông tin nào là thông tin riêng của lớp (các thuộc tính private), những thông tin nào có thể chia sẻ trong mối quan hệ với lớp khác (các thuộc tính protected hoặc public).

Tiếp theo, người phát triển hệ thống xem xét các động từ đi kèm với các danh từ biểu diễn lớp trong scenario và xem xét xem các động từ ấy có trở thành các phương thức được hay không. Tuy nhiên, trong pha phân tích, chúng ta chỉ có thể xác định một số phương thức dễ nhận thấy và cũng chưa cần xác định chi tiết giá trị trả về cũng như các tham số. Các thông tin này sẽ được cụ thể hóa trong pha thiết kế.

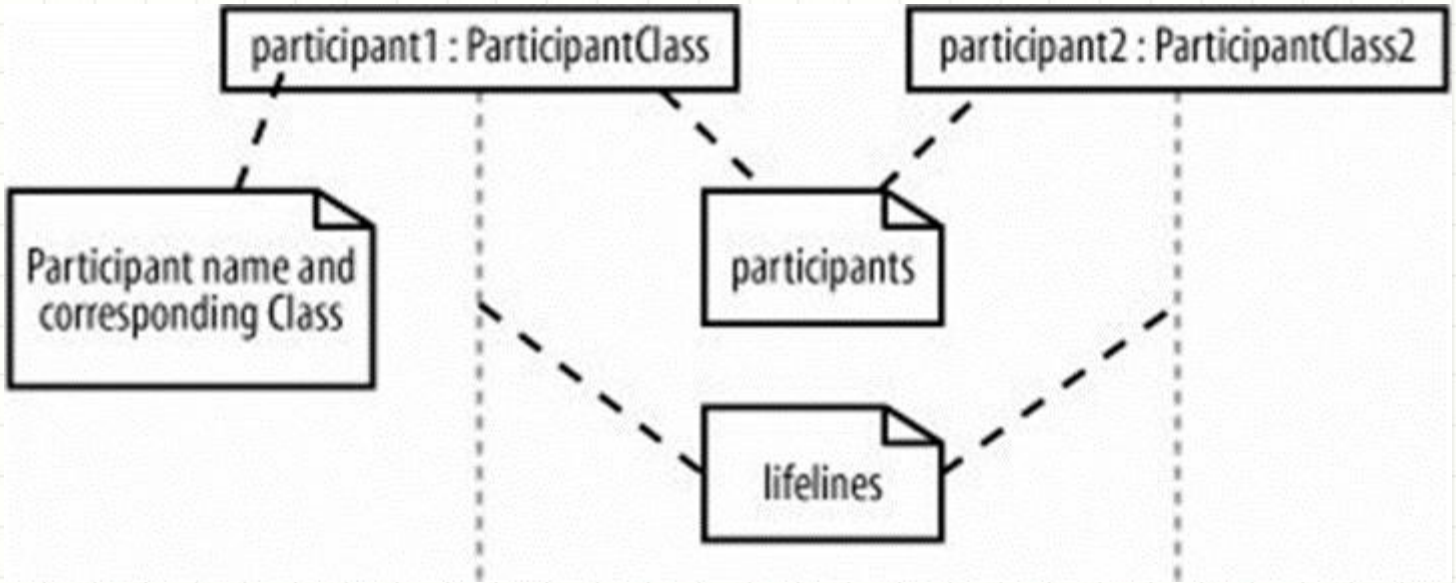
Biểu đồ lớp bước đầu của hệ quản lý thư viện được biểu diễn như trong Hình sau. Các lớp Bạn đọc và Thủ thư được kế thừa từ một lớp chung tên là Người. Tại một thời điểm, một bạn đọc có tương ứng một Thẻ mượn. Một thẻ mượn có thể cho mượn cùng một lúc một hoặc nhiều cuốn sách.


Nguoi
ten : String
diachi : String
ngaysinh : Date
getTen()
setTen()
getDiachi()
setDiachi()
getNgaysinh()
setNgaysinh()
Thuthu
maThuthu : String
password : String
vaitro : String
getVaitro()
setVaitro()
Bandoc
maBandoc : string
lop : string
khoa : string
1
1...
Themuon
maThemuon : string
maBandoc : String
ngayMuon : Date
ngayTra_dukien : Date
dsMasach
ngaytra_Thucte : Date
tinhtrangHienthoi : String
getThemuon()
getBandoc()
1
1..n
Sach
maSach : string
tenSach : string
tacGia : string
nhaXB : string
phienban : string
danhmuc : string
namXB : int
soTrang : int
mota : memotext
soluong : int
ngayTao : Date
ngayCapnhat : Date
getTensach()
getMaSach()

3.3. Biểu đồ tuần tự - Sequence

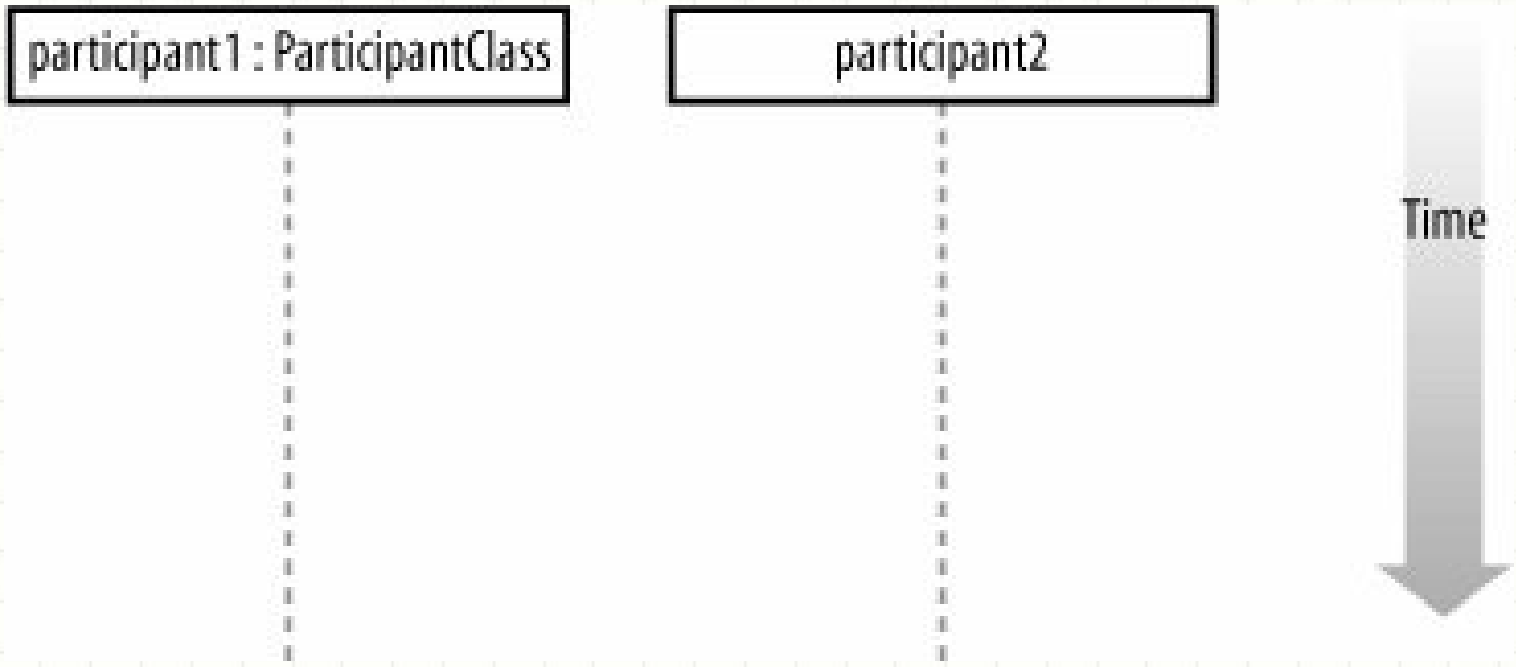
- Minh họa cách **các đối tượng** tương tác với nhau.
- Tập trung vào các **chuỗi thông điệp**, có nghĩa là các thông điệp được gửi và nhận giữa một loạt các đối tượng như thế nào.
- Biểu đồ tuần tự có hai trục: trục nằm dọc chỉ thời gian, trục nằm ngang chỉ ra một tập hợp các đối tượng.
- Một biểu đồ tuần tự cũng nêu bật sự tương tác trong một cảnh kịch (scenario) – một sự tương tác sẽ xảy ra tại một thời điểm nào đó trong quá trình thực thi của hệ thống.

- Participant: bộ phận tham gia vào các tương tác



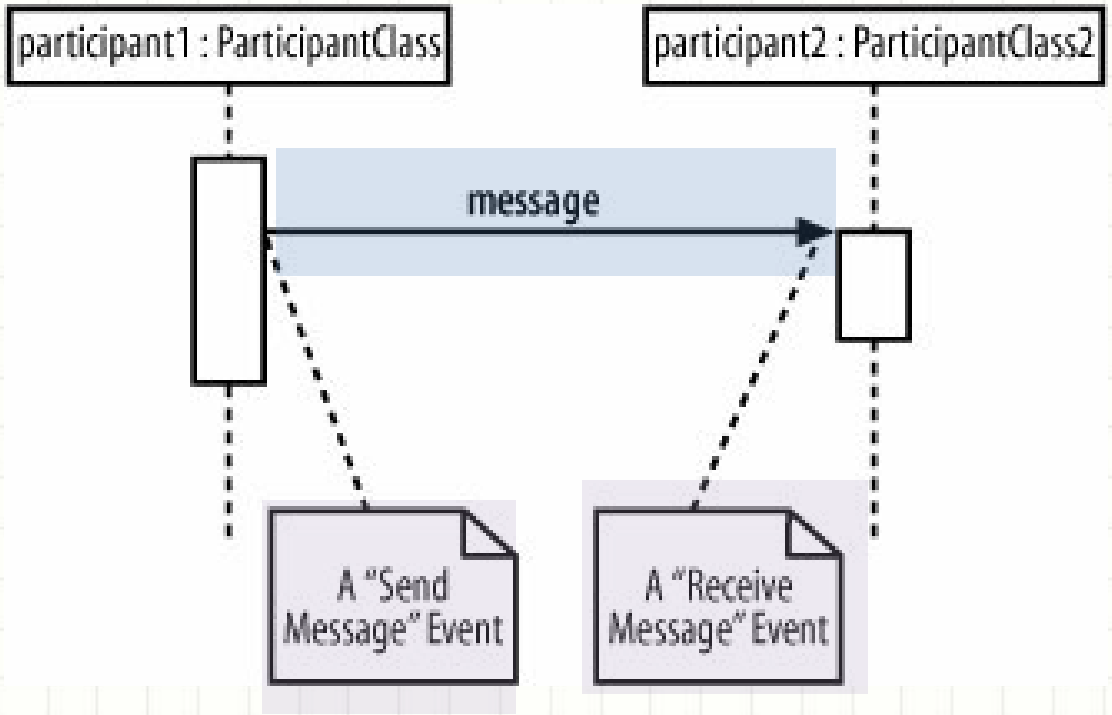
Thời gian trong biểu đồ tuần tự

- Tăng dần từ trên xuống dưới
- Chỉ quan tâm đến thứ tự của các tương tác, không quan tâm đến mức chi tiết hơn

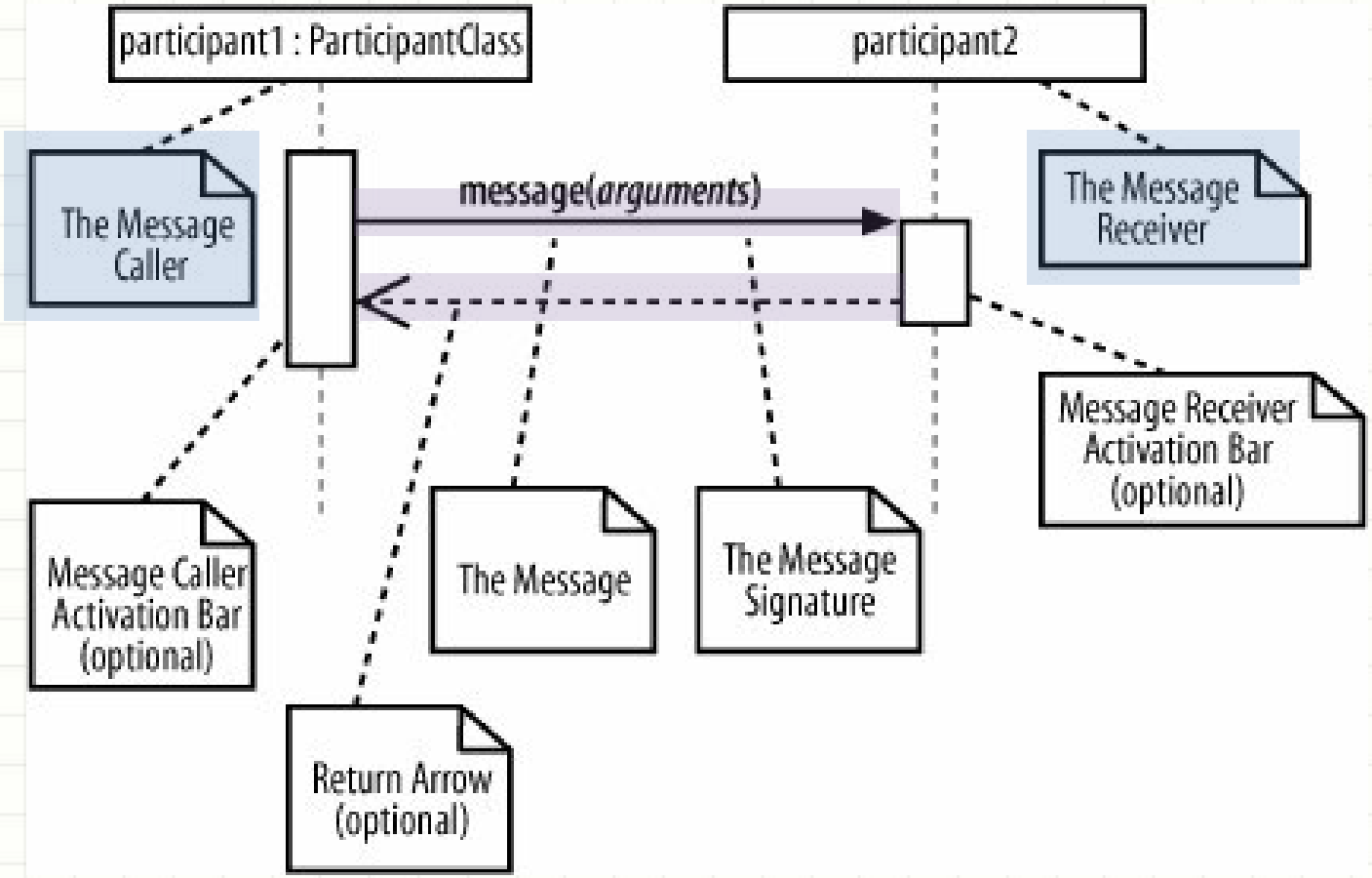


Event, Signal, và Message

- Truyền thông điệp giữa các participant



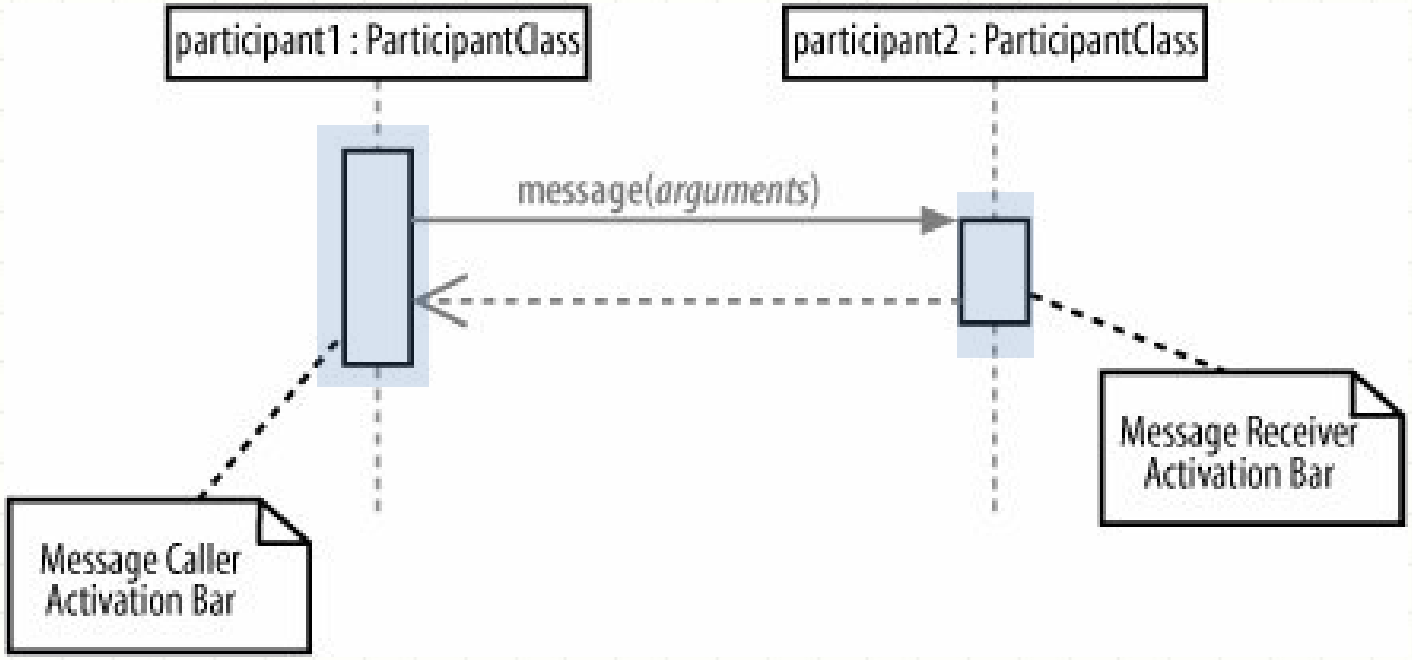
Event, Signal và Message (tt)



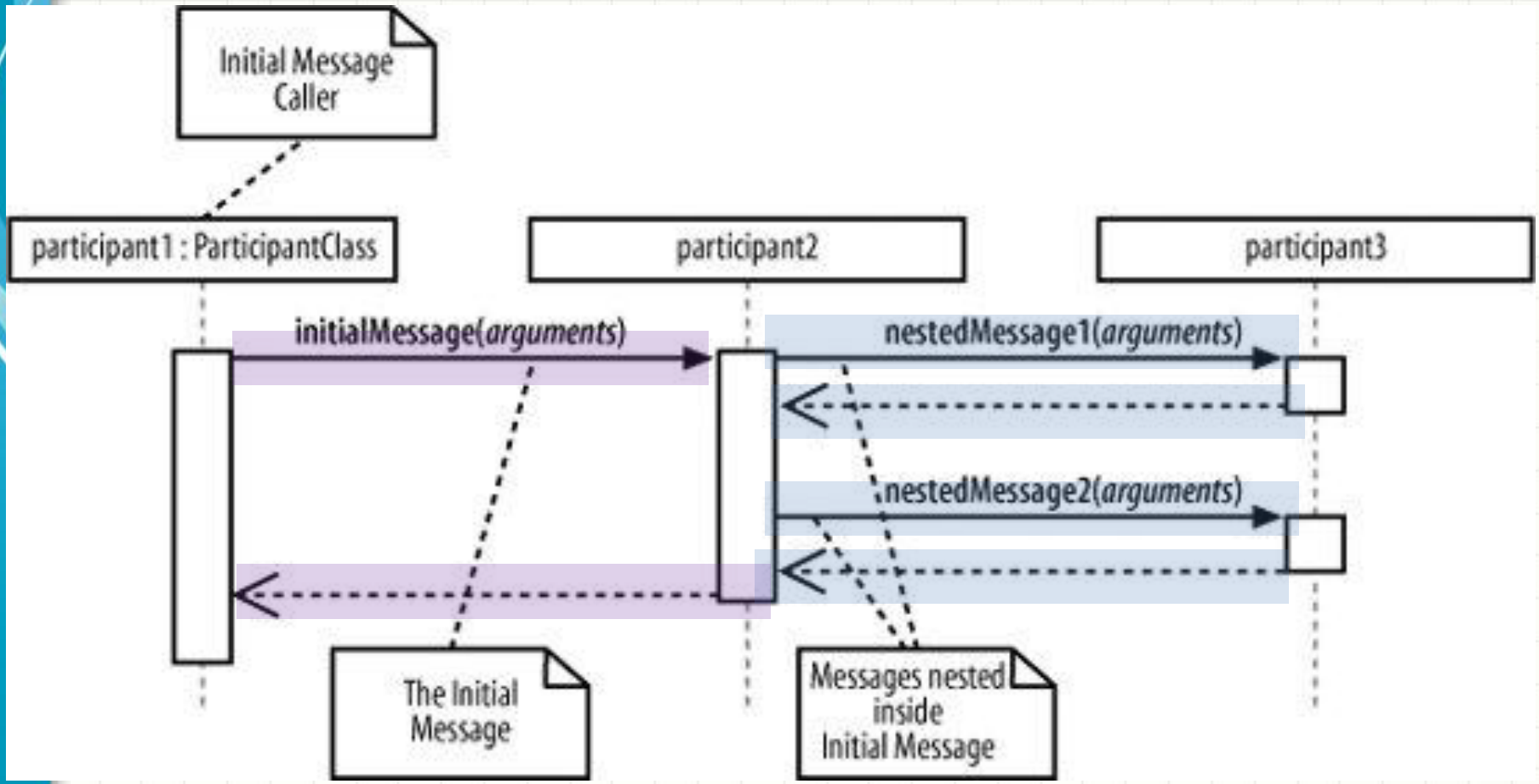
Demo: Bổ sung các message giữa các participant

Activation bar

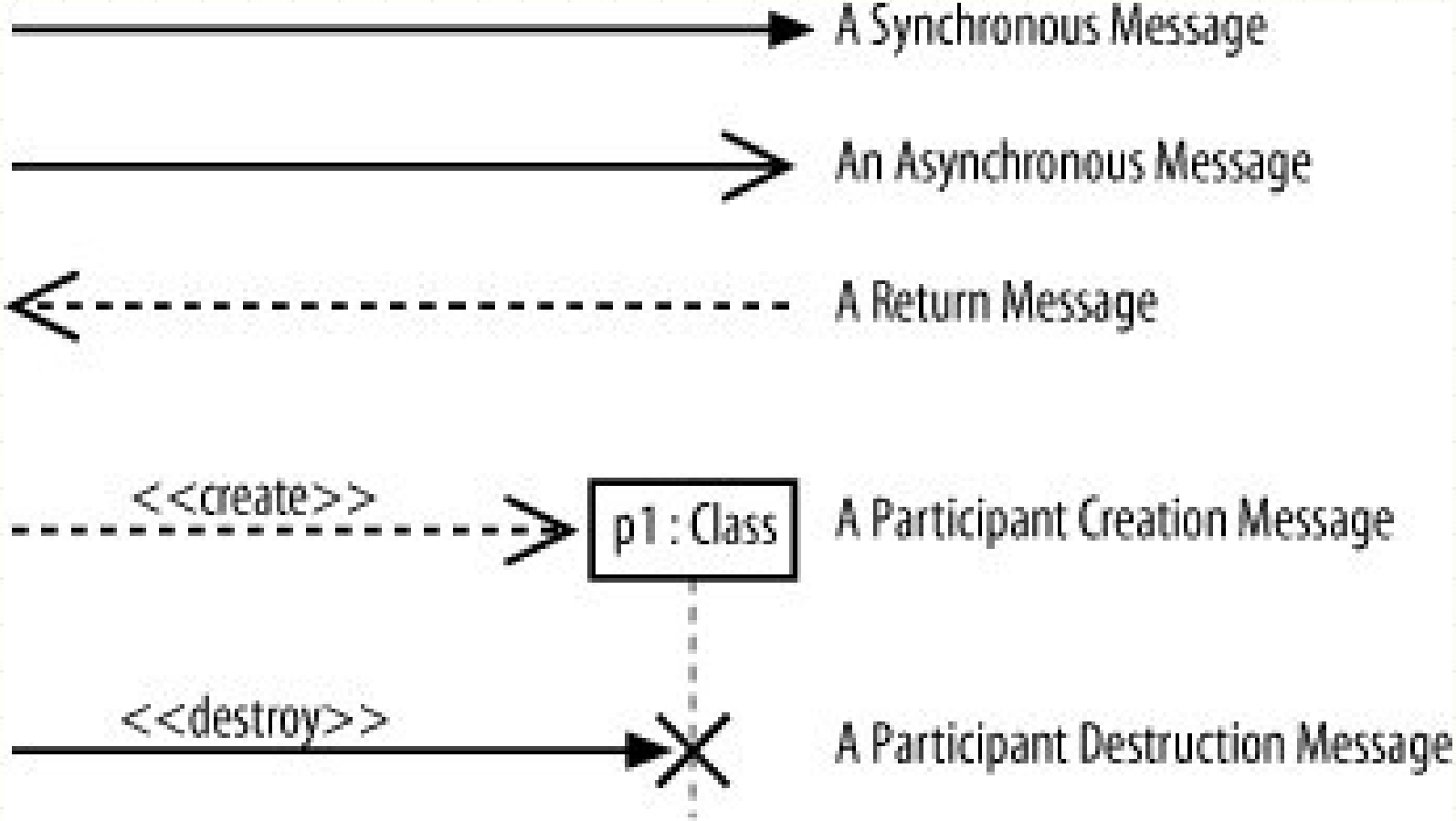
- Biểu diễn participant đang hoạt động trong đoạn tương ứng



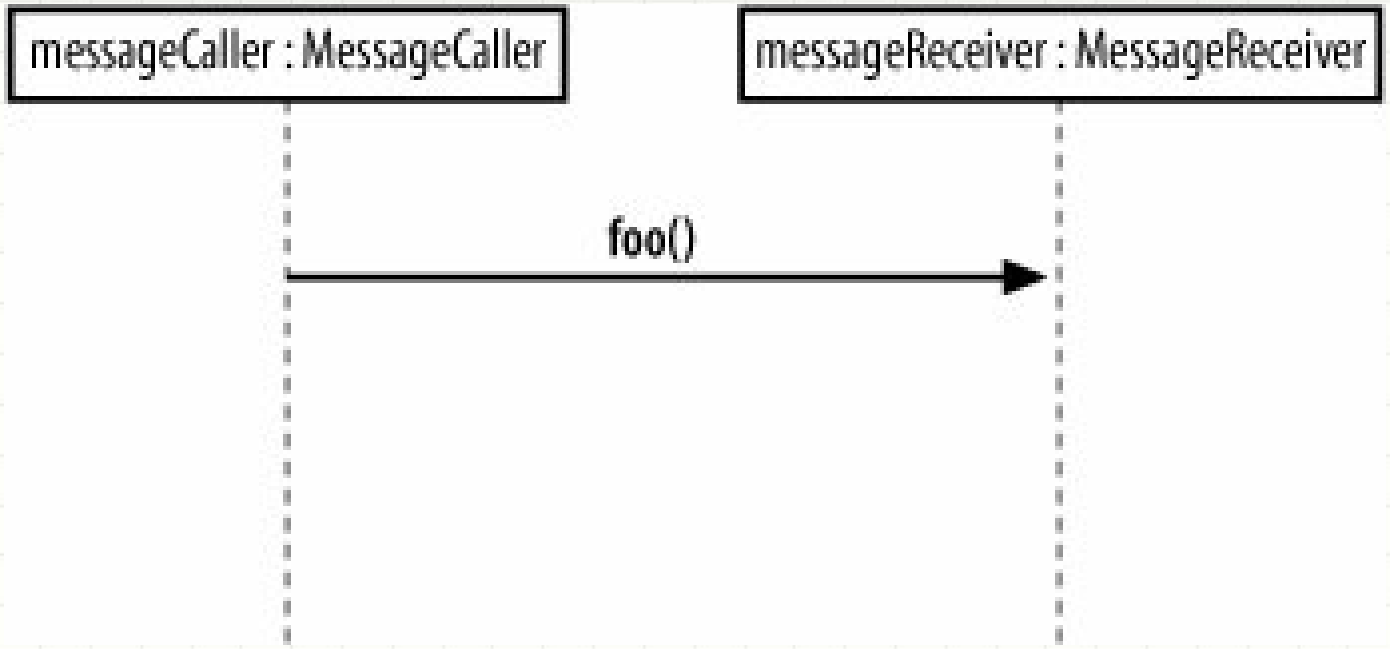
Thông điệp lồng nhau



Các kiểu truyền thông điệp

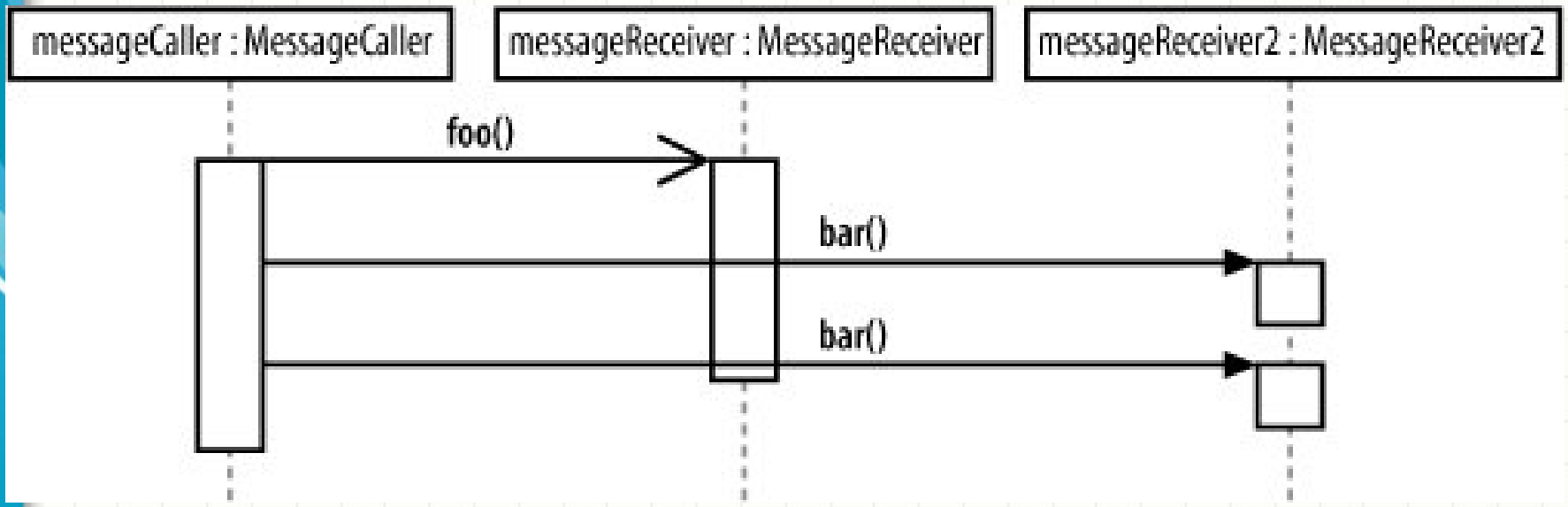


Thông điệp đồng bộ



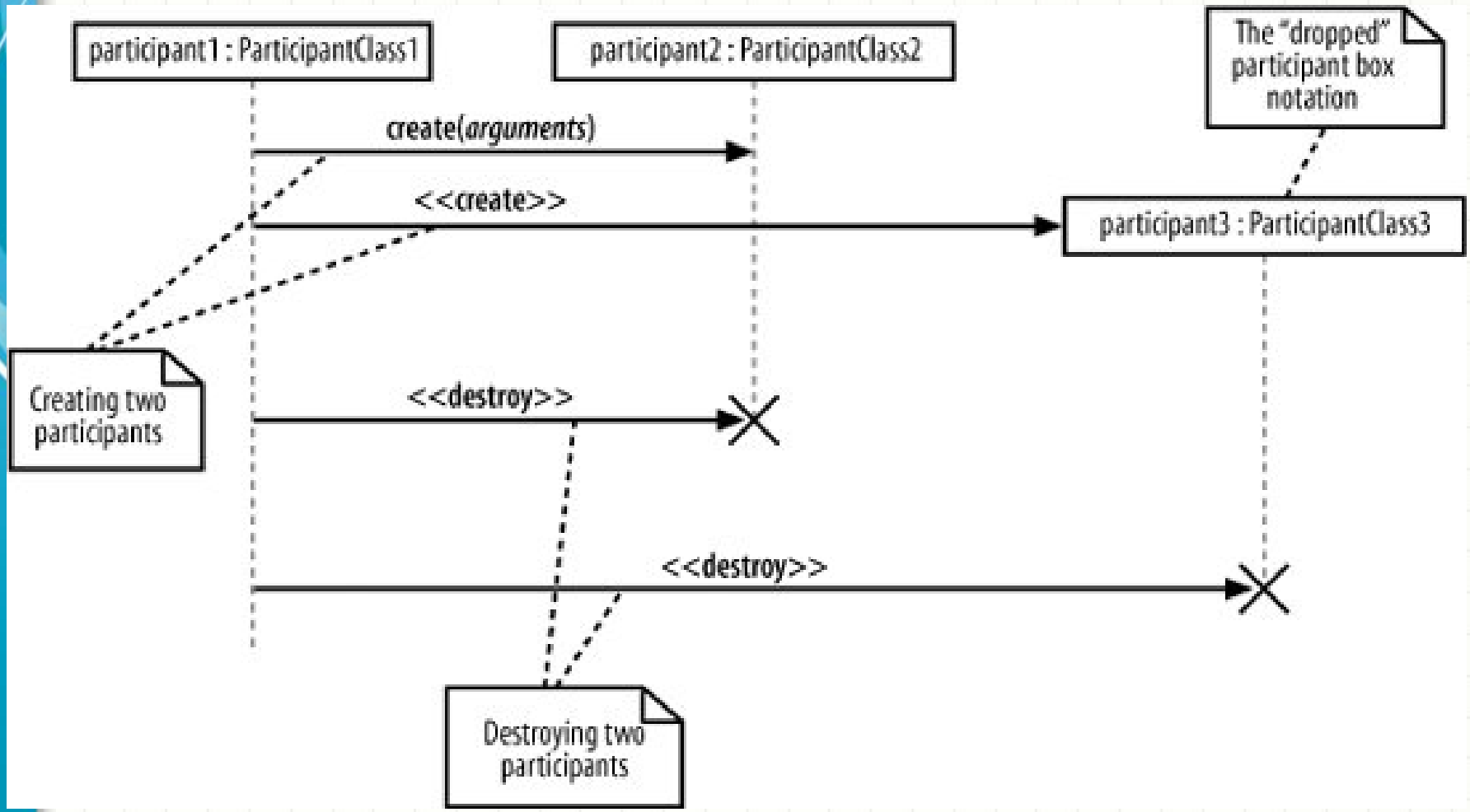
Thông điệp đồng bộ cần có một request trước hành động tiếp theo.

Thông điệp bất đồng bộ



Thông điệp không đồng bộ không cần có một request trước hành động tiếp theo

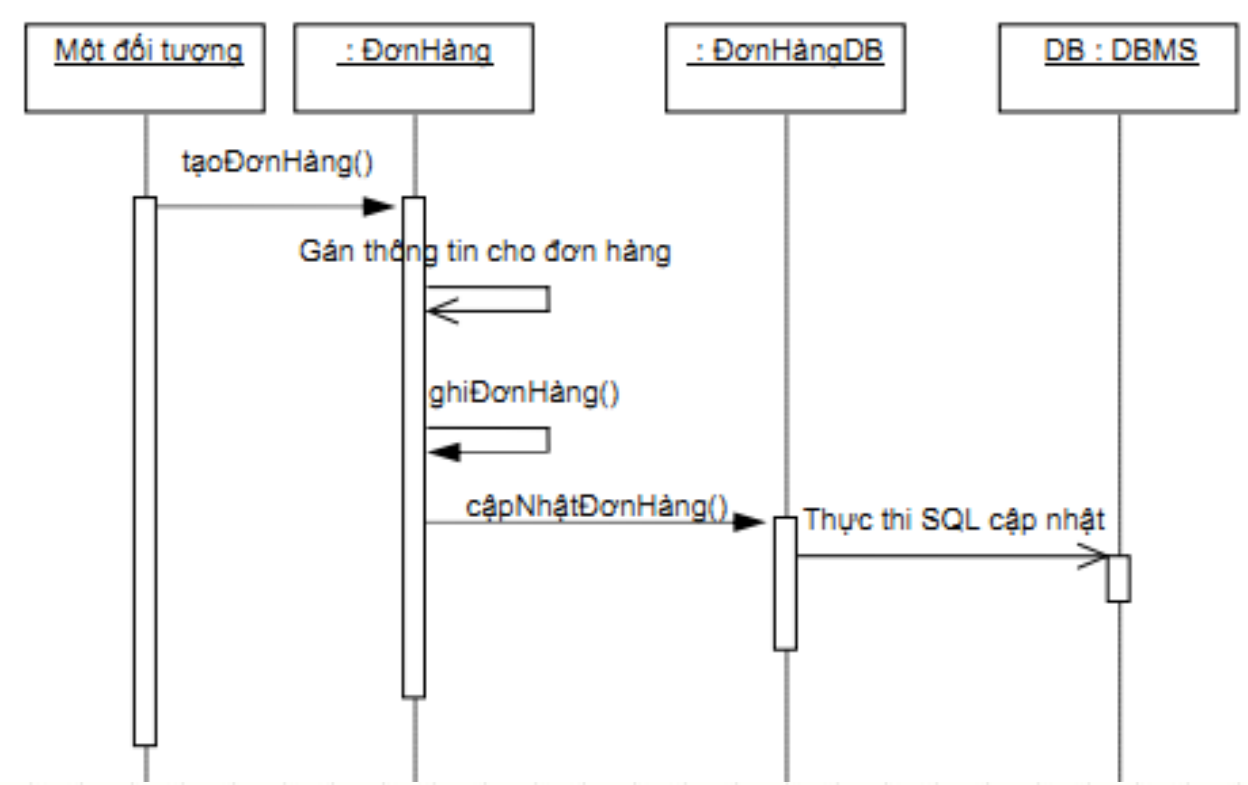
Tạo và huỷ đối tượng



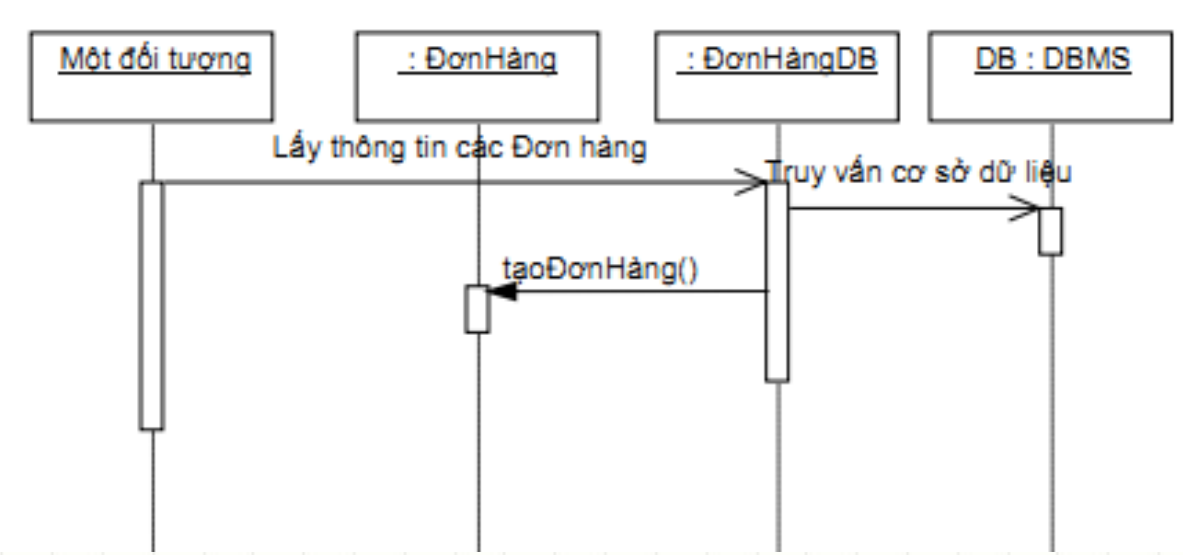
Các đối tượng xuất hiện trên biểu đồ tuần tự.

- Tác nhân
- Giao diện (form)
- Class (object, đối tượng)
- Database

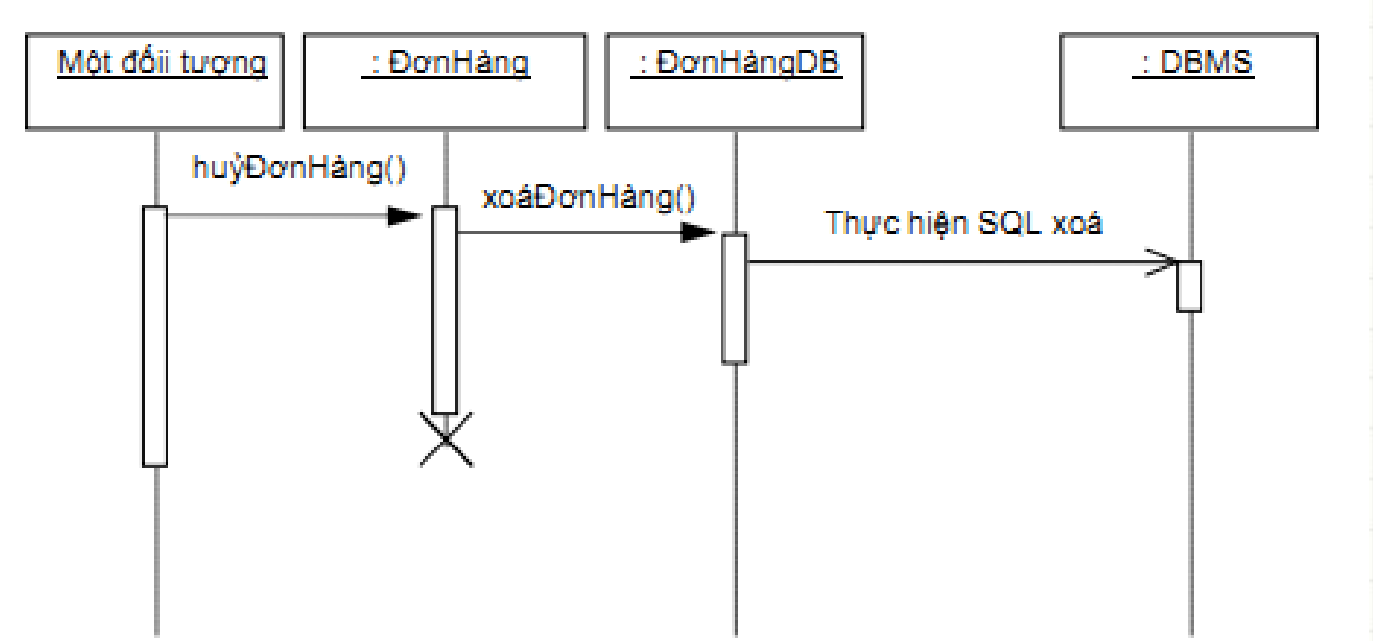
Xây dựng biểu đồ tuần tự cho UC Cập Nhật Hóa Đơn



Đọc đối tượng

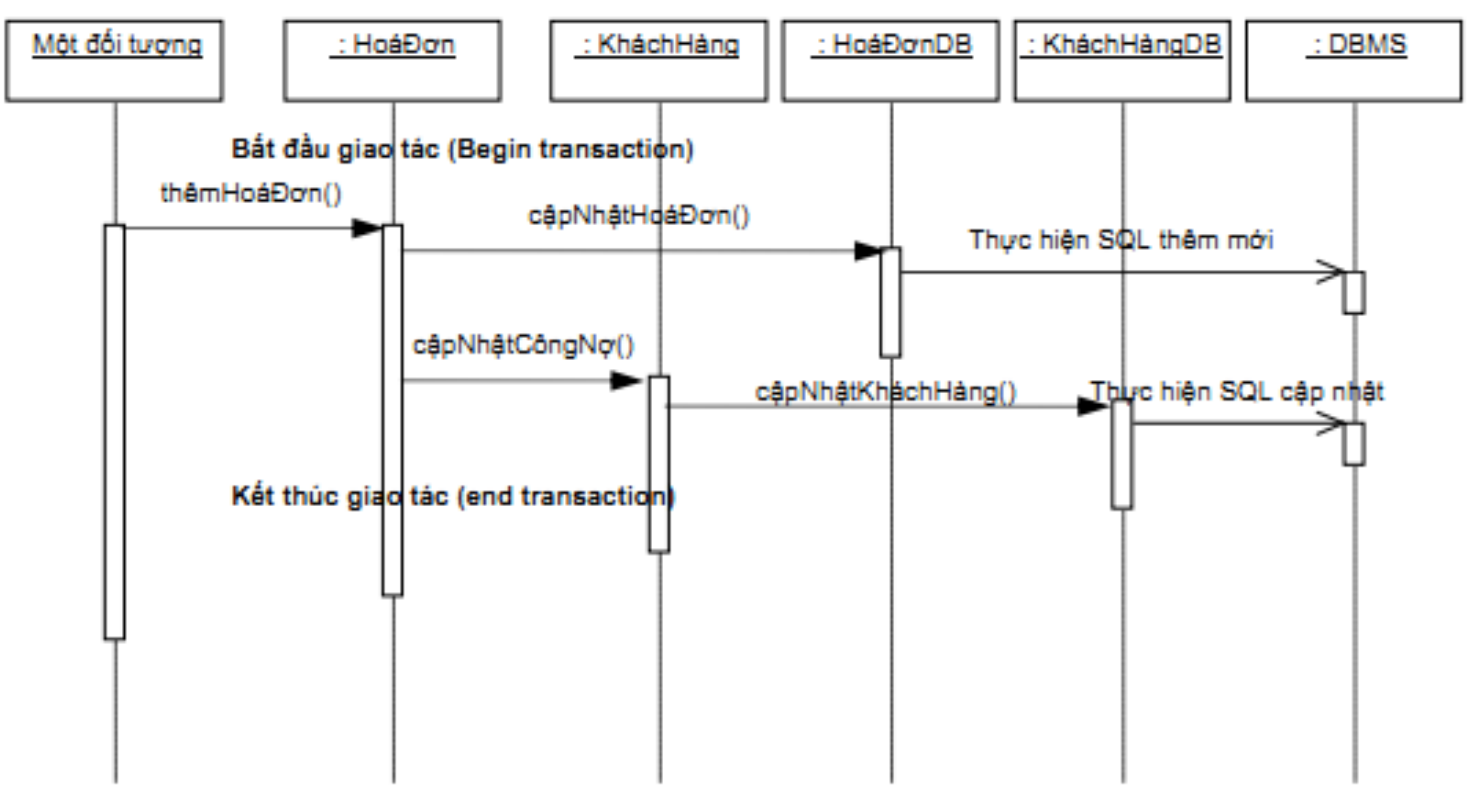


Xóa đối tượng



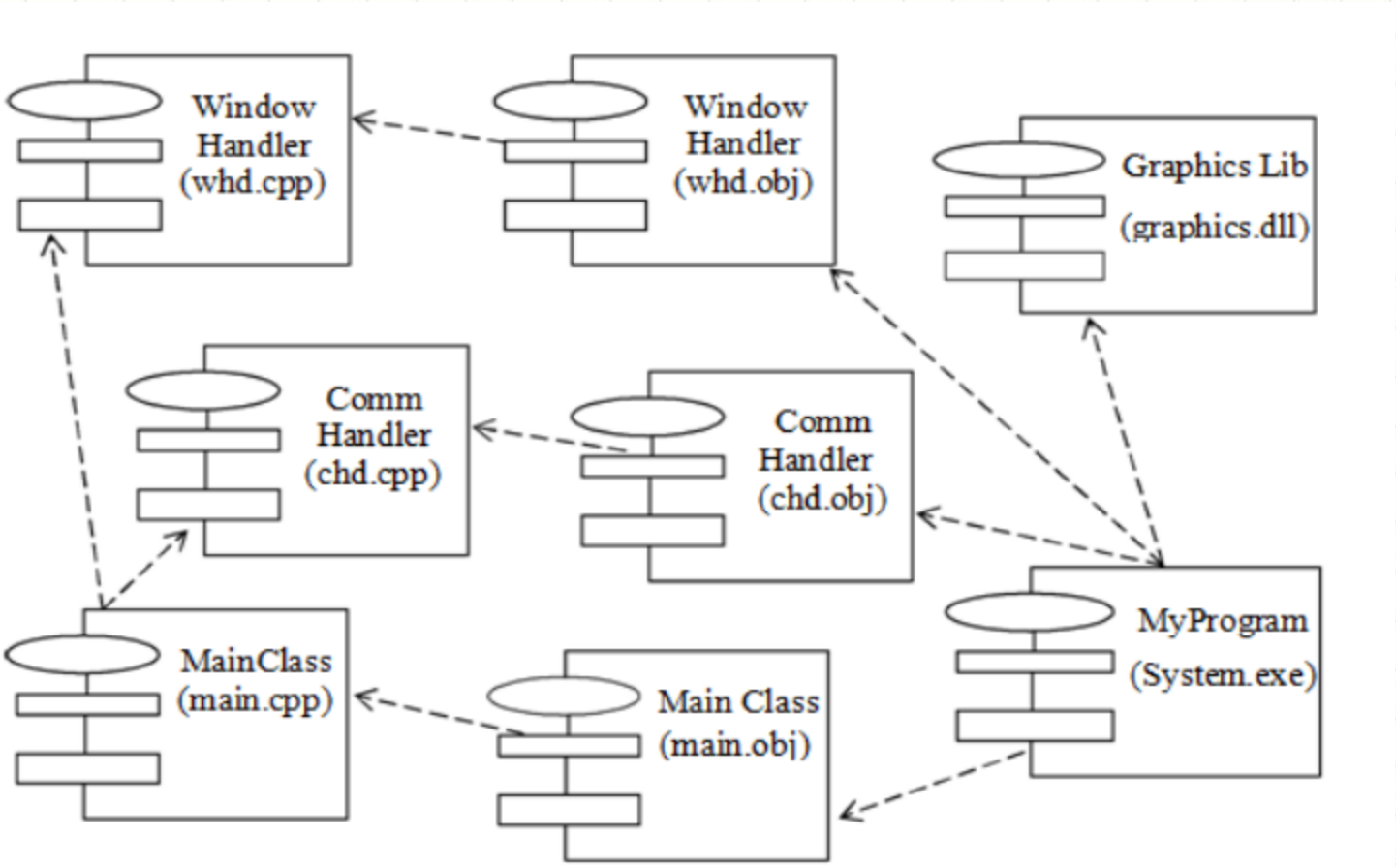
Quản lý giao tác

- Một giao tác xác định một tập các phép toán cập nhật trong cơ sở dữ liệu thành một đơn vị nhằm đáp ứng cho một yêu cầu cập nhật nghiệp vụ



3.4. Biểu đồ thành phần

- *Biểu dồ thành phần (Component Diagram) là biểu dồ mô tả các thành phần và sự phụ* thuộc của chúng trong hệ thống. Các thành phần của hệ thống có thể là:
  - *Thành phần mã nguồn, có ý nghĩa vào thời điểm dịch chuong trình. Thông* thường nó là tập các chương trình cài đặt các lớp. Trong C++, mỗi tệp .cpp và .h là một thành phần. Trước khi phát sinh mã chương trình, phải thực hiện ánh xạ từng tệp vào thành phần tuong ứng, thông thuường mỗi lớp được ánh xạ vào hai tệp (.cpp, và .h).
  - Thành phần mã nhị phân là mã trình nhị phân được dịch từ mã chương trình nguồn. Nó có thể là tệp mã đích (.obj), tệp thư viện tĩnh (.lib) hay tệp thu viện động (.dll). Thành phần nhị phân được sử dụng dể liên kết, hoặc để thực thi chương trình
  - Thành phần thực thi là tệp chương trình có thể thực thi được (các tệp .exe).

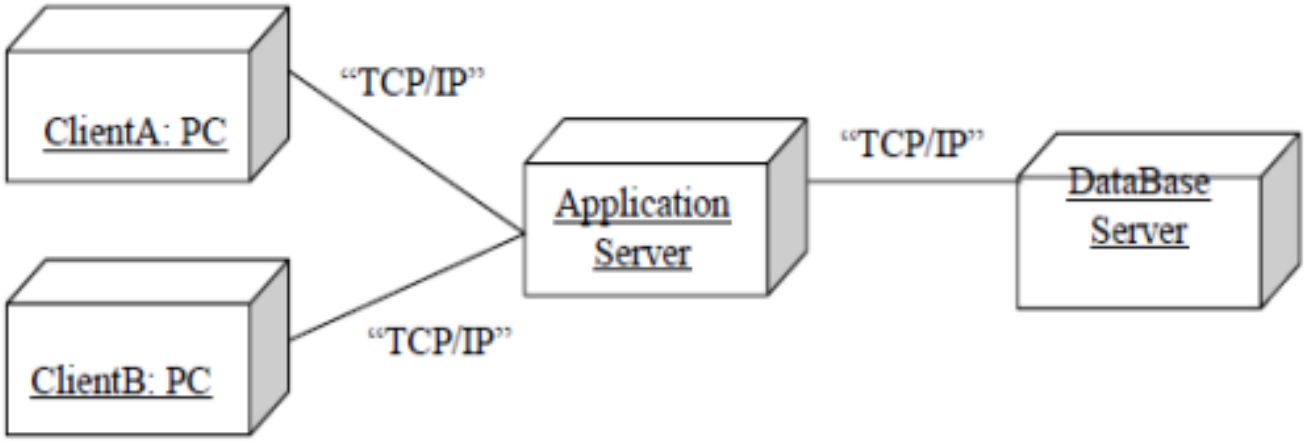


Sự phụ thuộc của các thành phần trong biểu đồ thành phần

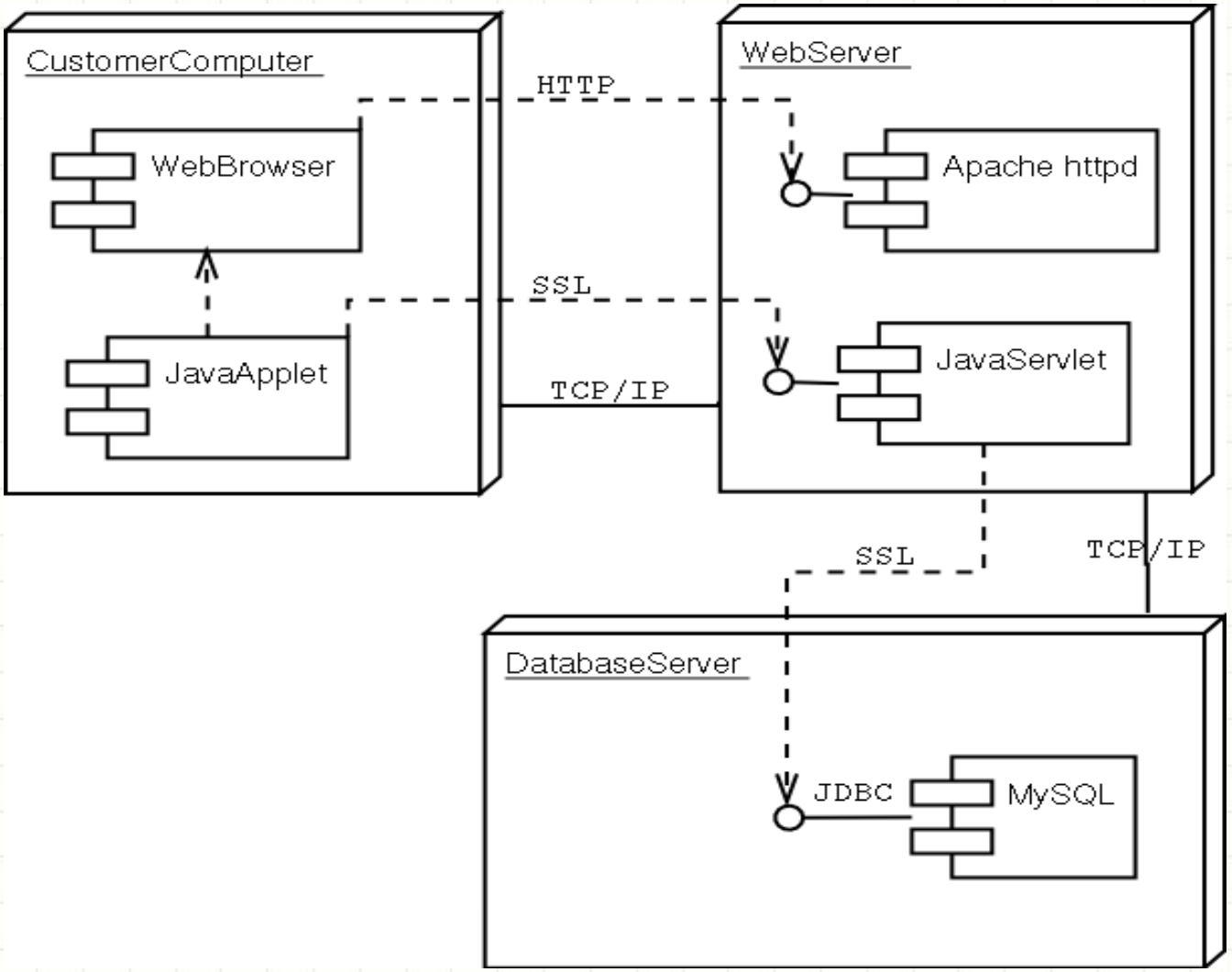
3.5.Biểu đồ triển khai

- Biểu diễn quan hệ vật lý giữa thành phần phần cứng và phần mềm như khi hệ thống đang thực hiện
- Biểu đồ triển khai có thể được sử dụng để chỉ ra thành phần nào có thể chạy trong node nào





Biểu đồ triển khai của hệ thống


CustomerComputer
WebBrowser
JavaApplet
HTTP
SSL
TCP/IP
WebServer
Apache httpd
JavaServlet
SSL
TCP/IP
DatabaseServer
JDBC
MySQL